INFO FACULTATEA DE INFORMATICĂ

WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: INTELLIGENT SOFTWARE ROBOTICS

MASTER THESIS

SUPERVISOR:
Lect. Dr. Todor Ivașcu

GRADUATE:
Raluca-Bianca Biraș

TIMIŞOARA
2026

WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: INTELLIGENT SOFTWARE ROBOTICS

AffectLab: Design and Evaluation of an Emotion-Aware Multimodal System for Adaptive Difficult-Conversation Rehearsal

SUPERVISOR:
Lect. Dr. Todor Ivașcu

GRADUATE:
Raluca-Bianca Biraș

TIMIŞOARA
2026

Abstract

Practising difficult interpersonal conversations can be inaccessible, inconsistent, or uncomfortable, while conventional conversational systems often ignore context, vocal cues, and uncertainty. This dissertation presents AffectLab, a non-clinical application for adaptive communication rehearsal that combines contextual text and speech emotion recognition with deterministic dialogue, feedback, safety, and fallback policies. Emotion models were evaluated on five speaker-independent IEMOCAP folds using classification, calibration, and dialogue-level bootstrap measures; the integrated application was assessed through authored cases, automated and full-stack tests, accessibility checks, restart recovery, and local latency measurements. Three-turn context increased text macro F1 from 0.6651 to 0.7332. Equal-weight text–audio fusion reached 0.7883 macro F1 and 0.7832 accuracy, improving macro F1 over contextual text by 0.0551 (95% interval 0.0407–0.0713), although its ECE of 0.1619 exposed a calibration trade-off. The application passed 240 backend tests, production frontend checks, desktop and mobile workflows, and two database-backed restart journeys; deterministic conversation turns had a 5.836 ms 95th-percentile local latency. In a supplementary single-arm pilot, six of ten enrollees completed the protocol, while realism and feedback usefulness each had a median rating of 5/7. The small, uncontrolled, incomplete pilot cannot establish causal benefit or real-world transfer. Within these limits, AffectLab demonstrates a reproducibly evaluated, uncertainty-aware system suitable for controlled deployment and further study.

Keywords: affective computing; emotion recognition; multimodal fusion; conversational context; difficult-conversation rehearsal; human–computer interaction.

Statement of Scientific integrity

I declare that this MSc thesis is an original report and represents my own work, conducted under supervision of Lect. Dr. Todor Ivașcu. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

I used generative AI during the writing and research process. The details are in the Appendix A.

Contents

1 Introduction **11**
1.1 Context and Motivation 11
1.2 Problem Statement 14
1.3 Aim of the Dissertation 17
1.4 Objectives 17
1.5 Research Questions 19
1.6 Proposed Approach 20
1.6.1 Multimodal Emotion Recognition 20
1.6.2 Structured Conversational Role-Play 21
1.6.3 Uncertainty-Aware Affect Adaptation 21
1.6.4 Evidence-Grounded Feedback and Reflection 22
1.6.5 System Verification and Supporting User Feedback 22
1.7 Expected Contributions 23
1.8 Scope and Limitations 25
1.8.1 System Scope 25
1.8.2 Machine-Learning Scope 26
1.8.3 Role-Play and Feedback Scope 26
1.8.4 Dataset and Generalisation Limitations 27
1.8.5 Uncertainty and Calibration Limitations 27
1.8.6 Generative Artificial Intelligence Limitations 28
1.8.7 Safety and Clinical Limitations 28
1.8.8 Privacy and Security Limitations 29
1.8.9 Evaluation Limitations 29
1.8.10 Delimitations 30
1.9 Dissertation Structure 30

2 State of the Art **33**
2.1 Introduction 33
2.2 Role-Play and Simulation for Communication-Skills Development . 33
2.2.1 Role-play as a practice-based learning method 33
2.2.2 Simulation realism, repeatability, and transfer 34
2.3 Feedback and Reflection 35
2.3.1 From conversational experience to actionable feedback . . . 35
2.3.2 Reviewing alternatives instead of rewarding one script 36
2.4 Conversational Agents for Structured and Generative Role-Play . . 36
2.4.1 Deterministic dialogue management 36
2.4.2 Language-model simulation and its limits 36
2.5 Affective Computing and Emotion Labels 37

- 2.5.1 Categorical and dimensional representations . . . 37
- 2.5.2 Expression, interpretation, and uncertainty . . . 38
- 2.6 Emotion-Recognition Datasets and Protocols . . . 38
  - 2.6.1 IEMOCAP and the four-class benchmark . . . 38
  - 2.6.2 Other datasets and domain differences . . . 39
  - 2.6.3 Evaluation splits and information leakage . . . 39
- 2.7 Textual Emotion Recognition and Conversational Context . . . 40
  - 2.7.1 Pre-trained language representations . . . 40
  - 2.7.2 Speaker and dialogue-context modelling . . . 40
  - 2.7.3 Causal context and deployment compatibility . . . 41
- 2.8 Speech Emotion Recognition . . . 41
  - 2.8.1 Acoustic cues and conventional representations . . . 41
  - 2.8.2 Self-supervised speech representations . . . 42
  - 2.8.3 Operational constraints of speech input . . . 42
- 2.9 Multimodal Emotion Recognition and Fusion . . . 42
  - 2.9.1 Complementarity of text and speech . . . 42
  - 2.9.2 Decision-level fusion and its trade-offs . . . 43
- 2.10 Confidence Calibration, Uncertainty, and Generalisation . . . 44
  - 2.10.1 Accuracy does not establish trustworthy probabilities . . . 44
  - 2.10.2 Evaluating classification and calibration separately . . . 44
  - 2.10.3 Distribution shift and restrained use of predictions . . . 45
- 2.11 Affective and Empathetic Conversational Agents . . . 45
  - 2.11.1 Generating emotion-sensitive responses . . . 45
  - 2.11.2 Bounded adaptation rather than emotion-driven control . . . 45
- 2.12 Trust, Safety, Privacy, and Human Oversight . . . 46
  - 2.12.1 Boundaries between rehearsal and psychological support . . . 46
  - 2.12.2 Data minimisation and legal context . . . 46
- 2.13 Model and Application Evaluation . . . 47
- 2.14 Synthesis of Existing Research . . . 47
- 2.15 Research Gap and Positioning of AffectLab . . . 49
- 2.16 Chapter Summary . . . 50

3 Proposed System Architecture 51

- 3.1 System Overview . . . 51
- 3.2 System Requirements and Scope . . . 54
  - 3.2.1 Overview . . . 54
  - 3.2.2 Scope of the System . . . 55
  - 3.2.3 Functional Requirements . . . 56
  - 3.2.4 Non-Functional Requirements . . . 59
  - 3.2.5 Assumptions . . . 60
  - 3.2.6 System Boundaries . . . 61
  - 3.2.7 Summary . . . 61
- 3.3 Ethical Considerations and Limitations . . . 63
  - 3.3.1 Ethical Approach and Intended Use . . . 63
  - 3.3.2 Ethical Use of Affect Recognition . . . 63
  - 3.3.3 Ethical Limits of Cognitive-Pattern and Intent Heuristics . . . 64
  - 3.3.4 User Autonomy, Transparency, and Avoidance of Dependency 65
  - 3.3.5 Safety and Non-Clinical Boundaries . . . 65

3.3.6 Privacy, Data Protection, and Data Governance 66
3.3.7 Fairness, Representation, and Accessibility 67
3.3.8 Generative Artificial Intelligence and Feedback Integrity . . 68
3.3.9 Research Ethics and Participant Protection 68
3.3.10 Legal and Regulatory Context 69
3.3.11 Limitations of the Proposed System 70
3.3.12 Summary 71

4 Implementation 73
4.1 Chapter Overview 73
4.2 Technology Stack and Project Organisation 74
4.2.1 Frontend Technologies 74
4.2.2 Backend Technologies 75
4.2.3 Persistence and External Services 76
4.2.4 Repository Structure and Configuration 77
4.3 Emotion-Recognition Pipeline 78
4.3.1 Dataset Preparation and Label Mapping 78
4.3.2 Contextual Text-Emotion Model 80
4.3.3 Speech-Emotion Model 81
4.3.4 Multimodal Late Fusion 81
4.3.5 Confidence Calibration and Uncertainty Handling 82
4.3.6 Model Artefacts, Provenance, and Inference Loading 83
4.4 Backend Application 84
4.4.1 Application and Service Architecture 84
4.4.2 Domain Models and API Schemas 85
4.4.3 Authentication and Account Management 86
4.4.4 Session Persistence and Ownership Controls 86
4.4.5 Conversation Request Processing 87
4.4.6 Optional Services and Deterministic Fallbacks 88
4.4.7 Rule-Based Cognitive Features and Support Routing 89
4.5 Role-Play and Adaptation Engine 90
4.5.1 Scenario Representation and Configuration 90
4.5.2 Dialogue State and Transition Logic 91
4.5.3 Communication-Evidence Extraction 91
4.5.4 Affect-Aware Pacing Policy 93
4.5.5 Response Realisation 94
4.5.6 Safety Precedence and Crisis Interruption 94
4.6 Feedback and Reflection Features 95
4.6.1 Evidence-Grounded Feedback Generation 95
4.6.2 Conversation Replay 95
4.6.3 Rewind, Branching, and Comparison 96
4.6.4 Takeaways and Action Cards 96
4.7 Frontend Application 98
4.7.1 Application Interface Overview 98
4.7.2 Application Navigation and User Workflow 98
4.7.3 Conversation Preparation 102
4.7.4 Role-Play Workspace 102
4.7.5 Voice Capture and Multimodal Interaction 103

4.7.6 Presentation of Affect and Uncertainty . . . 103
4.7.7 Feedback, Replay, and Comparison Interfaces . . . 104
4.7.8 Optional Research Interfaces . . . 104
4.8 Optional Research Support and Data Export . . . 106
4.8.1 Participant Enrolment and Consent . . . 106
4.8.2 Study Progress and Questionnaires . . . 106
4.8.3 Pseudonymous Research Records . . . 107
4.8.4 Research Dashboard and Data Export . . . 107
4.9 Security, Privacy, and Data Protection . . . 108
4.9.1 Authentication and Token Management . . . 108
4.9.2 Authorisation and Resource Isolation . . . 109
4.9.3 Audio and Conversation Data Handling . . . 109
4.9.4 Retention, Withdrawal, and Account Deletion . . . 110
4.9.5 Deployment Security Controls . . . 110
4.10 Deployment and Configuration . . . 111
4.10.1 Local and Containerised Deployment . . . 111
4.10.2 Offline Demonstration Mode . . . 112
4.10.3 Environment-Based Configuration . . . 112
4.10.4 Health Checks and Operational Failure Handling . . . 112
4.11 Implementation Verification . . . 113
4.11.1 Backend Unit and Integration Testing . . . 113
4.11.2 Frontend End-to-End Testing . . . 114
4.11.3 Performance Regression Testing . . . 114
4.11.4 Full-Stack Smoke Testing . . . 114
4.11.5 Reproducibility and Model-Provenance Checks . . . 115
4.12 Chapter Summary . . . 116

5 Evaluation and Results **117**
5.1 Chapter Overview . . . 117
5.2 Evaluation Scope and Evidence Hierarchy . . . 117
5.2.1 Research Questions and Evaluation Levels . . . 117
5.2.2 Separation of Development, Evaluation, and Deployment Evidence . . . 118
5.3 Emotion-Recognition Evaluation Methodology . . . 119
5.3.1 Benchmark Task and Speaker-Independent Splits . . . 119
5.3.2 Classification and Calibration Measures . . . 119
5.3.3 Paired Dialogue-Level Statistical Comparison . . . 120
5.4 Contextual Text-Emotion Results . . . 120
5.4.1 Primary Four-Class Comparison . . . 120
5.4.2 Exploratory Six-Class Comparison . . . 120
5.4.3 Answer to RQ1 . . . 121
5.5 Speech and Multimodal Results . . . 121
5.5.1 Audio-Only Baseline . . . 121
5.5.2 Predeclared Equal-Weight Late Fusion . . . 121
5.5.3 Classification–Calibration Trade-Off . . . 122
5.5.4 Answer to RQ2 . . . 126
5.6 Application Verification and Readiness . . . 126
5.6.1 Deterministic Dialogue Cases . . . 126

5.6.2 Cognitive-Heuristic and Support-Routing Cases 126
5.6.3 Evidence Extraction and Affect-Pacing Cases 129
5.6.4 Automated and Full-Stack Verification Results 129
5.6.5 Performance and Responsive-Interface Checks 130
5.6.6 Answer to RQ3 130
5.6.7 Answer to RQ4: Integrated Application Quality 131
5.6.8 Supplementary User-Pilot Results 131
5.7 Threats to Validity and Limitations 132
5.7.1 Model and Statistical Validity 132
5.7.2 External and Construct Validity 133
5.7.3 Supplementary User-Evidence Limitations 133
5.8 Chapter Summary 134

6 Conclusions and Future Work 135
6.1 Chapter Overview 135
6.2 Summary of the Dissertation 135
6.3 Conclusions for the Research Questions 136
6.3.1 RQ1: The Contribution of Conversational Context 136
6.3.2 RQ2: Multimodal Complementarity and Calibration 137
6.3.3 RQ3: Bounded Integration of Affective Evidence 137
6.3.4 RQ4: Integrated Application Quality and Readiness 138
6.4 Principal Contributions 138
6.4.1 Empirical and Machine-Learning Contributions 138
6.4.2 System and Interaction-Design Contributions 138
6.4.3 Application-Verification and Infrastructure Contributions . . 139
6.5 Limitations of the Conclusions 139
6.5.1 Model and Data Limitations 139
6.5.2 Application and Measurement Limitations 140
6.5.3 Supporting User-Evidence Limitations 140
6.6 Future Work 141
6.6.1 Validate Deployment Performance and Reliability 141
6.6.2 Broaden Model Validation and Improve Uncertainty Handling 142
6.6.3 Evaluate and Refine the Adaptation Policy 142
6.6.4 Validate Feedback and Communication Measures 142
6.6.5 Study Transfer, Accessibility, and Long-Term Use 143
6.6.6 Strengthen Deployment, Privacy, and Governance 143
6.7 Final Conclusion 144

Bibliography 149

A Generative AI Tools Usage 151
A.1 ChatGPT Details 151
A.1.1 Literature Exploration 152
A.1.2 Academic Writing Assistance 152
A.1.3 LaTeX Formatting Assistance 152
A.1.4 Technical Guidance and Debugging 152

Chapter 1

Introduction

1.1 Context and Motivation

Interpersonal communication is central to everyday life. People routinely need to negotiate workloads, communicate personal needs, establish boundaries, respond to disagreement, and discuss sensitive subjects with colleagues, managers, friends, or family members. Although these conversations are common, they can be difficult to navigate when the outcome matters or when strong emotions are involved. A person may know what they wish to communicate but struggle to express it clearly, maintain an appropriate tone, or respond constructively to resistance. Unsuccessful communication may contribute to unresolved conflict, damaged relationships, avoidable stress, and a reduced willingness to address similar situations in the future.

The broader importance of interpersonal relationships is reinforced by growing attention to social connection and relational well-being. The World Health Organization identifies social connection as an important contributor to individual and societal well-being and reports that social isolation and loneliness are widespread problems with consequences for both physical and mental health [Wor25]. Technology alone cannot resolve these complex social problems. Nevertheless, accessible tools that help people prepare for and practise challenging interactions may provide one way of supporting more deliberate and confident communication.

Communication is a practical skill and therefore cannot be developed through theoretical instruction alone. Role-play and simulation provide opportunities to practise language, experience conversational resistance, make mistakes, and consider alternative responses without facing the full consequences of a real interaction. Evidence from communication education suggests that role-play can improve communication performance and self-efficacy. For example, Bosse et al. found that both peer role-play and training with standardised patients improved medical students' communication performance and self-efficacy compared with a control condition [BSN+12]. Similarly, a systematic review of virtual-patient simulators concluded that appropriately designed simulations can provide a safe and comparatively affordable environment for communication-skills training [LKK+20].

Much of the existing evidence originates from healthcare education and cannot automatically be generalised to ordinary interpersonal conversations. Nevertheless, it supports the underlying principle that realistic and repeatable simulation can be a useful method of practising communication. The systematic review by Lee et al.

also found that effective instructional design was more important than technological sophistication alone [LKK+20]. This indicates that a useful digital rehearsal system requires explicit learning objectives, structured activities, opportunities for reflection, and meaningful feedback rather than merely a technically advanced conversational interface.

Conventional role-play has several practical limitations. It may require another person, a trained facilitator, scheduled sessions, or a participant who is comfortable practising in front of others. These requirements can restrict privacy, repeatability, consistency, and accessibility. A computer-based conversational system can instead make rehearsal available on demand, allow the same interaction to be repeated, and give users an opportunity to explore alternative wording from the same conversational point.

Recent research has begun to examine the use of language models for interpersonal-skills training. Lin et al. developed IMBUE, an interactive system that uses language models to simulate personalised communication situations and provide just-in-time feedback [LSR+24]. In a randomised study, the system demonstrated improvements in measures including skill mastery and self-efficacy, although the transfer of outcomes varied between situation-specific and novel scenarios [LSR+24]. This work illustrates the potential of language-model-based role-play while also demonstrating that conversational generation should be situated within a structured learning framework.

Large language models can produce flexible and natural dialogue, but unrestricted generation is not sufficient for a dependable rehearsal system. A general-purpose model may be inconsistent, overly agreeable, repetitive, or unable to preserve the intended role and difficulty of a scenario. It may also generate feedback that sounds persuasive without being grounded in what the user actually communicated. These problems are particularly important in sensitive conversations, where an apparently empathic response could be mistaken for reliable psychological understanding.

Consequently, an effective practice system requires a balance between generative flexibility and deterministic control. Scenario stages, success conditions, safety rules, evidence collection, and feedback metrics should remain governed by explicit application logic. A generative model may then be used to increase linguistic variety and improve the naturalness of responses without becoming the authoritative source of safety or assessment decisions. This requirement motivates the hybrid architecture adopted in AffectLab.

A further limitation of many conversational systems is that they respond primarily to the literal content of the most recent message. Human communication, however, conveys information through several channels. Word choice communicates semantic meaning, while vocal characteristics such as intonation, rhythm, energy, and emphasis may provide additional affective cues. Emotional interpretation also depends on conversational context. An isolated sentence can be ambiguous, sarcastic, or misleading when separated from the preceding exchange. Research in affective computing therefore increasingly treats emotion recognition as a contextual and multimodal task rather than solely as the classification of isolated utterances.

The Interactive Emotional Dyadic Motion Capture database, commonly known as IEMOCAP, was created to support research into multimodal and expressive human communication [BBL+08]. It contains approximately twelve hours of scripted and improvised dyadic interactions recorded from ten actors. The dataset includes speech, text transcriptions, visual information, and annotations describing emotional expression [BBL+08]. Its design reflects the understanding that expressive communication is conveyed through interacting verbal and non-verbal channels. IEMOCAP has consequently become a widely used benchmark for studying emotion recognition from text, speech, and multimodal data.

Using both text and audio offers a potential advantage because the two modalities provide different but complementary information. Text represents the explicit semantic content of an utterance, whereas audio can represent aspects of delivery that are not preserved in a transcript. Context from preceding dialogue can additionally help distinguish the intended meaning of an utterance within an unfolding interaction. Research on context-aware multimodal emotion recognition has demonstrated the relevance of combining conversational context with audio and textual information [LWC+22]. These findings motivated AffectLab's use of a context-aware text model, an audio model, and a late-fusion mechanism for combining their predictions.

However, automatic emotion recognition must be treated with considerable caution. Emotion is complex, dynamic, culturally influenced, and not always externally observable. A model trained to classify a limited set of benchmark categories cannot determine a person's true internal emotional state. Furthermore, IEMOCAP is a relatively small dataset composed of acted and improvised English-language interactions recorded under controlled conditions [BBL+08]. Performance measured on this dataset does not establish reliable operation across different cultures, accents, age groups, disabilities, recording environments, conversational settings, or spontaneous emotional experiences.

Models can also produce incorrect predictions with high confidence, particularly when they are applied to data that differ from their training distribution. Consequently, calibrated confidence and explicit uncertainty handling are important when affective predictions are incorporated into an interactive application. An emotion-aware system should not present every prediction as a fact or allow a low-confidence classification to determine a consequential decision. Instead, affective predictions can be treated as bounded contextual signals that inform modest and reversible interaction choices.

In AffectLab, uncertainty, disagreement between the textual and audio models, or unavailable inference can result in a neutral response or an explicit pacing choice rather than a psychological claim. Crisis and safety decisions remain separate from the trained affect model, and users retain the ability to pause, continue, or end a rehearsal. The system is therefore described as *emotion-aware* rather than emotionally intelligent: it uses modelled affective cues as limited contextual information but does not claim to understand the user's internal experience.

These considerations motivated the development of AffectLab, a production-oriented application for preparing for and rehearsing difficult interpersonal conversations. The system combines structured role-play scenarios with contextual text analysis, optional speech information, uncertainty-aware adaptation, deterministic communication-skill feedback, conversation replay, and branch-and-compare functionality. Its scenarios address communication tasks such as discussing workload, maintaining a personal boundary, and expressing a relationship need without blame. Users can repeat complete attempts, retry an individual exchange, compare alternative responses, and review the conversational evidence used to produce feedback.

The principal motivation for AffectLab is not to automate human relationships or provide psychological treatment. Rather, it is to deliver a helpful, accessible environment for reflective communication practice while investigating how multimodal affective computing and controlled conversational artificial intelligence can be integrated responsibly. The project addresses both a technical and a human-computer interaction problem: how to combine context-aware emotion recognition with an adaptive role-play experience while preserving transparency, user control, privacy, safety, and appropriate limits on what the system claims to know. Accordingly, the system is evaluated primarily through model performance and thorough application verification, with a small user pilot retained as supplementary feedback rather than as evidence of diagnosis, therapeutic effectiveness, or access to a person's private emotional state.

1.2 Problem Statement

Digital conversational systems provide a potentially accessible method for practising difficult interpersonal conversations. They can offer repeatable scenarios, immediate responses, and private access without requiring another participant or a trained facilitator. Previous research has shown that role-play and virtual simulation can support communication-skills training [BSN+12, LKK+20], while more recent work has demonstrated the potential of language models to generate personalised interpersonal-skills simulations and feedback [LSR+24]. However, transforming these capabilities into a coherent and responsible communication-practice system presents several interconnected problems.

The first problem concerns the tension between conversational flexibility and behavioural control. A fully scripted system can provide predictable progression and reproducible assessment, but its responses may appear repetitive or fail to accommodate the variety of language that users employ. Conversely, a system based entirely on a large language model may generate more natural dialogue but can behave inconsistently, abandon the assigned role, alter the intended difficulty of a scenario, or provide feedback that is not supported by observable evidence. Generative flexibility is therefore valuable for producing varied dialogue, but it is insufficient as the sole mechanism for controlling the rehearsal.

A useful role-play system must preserve a stable relationship between the user's communication, the simulated character's response, the progression of the scenario, and the feedback presented at the end of the interaction. For example, a workload conversation should not be considered successfully completed merely because the user has made an initial request. The simulated conversation may also need to explore constraints, respond to resistance, and establish whether a practical next step has been agreed. Similarly, success in a boundary-setting scenario should not depend on persuading the other character to agree. A user may successfully express and maintain a boundary even if the simulated character remains dissatisfied. These distinctions require scenario-specific dialogue rules and explicit definitions of conversational outcomes.

The second problem concerns the use of affective information during an interaction. Conventional conversational systems generally respond to the semantic content of the current message but may not account for affective cues conveyed through vocal delivery or previous turns. Research into context-aware multimodal emotion recognition suggests that combining dialogue context, textual information, and audio features can improve performance on benchmark emotion-classification tasks [LWC$^+$22]. The IEMOCAP corpus provides a widely used basis for investigating this problem because it contains annotated, multimodal dyadic interactions [BBL$^+$08].

Nevertheless, benchmark emotion recognition and responsible application behaviour are separate problems. A model can produce a statistically valid classification within a predefined benchmark while still being unsuitable for making strong claims about an individual user. The emotional categories available in IEMOCAP represent a simplified classification task and do not encompass the full range, ambiguity, or cultural variability of human emotional experience [BBL$^+$08]. Furthermore, predictions can be uncertain, and textual and acoustic models may disagree. A system must therefore determine not only how to generate an affective prediction, but also when and how that prediction may appropriately influence the interaction.

Directly presenting an inferred emotion as a fact could misrepresent the user's internal state. Similarly, automatically increasing or reducing scenario difficulty based on a single uncertain prediction could remove user control and produce an inappropriate interaction. The technical problem is consequently one of bounded adaptation: affective predictions should inform only limited, transparent, and reversible conversational actions. Low confidence, disagreement between modalities, unavailable inference, or explicit user preferences must be handled without interrupting the core role-play workflow.

The third problem relates to feedback and explainability. Communication feedback is useful only when users can understand how it was produced and relate it to their own language. Feedback generated exclusively by a large language model may be fluent but can introduce observations that are not supported by the conversation. A purely numerical score, on the other hand, may provide insufficient guidance about what the user did effectively or what could be changed. The system must therefore connect feedback metrics to explicit and observable features, such as whether the user made a clear request, provided a specific detail, used non-blaming language, maintained a boundary, or proposed a practical next step.

This creates a requirement for evidence-grounded feedback. The authoritative assessment should be produced from deterministic and inspectable communication features, while a language model may optionally improve the wording without inventing new evidence. Users should also be able to review the relevant turns, retry an exchange, and compare alternative responses. This is consistent with findings that effective simulation depends on instructional design, reflection, and feedback rather than technological sophistication alone [LKK$^+$20].

The fourth problem concerns safety, privacy, and appropriate scope. Difficult conversations may involve distress, conflict, or crisis-related language. An adaptive rehearsal system must recognise that it is not a therapist, diagnostic instrument, or emergency service. Safety handling cannot depend on a probabilistic emotion classifier because such a model may fail, be unavailable, or misclassify an utterance. Crisis detection and safety interruption must therefore remain independent of optional affective inference and generative dialogue. Users must also retain the ability to pause, finish, or leave a scenario without being penalised.

Audio introduces additional privacy considerations because a voice recording can contain sensitive and identifying information. A multimodal system should minimise audio collection, require an explicit user action, limit recording duration, and avoid retaining the recording when persistence is unnecessary. The text-based workflow must remain operational when microphone access is denied, when the user chooses not to provide audio, or when the trained models are unavailable. Multimodal processing should therefore enhance the experience without becoming a requirement for participation.

The fifth problem is evaluation. A complete system combines machine-learning models, deterministic dialogue policies, optional generative services, safety controls, persistent data, and a user interface. Evaluating only the emotion-classification model would not demonstrate that the application behaves correctly. Conversely, a usability demonstration would not establish the validity of the machine-learning component. The different contributions require separate forms of evidence.

The emotion-recognition models must be evaluated using speaker-independent data partitions, appropriate class-sensitive metrics, and calibration measures. Comparisons between text-only, audio-only, contextual, and fused systems must use consistent held-out examples. The application must be verified through reproducible scenarios and automated tests covering dialogue transitions, uncertainty handling, fallback behaviour, evidence detection, access control, replay, branching, persistence, recovery, accessibility, responsive layouts, and local response time. A small user pilot can provide additional product feedback, but it must remain distinct from claims about technical conformance or real-world behavioural change.

Accordingly, the central problem addressed by this dissertation can be stated as follows:

> *How can contextual text and audio-based affect recognition be integrated into a structured conversational role-play system that provides adaptive, evidence-grounded communication practice while preserving predictable dialogue behaviour, calibrated uncertainty, user control, privacy, safety, and transparent evaluation?*

To address this problem, AffectLab adopts a hybrid design. Deterministic policies control scenario progression, safety decisions, observable-skill assessment, and the permitted effects of affective predictions. Trained models provide contextual text, audio, and fused emotion distributions as exploratory signals. Generative artificial intelligence is used only within constrained boundaries and is accompanied by a deterministic fallback. The interface supports repeated practice, conversation replay, alternative branches, evidence-linked feedback, and user-authored takeaways.

The resulting research problem is therefore not limited to improving the accuracy of an emotion classifier or producing realistic generated dialogue. It concerns the integration of machine learning, controlled conversational behaviour, uncertainty management, and human-centred safeguards into a coherent, deployment-ready application. The aim is to establish whether these components can operate together as a helpful, technically reproducible, robust, and responsive system without overstating its ability to interpret emotion or improve users' real-world relationships.

1.3 Aim of the Dissertation

The aim of this dissertation is to design, implement, and evaluate AffectLab, an emotion-aware conversational system for rehearsing difficult interpersonal conversations. The system integrates structured role-play with contextual text and audio-based emotion recognition, uncertainty-aware adaptation, and evidence-grounded feedback.

The dissertation investigates how affective machine-learning models and generative artificial intelligence can be incorporated into a communication-practice application while maintaining predictable behaviour, transparency, privacy, safety, and user control. Rather than treating model predictions as direct measurements of a person's internal emotional state, AffectLab uses them as uncertain and bounded contextual signals that may support modest adaptations to the interaction.

The work also aims to bridge the gap between emotion-recognition research and its responsible use within an interactive application. Consequently, the dissertation considers not only the predictive performance and calibration of the trained models, but also how their outputs influence role-play dialogue, how uncertainty and inference failures are handled, how feedback is connected to observable evidence, and how the complete system behaves under realistic usage conditions.

The purpose of AffectLab is not to provide diagnosis, therapy, crisis care, or an objective assessment of personal competence. Its intended role is to provide a controlled and repeatable environment in which users can prepare for, practise, review, and compare different approaches to challenging conversations. Accordingly, the dissertation evaluates it as a complete communication-rehearsal application suitable for controlled production deployment, while recognising that successful interaction with the application does not by itself establish improved real-world communication.

Overall, the dissertation aims to demonstrate a technically reproducible and human-centred approach to emotion-aware role-play in which trained models, deterministic dialogue policies, optional generative components, and safety mechanisms have clearly defined and appropriately limited responsibilities.

1.4 Objectives

To achieve the aim of the dissertation, the following objectives were defined:

1. To review relevant research concerning communication-skills rehearsal, conversational agents, affective computing, emotion recognition in conversation, multimodal learning, and responsible human–AI interaction.

2. To prepare and document an experimental workflow based on the IEMOCAP dataset, using speaker-independent partitions to reduce the risk of evaluating the models on speakers observed during training.

3. To develop an utterance-level textual emotion-recognition baseline and a context-aware text model that incorporates preceding conversational turns.

4. To evaluate whether conversational context improves textual emotion recognition compared with the utterance-only baseline.

5. To develop an audio-based emotion-recognition model capable of extracting affective information from speech while operating independently of the textual model.
6. To combine the context-aware textual and audio models through late fusion and determine whether the multimodal system improves classification performance over the individual modalities.
7. To calibrate the textual, audio, and fused predictions so that model confidence can be interpreted more cautiously when used by the application.
8. To design and implement structured role-play scenarios for difficult interpersonal conversations, including workload negotiation, personal boundary-setting, and the expression of relationship needs.
9. To define explicit dialogue stages, character profiles, difficulty levels, completion conditions, and unresolved outcomes so that scenario progression remains predictable and reproducible.
10. To develop a bounded adaptation policy that determines whether affective predictions may influence the interaction based on confidence, agreement between modalities, model availability, user preferences, and safety conditions.
11. To ensure that trained emotion-recognition models do not determine crisis handling, scenario success, feedback scores, participant eligibility, or other consequential application decisions.
12. To produce communication feedback from deterministic and observable language features, with each metric linked to evidence from specific conversational turns.
13. To support reflective and repeated practice through conversation replay, retrying individual exchanges, alternative conversation branches, comparison between attempts, personal scenario preparation, and action-card creation.
14. To implement appropriate security, privacy, and research-data controls, including authentication, ownership-protected sessions, optional and transient voice processing, pseudonymous research records, withdrawal support, and data-retention mechanisms.
15. To provide deterministic fallbacks for optional external services so that the core text-based workflow remains operational when generative, transcription, or multimodal inference services are unavailable.
16. To evaluate the machine-learning components using accuracy, macro-averaged F1, weighted F1, per-class measures, calibration metrics, and paired statistical comparisons where appropriate.
17. To evaluate the application through automated unit, integration, browser, persistence, recovery, accessibility, responsive-layout, performance, and end-to-end tests, together with reproducible authored engineering cases.

18. To report a small user pilot as supplementary product feedback on completion, perceived realism, feedback usefulness, and self-reported confidence without making therapeutic or causal-effectiveness claims.

19. To document the limitations of the dataset, trained models, dialogue policies, evaluation procedures, and intended use so that the conclusions remain proportionate to the evidence collected.

1.5 Research Questions

This dissertation examines both the performance of the multimodal emotion-recognition models and their responsible integration into an adaptive communication-practice application. It also considers the quality, robustness, responsiveness, and deployment readiness of the complete system as a tool for rehearsing difficult interpersonal conversations. The dissertation is therefore guided by the following research questions:

RQ1. Does incorporating preceding conversational turns improve text-based emotion recognition compared with classifying each utterance independently?

This question investigates whether a context-aware textual model can represent the development of emotion across a conversation more effectively than an utterance-only baseline. The comparison is based on speaker-independent evaluation using the same held-out IEMOCAP sessions.

RQ2. Does combining contextual textual information with acoustic information improve emotion-classification performance compared with the individual text and audio modalities, and what trade-off exists between classification performance and confidence calibration?

This question evaluates whether text and speech provide complementary affective information. It also distinguishes between predictive performance and probability calibration, since a fusion method may improve classification while producing confidence estimates that require additional calibration before operational use.

RQ3. How can uncertain multimodal affect predictions be integrated into a structured role-play dialogue policy while keeping the system's behaviour explainable, controllable, safe, and robust to disagreement or inference failure?

This question concerns the integration of affective computing into the application. It examines how confidence, modality agreement, model availability, explicit user preferences, and safety conditions can be translated into bounded conversational actions without allowing an emotion prediction to determine safety decisions, feedback scores, or scenario success.

RQ4. To what extent does the completed AffectLab application satisfy its functional, reliability, recovery, accessibility, responsiveness, and

deployment requirements under reproducible engineering evaluation?

This question concerns the quality of the complete AffectLab workflow. It is answered through authored policy cases, automated backend and frontend tests, real-database integration checks, desktop and mobile browser journeys, accessibility scans, failure and restart recovery, production builds, and scoped response-time measurements. A small user pilot is reported separately as supporting feedback rather than as the principal evidence for application readiness.

1.6 Proposed Approach

To address the research questions, this dissertation proposes AffectLab, a full-stack, production-oriented application that combines multimodal emotion recognition with structured and adaptive conversational role-play. The proposed approach separates emotion inference, dialogue control, response generation, feedback, and safety into distinct components. This separation is intended to preserve predictable application behaviour while still allowing trained models and generative artificial intelligence to contribute to the interaction.

The work is organised into four connected areas: multimodal emotion recognition, structured role-play, uncertainty-aware adaptation, and multi-level evaluation.

1.6.1 Multimodal Emotion Recognition

The first part of the approach concerns the development of models for recognising affective information from text and speech. The models are trained and evaluated using the IEMOCAP dataset, which contains annotated dyadic interactions with textual and acoustic information.

An utterance-only text classifier is established as the initial baseline. A context-aware text model is then developed by combining the current utterance with a fixed number of preceding conversational turns. Comparing these models makes it possible to evaluate whether causal dialogue context improves emotion classification without using future information that would be unavailable during a live interaction.

A separate audio model is trained using the speech signal associated with each utterance. The audio model captures characteristics of vocal delivery that are not represented in the transcript. Textual and acoustic outputs are retained separately so that the contribution, confidence, and possible disagreement of each modality can be inspected.

Late fusion is used to combine the probability distributions produced by the contextual text and audio models. Both a predeclared equal-weight fusion and validation-fitted fusion configurations are considered. Temperature scaling and validation-derived modality weights are used to investigate confidence calibration. This separates the question of which configuration achieves the strongest classification performance from the question of which configuration produces the most reliable confidence estimates for operational use.

The models are evaluated using five speaker-independent folds corresponding to the five IEMOCAP sessions. This prevents the speakers used for testing from appearing in the corresponding training data. Evaluation includes accuracy, macro-averaged F1, weighted F1, per-class measures, expected calibration error, negative log-likelihood, and the multiclass Brier score. Paired dialogue-level bootstrap comparisons are used to estimate the uncertainty surrounding differences between selected model configurations.

1.6.2 Structured Conversational Role-Play

The second part of the approach concerns the design of a structured environment for rehearsing difficult interpersonal conversations. AffectLab provides scenarios involving workload negotiation, personal boundary-setting, and the expression of relationship needs. Each scenario defines an objective, expected communication features, difficulty-dependent behaviour, conversational stages, and explicit completion conditions.

Scenario progression is controlled by deterministic dialogue policies rather than by an unconstrained language model. The policies inspect observable features in the user's message and determine the next dialogue stage, the simulated character's intended action, and whether the scenario has reached a resolved, boundary-held, next-step, unresolved, manually completed, or maximum-turn outcome.

The deterministic policy produces a response plan that constrains the simulated character's role and intended behaviour. An optional generative model may phrase the response more naturally, but it cannot change the recorded dialogue decision or determine whether the user has satisfied a scenario requirement. If generation is unavailable, rejected, or inconsistent with the required role, the system uses a deterministic template response.

The role-play design also includes different difficulty levels and character profiles. These vary the amount of resistance, time pressure, scepticism, or cooperation expressed by the simulated character while preserving the scenario's safety and completion rules. Users remain able to pause, resume, or finish a rehearsal regardless of the selected difficulty.

1.6.3 Uncertainty-Aware Affect Adaptation

The third part of the approach connects multimodal inference to the role-play experience through a bounded and versioned adaptation policy. The purpose of this policy is not to declare how the user feels, but to determine whether the current evidence supports a modest change in conversational pacing.

The policy considers the confidence of the fused prediction, agreement between the textual and acoustic models, the availability of inference, the current scenario state, and any explicit pacing preference selected by the user. Matching anger or sadness predictions may permit a brief acknowledgement when the fused, textual, and acoustic confidences all meet the configured low-confidence boundary, normally 0.55. This includes predictions in either the moderate or high presentation band; the high-band boundary of 0.75 is not an additional policy requirement. A confidence below the configured boundary in any of the three distributions, or disagreement between modalities, results in an offer of user choice rather than an automatic interpretation. Missing or failed inference results in baseline dialogue behaviour.

The adaptation policy does not determine crisis handling, scenario completion, communication-skill scores, participant eligibility, or research outcomes. Crisis-related language is processed through a separate text-first safety mechanism that takes precedence over ordinary generation and affect adaptation. This separation ensures that a trained emotion model is not treated as a safety-critical component.

Voice use is optional. When enabled, a short recording is converted into the required audio format and submitted with the user's text for the current inference request. Raw audio is not required for the text-based workflow and is not retained as part of the conversation record. If microphone access is denied or multimodal inference is unavailable, the user can continue through the standard text-based experience.

1.6.4 Evidence-Grounded Feedback and Reflection

The fourth part of the proposed system concerns feedback and reflective practice. AffectLab evaluates observable communication features rather than inferring personality, general competence, or psychological characteristics. Depending on the scenario, these features may include a clear request, a specific detail, boundary maintenance, non-blaming language, an "I" statement, acknowledgement of another perspective, or a practical next step.

Each feedback metric is connected to the conversational turns in which the relevant evidence was detected. Deterministic metrics remain the authoritative feedback source. A generative model may optionally improve the wording of strengths and suggestions, but it cannot introduce evidence that was not recorded by the application.

After a rehearsal, the user can review the conversation, inspect recorded dialogue and adaptation decisions, retry the most recent exchange, or create an alternative branch from an earlier point. Branch comparison separates copied context from the newly attempted continuation and compares only the relevant evidence. Users can also record a personal takeaway and produce an action card summarising the wording or next step that they wish to remember.

1.6.5 System Verification and Supporting User Feedback

The proposed evaluation follows a multi-level design because the trained models and the complete application support different types of claims.

First, the machine-learning evaluation compares utterance-only text, contextual text, audio-only, and multimodal configurations using held-out speaker-independent data. This addresses the first two research questions concerning conversational context, multimodal fusion, and confidence calibration.

Second, a comprehensive engineering evaluation examines dialogue transitions, scenario outcomes, affect-policy decisions, evidence detection, fallback behaviour, security boundaries, persistence, replay, branching, recovery, accessibility, responsive layouts, and response time. Authored cases test specific expected behaviours and known failure modes. Automated backend, frontend, database, and end-to-end suites, production builds, and full-stack smoke journeys provide complementary evidence that the integrated system behaves consistently.

Third, a reproducible local performance check measures liveness, authenticated scenario retrieval, session creation, role-play start, and a deterministic conversation turn through the HTTP application boundary. Its purpose is to detect regressions in the locally controlled path; deployment-specific concurrent-load and provider latency remain separate operational checks.

Finally, results from a small single-arm user pilot are retained in one supporting subsection. Completion, realism, feedback usefulness, and confidence ratings provide useful product context, but the study is not used to carry the technical-readiness claim or to assert lasting communication improvement.

1.7 Expected Contributions

The expected contribution of this dissertation is a reproducible demonstration of how multimodal affect recognition can be integrated into a structured communication-rehearsal system without allowing uncertain predictions or unconstrained generation to control consequential decisions. This overall contribution is divided into the following technical, methodological, and applied contributions:

1. **An empirical evaluation of conversational context for textual emotion recognition.**

   The dissertation provides a controlled comparison between an utterance-only text classifier and a context-aware model that incorporates preceding conversational turns. The comparison uses identical speaker-independent test partitions and dialogue-level statistical analysis, contributing evidence about the value of causal conversational context in emotion recognition.

2. **A comparative multimodal emotion-recognition pipeline.**

   The work develops text-only, audio-only, and late-fusion configurations using a consistent four-class IEMOCAP benchmark. This makes it possible to examine the complementary contribution of linguistic and acoustic information rather than reporting only the performance of a final combined model.

3. **A distinction between classification performance and operational confidence.**

   The dissertation evaluates both a predeclared equal-weight fusion configuration and validation-fitted calibration configurations. This distinguishes the strongest confirmatory classification result from the model configuration selected for presenting or thresholding confidence within the application. The distinction helps avoid choosing an operational model solely according to its test-set classification score.

4. **An uncertainty-aware affect adaptation policy.**

   AffectLab contributes a versioned policy that translates confidence, modality agreement, inference availability, and user preference into a restricted set of conversational actions. The policy explicitly defines situations in which the system should acknowledge a signal, offer a pacing choice, use baseline behaviour, or defer to safety handling.

5. **A hybrid architecture for controllable generative role-play.**

   The system separates deterministic dialogue decisions from optional generative wording. Scenario progression, outcomes, safety, and assessment remain controlled by inspectable application logic, while generative artificial intelligence is used within constrained boundaries to improve conversational variety. Deterministic fallbacks preserve the core workflow when external generation is unavailable.

6. **Scenario-specific models of difficult conversations.**

   The dissertation defines multi-stage dialogue policies for workload negotiation, personal boundary-setting, and the expression of relationship needs. These policies distinguish different successful and unresolved outcomes, including agreement, practical next steps, respectfully maintained boundaries, and recorded disagreement. This avoids defining success only as compliance by the simulated character.

7. **Evidence-grounded communication feedback.**

   AffectLab links feedback metrics to observable language features and specific conversational turns. This provides a more transparent alternative to ungrounded natural-language assessment and allows users to inspect why a particular strength or suggestion was produced.

8. **Support for counterfactual and reflective practice.**

   The system enables users to replay conversations, retry individual exchanges, create alternative branches, and compare continuations from the same conversational context. This contribution turns a completed dialogue into an inspectable practice record rather than presenting only a final score or summary.

9. **A privacy- and safety-conscious multimodal workflow.**

   The proposed implementation keeps voice input optional and transient, separates affect inference from crisis handling, and maintains the usability of the text-based system when multimodal services fail. It also includes ownership-protected sessions, explicit retention behaviour, research consent, pseudonymous exports, and withdrawal mechanisms.

10. **A reproducible application-verification framework.**

    The dissertation combines speaker-independent model experiments with authored cases, automated unit and integration tests, real-database checks, browser and accessibility suites, full-stack restart and recovery journeys, and a scoped HTTP latency benchmark. This framework supports claims about both the machine-learning components and the complete application's behaviour, while a small user pilot remains clearly supplementary.

11. **An integrated application ready for controlled deployment and future study.**

    AffectLab provides an end-to-end platform containing user workflows, model integration, safety controls, secure account and session management, health checks, containerised production builds, deterministic fallbacks, structured data collection, and reproducible exports. The tested configuration is suitable for controlled production deployment and can support future research into communication rehearsal, affect-aware interaction, model generalisation, and comparative evaluation, subject to deployment-specific security, legal, and operational review.

These contributions are expected to demonstrate that emotion-aware adaptation can be incorporated into a role-play application in a limited and inspectable manner. The contribution does not consist of a claim that AffectLab understands a user's true emotions or that practising with the system necessarily improves real-world relationships. Instead, it lies in the design, implementation, and evaluation of a transparent architecture that connects multimodal affective models to controlled conversational behaviour while preserving uncertainty, user agency, and clearly defined system boundaries.

1.8 Scope and Limitations

This dissertation investigates the design, implementation, and evaluation of AffectLab as a production-oriented application for emotion-aware communication rehearsal. Its scope includes multimodal emotion recognition, structured role-play, uncertainty-aware adaptation, evidence-grounded feedback, reflective practice, deployment controls, and comprehensive developer-led verification. The conclusions are restricted to the tested models, application behaviour, deployment configuration, and small supporting user pilot.

1.8.1 System Scope

AffectLab is intended for adults who wish to rehearse ordinary but potentially difficult interpersonal conversations. The implemented scenarios focus on three categories: discussing workload with a manager, maintaining a personal boundary, and expressing a relationship need without blame. Users may also prepare custom scenarios within the communication features supported by the application.

The system provides text-based role-play with optional voice input. It supports scenario selection, difficulty levels, character profiles, pausing, manual completion, conversation replay, retrying an exchange, branching from an earlier conversational point, comparison between attempts, personal takeaways, and action-card creation. These functions are intended to support preparation and reflection rather than reproduce every aspect of a real interpersonal interaction.

The application includes optional language-model-based response generation, feedback phrasing, moderation, and transcription. These services are placed behind deterministic interfaces and fallbacks so that the core workflow remains available when an external service is disabled or unavailable. The system does not attempt to provide unrestricted open-domain conversation. Its adaptive role-play functions are limited to the scenarios, dialogue stages, character behaviours, and communication features explicitly represented by the application.

Outside active role-play, the application also contains a small rule-based routing aid. It matches surface phrases associated with a requested form of help, a possible explanatory clause, conversational pushback, and ten named cognitive-pattern vocabularies, then uses those matches to choose among general supportive response templates. This subsystem is included in the implementation scope because its labels are visible to the user and can affect a response. It is not part of the trained emotion-recognition contribution and is not evaluated as a psychological assessment. Its tests establish only that declared lexical rules and priority order are implemented consistently; they do not establish construct validity, diagnostic accuracy, therapeutic value, or optimal strategy selection.

1.8.2 Machine-Learning Scope

The machine-learning component focuses on supervised categorical emotion recognition from text and audio. The primary experimental task uses four IEMOCAP classes: anger, happiness, sadness, and neutral. Excitement is merged with happiness in accordance with the selected benchmark configuration. The models do not represent the complete range of human affect and do not identify complex or mixed states such as anxiety, guilt, shame, uncertainty, embarrassment, or frustration as independent operational classes.

The textual model uses the current utterance and a limited number of preceding conversational turns. The context is causal because future utterances are excluded from application inference. The audio model processes short speech recordings, while late fusion combines the textual and acoustic probability distributions. Facial expressions, physiological signals, body posture, gaze, and video are outside the scope of the implemented multimodal system.

Model evaluation is performed using five session-held-out IEMOCAP folds. This reduces direct speaker overlap between training and testing, but it does not constitute cross-dataset or real-world validation. The final full-data models are deployment artifacts trained after the cross-validation design was completed. Their training performance is not treated as independent evaluation evidence.

The trained models are used only as exploratory affective components. Their outputs do not determine participant eligibility, crisis handling, safety interruption, scenario success, feedback scores, or research-study completion. The application can operate without the multimodal models, and optional voice input is not required for participation.

1.8.3 Role-Play and Feedback Scope

AffectLab evaluates observable features of the user's language rather than underlying intentions, personality, emotional health, or general communication ability. Depending on the scenario, the system may identify features such as a clear request, a specific detail, an "I" statement, non-blaming wording, acknowledgement of another perspective, boundary maintenance, or a proposed next step.

These features are detected through deterministic rules. They provide reproducible evidence but cannot capture every valid form of communication. Indirect expressions, paraphrases, sarcasm, reported speech, unusual vocabulary, grammatical errors, and culturally specific communication styles may be interpreted incorrectly. A detected feature may therefore be a false positive, while an appropriate response may fail to activate the relevant rule.

Feedback scores describe only the language observed during a particular attempt. They are not validated measures of personal competence and should not be used to compare individuals, make employment or educational decisions, or assess psychological improvement. The system may help users inspect and revise their wording, but it cannot determine whether a response would be effective in a real relationship or workplace.

The simulated character is necessarily a simplification. Real people may react unpredictably, misunderstand a request, introduce new information, refuse to cooperate, or behave in ways not represented by the dialogue policy. Reaching a successful outcome in AffectLab does not guarantee that the same wording will produce a similar result in a real conversation. Conversely, an unresolved simulated outcome does not imply that the user communicated poorly.

1.8.4 Dataset and Generalisation Limitations

IEMOCAP is a comparatively small dataset recorded from ten actors in controlled dyadic sessions. Its scripted and improvised interactions do not fully represent spontaneous everyday communication. The speakers, recording equipment, acoustic environment, conversational tasks, and annotation procedures differ from those encountered in deployment.

The dataset is English-language and does not establish generalisation across languages, cultures, accents, dialects, age groups, disabilities, or communication styles. Emotional expression and interpretation can vary across individuals and social contexts. Consequently, performance on IEMOCAP cannot be interpreted as evidence that the models will recognise emotion with equivalent accuracy in AffectLab users.

The four-class benchmark imposes a single categorical label even when an utterance may be ambiguous or express several affective states. The ground-truth annotations represent the judgements produced through the dataset's annotation process rather than direct measurements of a speaker's internal experience. A prediction may therefore match a benchmark label without accurately representing how the speaker understood their own emotional state.

Class performance is also uneven. Aggregate accuracy or macro-averaged F1 can conceal weaker recognition for an individual category. Model results must therefore be interpreted together with per-class metrics, confusion patterns, statistical uncertainty, and the limitations of the evaluation corpus.

1.8.5 Uncertainty and Calibration Limitations

Confidence calibration estimates how model probabilities correspond to correctness within the available validation and test distributions. Good calibration on IEMOCAP does not guarantee calibrated behaviour in AffectLab conversations, where language, microphones, recording conditions, and user populations may differ.

The low-confidence threshold used by the interface is a presentation and adaptation policy rather than a scientifically validated boundary between reliable and unreliable emotion inference. Similarly, agreement between the text and audio models does not prove that the resulting prediction is correct. Both models may agree on the same incorrect category.

Late fusion also introduces a trade-off between classification performance and calibration. A configuration that produces the highest classification score may not produce the most reliable probability estimates. The dissertation therefore distinguishes the confirmatory classification configuration from the calibrated operational configuration rather than treating one result as optimal for every purpose.

1.8.6 Generative Artificial Intelligence Limitations

Optional generative services can improve the variety and naturalness of role-play responses, but they remain probabilistic. Generated wording may be repetitive, overly agreeable, culturally inappropriate, inconsistent with the intended character, or factually incorrect. External model behaviour may also change between provider versions.

The application constrains generation through structured prompts, response plans, validation, and deterministic fallbacks. These measures reduce but do not eliminate inappropriate output. A generated response must not be interpreted as professional advice, and the availability of a language model does not make AffectLab a therapist, mediator, counsellor, or authoritative communication expert.

The system may send conversation content to an external provider when the relevant services are configured and the user has accepted the disclosure. This creates a different privacy boundary from fully offline operation. Although the application avoids placing raw prompts and model responses in ordinary application logs, use of an external provider remains subject to the provider's infrastructure and applicable data-handling terms.

1.8.7 Safety and Clinical Limitations

AffectLab is not a medical device, mental-health intervention, diagnostic system, or emergency service. It is not intended to diagnose emotion, anxiety, depression, cognitive distortion, relationship problems, or any other psychological condition. Its affective outputs are model predictions over predefined labels and must not be interpreted as clinical assessments.

In particular, the interface term "possible cognitive pattern" denotes a match to a short, authored phrase list rather than an assessment of the user's beliefs or cognition. The associated intent, cause, readiness, and resistance fields are likewise routing variables computed from the current wording, not findings about a person. A missing match does not show that a pattern is absent, and a match does not show that it is present in a clinical sense.

The application includes a conservative text-based crisis-phrase check and optional provider moderation. These mechanisms cannot detect every expression of risk, particularly when language is indirect, ambiguous, culturally specific, misspelled, or expressed through audio characteristics rather than explicit text. The safety layer therefore reduces specific foreseeable risks but does not provide complete crisis detection or clinical triage.

Safety decisions remain text-first and independent of the multimodal model. This prevents uncertain emotion predictions from overriding explicit crisis-related language, but it also means that distress expressed only through tone of voice may not activate the safety mechanism. Users requiring urgent or professional support must be directed to appropriate human and emergency services.

1.8.8 Privacy and Security Limitations

The system applies authentication, ownership checks, password hashing, session expiry, pseudonymous research records, and restricted researcher exports. Optional audio is processed transiently and is not intentionally retained as part of the conversation record. These controls reduce privacy and security risks but cannot guarantee absolute protection.

AffectLab stores conversation text for the configured retention period because replay, feedback, branching, and session recovery depend on it. Conversation text may contain sensitive personal information entered by the user. Participants must therefore receive clear information about what is collected, why it is required, how long it is retained, and when it may be transmitted to an external service.

The implemented security controls require deployment-specific configuration and independent review before wider public use. Secure hosting, transport encryption, secret management, database backups, dependency monitoring, rate limiting, incident response, and regulatory compliance depend partly on the environment in which the application is deployed and cannot be established solely by the source code.

1.8.9 Evaluation Limitations

The model experiments evaluate classification on IEMOCAP rather than on recordings collected from AffectLab participants. No claim is made that the reported benchmark accuracy represents recognition accuracy during application use. Participant affect predictions cannot be evaluated for correctness without appropriate ground-truth labels and a separately approved validation procedure.

The engineering evaluation uses authored synthetic cases to exercise known dialogue, evidence, adaptation, persistence, and fallback behaviours. These cases are useful for testing whether the implementation follows its declared rules, but they are not an independent sample of human conversations. Passing the authored cases demonstrates policy conformance rather than improved communication outcomes.

The supplementary user pilot uses a single-arm, within-participant design and a small convenience sample. It provides product feedback about completion, perceived realism, feedback usefulness, and self-reported confidence. Without a randomised non-adaptive control group, it cannot establish that affect-aware adaptation causes better outcomes than a non-adaptive system.

Self-reported confidence may be influenced by familiarity with the application, demand characteristics, or repeated exposure to a scenario. Deterministic skill measures may change because users learn how the application detects particular language features. Neither type of change establishes that participants communicate more effectively outside the system.

The study is not powered to demonstrate clinical effectiveness, long-term behaviour change, or differences between demographic subgroups. Any observed changes are exploratory and should be interpreted as preliminary evidence for future research rather than as confirmatory proof of effectiveness.

1.8.10 Delimitations

The following areas are intentionally outside the scope of this dissertation:

- clinical diagnosis, treatment, counselling, or therapeutic intervention;
- autonomous crisis assessment or emergency response;
- emotion inference for employment, education, surveillance, or decisions about another person;
- use by children or individuals unable to provide informed consent;
- multilingual emotion recognition or multilingual role-play;
- video, facial-expression, gaze, gesture, or physiological-signal analysis;
- unrestricted simulation of arbitrary real people;
- claims of long-term improvement or transfer to real-world relationships;
- assessment, diagnosis, or treatment of cognitive distortions or other psychological constructs;
- population-level validation across cultures, accents, ages, or clinical groups;
- internet-scale capacity, multi-region high-availability hosting, or public clinical deployment; and
- replacing communication professionals, mediators, counsellors, or other forms of human support.

Within these boundaries, the dissertation evaluates AffectLab as an integrated product and engineering contribution. Its findings concern the performance of the selected models on the defined benchmark and the behaviour, robustness, accessibility, recovery, and local responsiveness of the implemented application. Conclusions beyond controlled production deployment require environment-specific load and security testing, additional datasets, comparative studies, and independent ethical and technical validation.

1.9 Dissertation Structure

The dissertation is organised into six chapters.

Chapter 1 introduces the research context and motivation for developing an emotion-aware system for difficult-conversation rehearsal. It defines the problem statement, aim, objectives, and research questions before outlining the proposed approach, expected contributions, scope, and limitations of the dissertation.

Chapter 2 presents the state of the art. It reviews communication-skills rehearsal and conversational simulation, affective computing, textual and speech emotion recognition, emotion recognition in conversation, contextual modelling, multimodal fusion, confidence calibration, conversational agents, large language models, adaptive dialogue systems, evidence-grounded feedback, and the safety and ethical limitations of emotion-aware technologies.

Chapter 3 presents the proposed system architecture. It defines the principal system components and their interactions, application requirements, scope and boundaries, data-management approach, safety and privacy constraints, ethical considerations, and limitations of the proposed design.

Chapter 4 describes the implementation of AffectLab. It documents IEMOCAP preparation and label mapping, contextual text and speech modelling, late fusion and confidence calibration, and the implemented frontend, backend, persistence, and inference components. It also presents the structured role-play policies, uncertainty-aware adaptation, constrained response generation, evidence-grounded feedback, replay and branching functionality, safety and privacy controls, research-data workflow, deployment configuration, fallback behaviour, and implementation-verification provisions.

Chapter 5 presents the evaluation methodology and results. It defines the speaker-independent experimental protocol, classification and calibration measures, development-data separation, and paired dialogue-level statistical procedure before reporting the comparison between utterance-only and context-aware text models, the audio-only baseline, multimodal fusion performance, and confidence-calibration results. It then presents the developer-led evaluation of dialogue behaviour, affect-policy decisions, evidence detection, fallback handling, persistence, replay, branching, accessibility, responsive layouts, end-to-end workflows, restart recovery, and local HTTP performance. A short subsection retains the principal user-pilot results as supplementary product feedback.

Chapter 6 concludes the dissertation by summarising the main findings and contributions in relation to the research questions. It discusses the limitations of the models, dataset, system design, and evaluation, and identifies directions for future work, including deployment-scale performance testing, broader validation, multilingual support, improved communication-feature detection, and further research into responsible affect-aware adaptation.

Chapter 2

State of the Art

2.1 Introduction

The design of an emotion-aware communication-rehearsal system lies at the intersection of communication-skills education, conversational artificial intelligence, affective computing, and human–computer interaction. Progress in any one of these areas does not, by itself, establish that an integrated application will be useful, reliable, or safe. A classifier may perform well on annotated recordings without being reliable for a new speaker; a dialogue generator may produce plausible replies without consistently following a scenario; and a realistic simulation may be enjoyable without producing a measurable learning benefit. A state-of-the-art review must therefore examine both the capabilities of existing techniques and the evidence that supports their intended use.

This chapter first reviews role-play, digital simulation, feedback, and the design of conversational agents. It then examines the theoretical and empirical foundations of affect recognition, with particular attention to the IEMOCAP benchmark, emotion recognition in conversation, contextual text modelling, speech-based methods, and multimodal fusion. The subsequent sections discuss uncertainty calibration, affect-conditioned dialogue, generative systems, interpretability, safety, privacy, and evaluation. Finally, the literature is synthesised to identify what has already been demonstrated, which questions remain open, and how the proposed AffectLab system is positioned with respect to prior work. Throughout, a distinction is maintained between *predicting a dataset label*, *controlling an application*, and *improving a person's communication skills*. These are different tasks and require different forms of evidence.

2.2 Role-Play and Simulation for Communication-Skills Development

2.2.1 Role-play as a practice-based learning method

Interpersonal communication requires more than knowledge of recommended expressions or communication rules. A speaker must formulate a message, respond to objections, clarify intentions, and sometimes sustain a position without controlling the other person's reaction. Role-play makes these decisions observable by placing the learner in a simulated exchange with an interlocutor. It permits errors and alternative responses in a setting with lower consequences than the corresponding real-world interaction. However, the educational value of role-play is not inherent in the mere presence of a simulation: the target skills, the quality of the interaction, and the opportunity to receive and use feedback also matter.

Evidence for structured communication rehearsal is available in professional education. Bosse et al. conducted a randomised controlled trial with 103 medical students assigned to peer role-play, training with standardised patients, or a control group. Both active training approaches improved measures of communication performance and self-efficacy relative to the control condition, with some outcomes favouring peer role-play [BSN+12]. The findings support the value of active and repeatable practice, but the participants, clinical tasks, and assessment environment were specific. They do not establish that a software-based rehearsal system will produce equivalent improvements in everyday workplace or relationship conversations.

A systematic review by Lee et al. examined 14 studies of virtual-patient systems for medical communication training [LKK+20]. The review associated more effective systems with sound instructional features, such as preparation, scaffolding, debriefing, reflection, and feedback, rather than with technological sophistication alone. Importantly, the evidence was heterogeneous, and only some comparative evaluations found significant advantages for virtual patients. The review also distinguished human feedback from automated feedback; it did not establish that automatically generated comments are sufficient to produce learning. These qualifications matter for the design of non-clinical practice tools because conversational realism must not be confused with educational effectiveness.

2.2.2 Simulation realism, repeatability, and transfer

Digital simulations offer properties that conventional role-play cannot always guarantee: the same initial situation can be replayed, different responses can be tried from a common starting point, and an interaction can be reviewed without the presence of another person. Consistent scenarios also permit defined learning objectives and repeatable assessment. Their limitations are equally important. Simulated interlocutors cannot reproduce every social cue, personal history, consequence, or unpredictable response of real people. A user may learn to satisfy the rules of the simulation rather than improve performance in an unfamiliar situation.

A particularly relevant contemporary system is *IMBUE*, proposed by Lin et al., which uses language models to simulate personalised interpersonal situations and provide just-in-time feedback based on an established interpersonal-effectiveness framework [LSR+24]. In a randomised study involving 86 participants, the authors reported improvements in measures including self-efficacy, negative emotion, and communication-skill mastery. They also found that transfer to new and more difficult situations was more limited than improvements within the trained situations. IMBUE provides evidence that language-model-based rehearsal can be organised as a learning activity rather than as an open-ended chatbot, but its results depend on the specific instructional framework, outcome measures, and study design. They should not be interpreted as proof of general or long-term transfer for unrelated applications.

Taken together, these studies support a practice environment with clear scenarios, observable communication goals, structured resistance, multiple attempts, and opportunities for reflection [BSN^+12, LKK^+20, LSR^+24]. For AffectLab, this literature motivates separate scenario definitions for workload negotiation, personal boundary-setting, and the expression of relationship needs. It also suggests that successful rehearsal should not be equated with persuading the simulated character: stating and maintaining a boundary, agreeing on a feasible next step, or documenting an unresolved disagreement may all be legitimate conversational outcomes.

2.3 Feedback, Reflection, and Evidence in Rehearsal Systems

2.3.1 From conversational experience to actionable feedback

Practice becomes more useful when users can relate an assessment to something they actually said and identify a feasible improvement for their next attempt. Hattie and Timperley's synthesis of educational feedback research distinguishes several functions of feedback, including clarification of the learning goal, information about current performance, and guidance about what to do next [HT07]. This provides a useful rationale for moving beyond a generic score or an approving response. In a communication rehearsal, feedback can refer to explicit behaviours, such as stating a concrete request, giving a relevant reason, using non-accusatory language, acknowledging another perspective, or maintaining an expressed boundary.

An important design distinction concerns *feedback evidence* and *feedback presentation*. Evidence may be extracted using observable language features, annotations, or human assessment, whereas presentation determines how that evidence is explained to the learner. IMBUE combines domain-informed assessment and just-in-time guidance within its training process [LSR^+24]. By contrast, the virtual-patient review by Lee et al. found stronger support for the overall instructional design, including guided reflection and human facilitation, than for the intrinsic effectiveness of automated feedback [LKK^+20]. Consequently, an application should not assume that fluent or detailed comments generated by a language model are necessarily valid.

For automated rehearsal systems, this motivates traceable assessments in which each recorded feature points to the turn that activated it. If a message is credited for providing a specific request, the system should identify the relevant text rather than merely assert that the user was assertive. Rule-based detection makes these decisions relatively reproducible, but it is not synonymous with semantic understanding. A rule can miss a perfectly acceptable paraphrase or incorrectly detect a feature in negated text, quotation, sarcasm, or reported speech. Transparent feedback therefore requires both inspectable evidence and explicit acknowledgement that its measures are partial proxies for communication behaviour, not validated measures of an individual's competence.

2.3.2 Reviewing alternatives instead of rewarding one script

Replay, retry, and branch-based review allow users to examine the consequences of a wording choice while preserving the common conversational context. These mechanisms are consistent with the emphasis on reflection and post-simulation activities in effective virtual-patient designs [LKK+20]. Comparing alternative continuations can highlight differences between a vague request and a specific one, or between a boundary that is repeated and one that is abandoned under resistance. This remains an instructional design rationale rather than evidence that branching, by itself, causes real-world improvement.

The evaluation of such tools should also avoid a narrow definition of success. A simulation with deterministic scoring can inadvertently train users to repeat the phrases that trigger its rules. Accordingly, evidence about improvement inside a system must be separated from assessment by independent human raters or behaviour in new settings. This distinction also limits what can be inferred from changes in turn counts, rule-based scores, or self-reported confidence [HT07, LSR+24].

2.4 Conversational Agents for Structured and Generative Role-Play

2.4.1 Deterministic dialogue management

Classical task-oriented dialogue systems use explicit representations of conversational state and action selection. Their main advantage is not that they reproduce spontaneous conversation perfectly, but that state transitions and permissible outcomes can be specified and tested. For example, the RavenClaw framework described by Bohus and Rudnicky separates domain-specific task logic from reusable dialogue-management capabilities such as error handling and turn-taking [BR09]. This separation provides a precedent for keeping dialogue control distinct from the surface wording of an utterance.

In communication rehearsal, explicit dialogue states can represent stages such as opening an issue, clarifying a concern, responding to resistance, and agreeing on a next step. Scenario-specific rules can define both completed and unresolved outcomes. Such a design facilitates regression tests and makes it possible to explain why a scenario advanced or terminated. Its disadvantages include the cost of authoring policies and the difficulty of anticipating diverse linguistic expressions. A fully templated system may also appear repetitive. These limitations create a reasonable motivation for adding generative wording while retaining explicit state transitions.

2.4.2 Language-model simulation and its limits

Large language models extend conversational flexibility because they can paraphrase, preserve aspects of a character profile, and respond to inputs not anticipated in a fixed script. IMBUE demonstrates one use of this capability for personalised interpersonal-skills training [LSR+24]. Tamoyan et al. subsequently investigated

LLM-based role-play as a method for simulating multi-turn human–chatbot interaction, including persona construction, dialogue goals, and recovery from conversational failures [TSG25]. These studies are useful evidence that explicit roles and goals can structure generation, even though their evaluation tasks are not identical to everyday communication rehearsal.

Generative flexibility comes with risks. Lu et al. compared LLM-generated with human-authored responses in multi-turn professional role-play dialogues and reported that, in their evaluated setting, human judgements of generated-response quality deteriorated across turns, particularly for naturalness and context maintenance [LJG25]. This result cautions against evaluating a role-playing agent using only isolated responses. A simulated character may respond convincingly at the beginning of an exchange yet lose track of a constraint or become overly accommodating later. Likewise, a response that sounds supportive can undermine the exercise if it removes intended resistance or declares a difficult situation resolved prematurely.

The literature thus suggests a distinction between a *dialogue decision* and its *linguistic realisation*. In a hybrid design, the application determines the current stage, whether a communication feature has been observed, which outcomes are allowed, and how strongly the simulated character should resist. Optional generation then realises a bounded response plan; the generated text is checked against the plan and can be replaced by a deterministic template when necessary. This design is motivated by dialogue-management principles and observed weaknesses of multi-turn generation, rather than by a claim that prompting can eliminate all generative failures [BR09, LJG25].

2.5 Affective Computing and the Meaning of Emotion Labels

2.5.1 Categorical and dimensional representations

Affective computing investigates computational systems that model or respond to affect-related signals, but the underlying concept of emotion is not universally defined. Ekman's work argues for distinguishing a set of basic emotions with characteristic features [Ekm92]. An alternative tradition represents affect using continuous dimensions. Russell's circumplex model, for example, describes affective concepts in terms of related dimensions conventionally interpreted as pleasantness or valence and activation or arousal [Rus80]. Scherer emphasises the importance of distinguishing emotion from related constructs such as mood, feeling, and other affective processes, and of making the operational definition explicit [Sch05].

These approaches lead to different machine-learning targets. A categorical classifier may predict *anger*, *happiness*, *sadness*, or *neutral*, whereas a dimensional predictor may estimate valence and arousal. Neither representation captures every relevant aspect of lived emotional experience. Categorical labels are convenient for benchmarking and error analysis, but require a finite label vocabulary, sometimes collapse distinct experiences into one class, and may impose a single answer on an ambiguous utterance. Dimensional representations can express degrees of affect, but introduce their own annotation and interpretation difficulties. Therefore, selecting a classification taxonomy is a modelling decision, not a demonstration that emotion naturally consists of the selected labels.

2.5.2 Expression, interpretation, and uncertainty

The observable relationship between an expression and a person's internal state is imperfect. Barrett et al. reviewed evidence challenging a simple one-to-one inference from facial movements to discrete emotional states [BAM+19b]. Their analysis specifically concerns faces and should not be misrepresented as an experimental result about text or speech. Nevertheless, its broader methodological warning is pertinent: observed behaviour and annotator interpretations are not direct access to subjective experience. Linguistic statements and vocal characteristics can likewise be shaped by social setting, personal speaking style, acting, irony, and the demands of a particular conversation [Sch05].

For this reason, the term *emotion recognition* is used in the conventional machine-learning sense of predicting operational labels, not as proof that a system understands the individual. The result of a classifier can be useful as a contextual signal even when its label is uncertain, provided that its effects remain modest and reversible. An application that uses uncertain predictions to diagnose distress, infer intention, or judge a person's competence would require a substantially stronger evidential basis than ordinary benchmark classification.

2.6 Datasets and Evaluation Protocols for Emotion Recognition

2.6.1 IEMOCAP and the four-class benchmark

The Interactive Emotional Dyadic Motion Capture database (IEMOCAP) was introduced by Busso et al. to study expressive communication through multiple channels [BBL+08]. It contains approximately twelve hours of scripted and improvised dyadic interactions by ten actors arranged into five recording sessions, together with textual, acoustic, visual, and annotation information. Because participants engage in exchanges rather than isolated monologues, the corpus permits investigations of conversational context and of how speech and text complement each other. Its availability has made it a common reference point for text-only, audio-only, and multimodal emotion-recognition work.

IEMOCAP experiments do not all use an identical classification problem. A frequently adopted configuration includes four categories: anger, happiness, sadness, and neutral. In this configuration, *excitement* may be merged into *happiness* [LWC+22]. Papers may instead use different subsets, annotation aggregation rules, segmentation procedures, or label mappings. Such differences can change class prevalence and reported performance. Consequently, results should only be compared quantitatively when the data, labels, folds, and metrics are compatible. The primary experiments in this dissertation adopt the four-class definition specified in Chapter 1; the exact label mapping and preprocessing are documented in Chapter 4.

2.6.2 Other datasets and domain differences

MELD was introduced by Poria et al. as a multimodal, multi-party conversational dataset derived from the television programme *Friends* [PHM+19]. It contains approximately 13,000 annotated utterances from 1,433 dialogues and includes textual, acoustic, and visual information. Unlike IEMOCAP's paired actor sessions, MELD contains multi-speaker television interactions, with different production, editing, and linguistic characteristics. DailyDialog, developed by Li et al., instead consists of human-written everyday dialogues with manually assigned emotion and communicative-intention labels [LSS+17]. It is useful for text-based conversational studies, but it does not provide a directly equivalent speech-recording benchmark.

Table 2.1: Representative conversational datasets and implications for AffectLab.

Corpus	**Main characteristics**	**Relevance and limitations**
IEMOCAP [BBL+08]	Five dyadic sessions with ten actors; dialogue transcripts, recorded speech, additional modalities, and emotion annotations.	Supports matched text–audio experiments and held-out-session testing; acted/improvised English speech and a small speaker pool constrain generalisation.
MELD [PHM+19]	Multi-party television dialogues with text, audio, visual information, and emotion labels.	Useful for studying multi-speaker context; sitcom interactions, production conditions, and label definitions differ from IEMOCAP.
DailyDialog [LSS+17]	Human-written multi-turn everyday dialogues labelled for emotion and communicative intention.	Relevant to dialogue semantics and context but cannot support the same matched audio–text inference task.

A benchmark corpus should be evaluated according to the claims it can support. High accuracy on acted English-language interactions does not demonstrate equivalent performance with spontaneous speech, different microphones, background noise, unfamiliar accents, other languages, or speakers whose expressive styles differ from the training actors. These discrepancies are especially consequential when an application makes use of a model's confidence to change its behaviour. Although additional datasets could be valuable for external validation, incompatible corpora cannot simply be pooled without addressing annotation and domain differences [BBL+08, PHM+19, LSS+17].

2.6.3 Evaluation splits and information leakage

Conversational emotion-recognition samples are not independent in the same way as unrelated text documents. Consecutive utterances share speakers, recording conditions, scenario content, and emotional trajectories. Randomly assigning individual utterances to training and test sets can create substantial overlap in the information observed during development and evaluation. IEMOCAP's five sessions enable a more conservative protocol in which a complete session is held out and the remaining sessions are used for training and validation. Li et al. explicitly evaluate their model under a five-fold, leave-one-session-out configuration, as well as a separate Session 5 setting [LWC+22].

Holding out a session limits direct overlap between training and test speakers, but it does not guarantee generalisation beyond IEMOCAP. The model may still benefit from the same recording studio, language, task structure, and annotation conventions. Validation and hyperparameter selection must also respect group boundaries, and conversational features for a held-out utterance must not contain annotation labels or future information unavailable at the moment of inference. Such precautions are central to a meaningful comparison of utterance-only, contextual, and multimodal systems.

2.7 Textual Emotion Recognition and Conversational Context

2.7.1 Pre-trained language representations

Textual emotion classification has progressed from handcrafted lexical indicators towards contextual neural representations. BERT demonstrates how a transformer can learn general-purpose bidirectional language representations through pre-training and then be adapted to downstream classification tasks [DCLT19]. RoBERTa revisits the training procedure and scale of BERT, showing that pre-training choices strongly influence performance [LOG+19]. For affective tasks, such encoders offer representations of sentence meaning and local word context that are more flexible than matching isolated emotional keywords. Their representations are nevertheless optimised for the training objectives and examples they have encountered; apparent emotional understanding must be assessed through downstream labels and held-out cases.

An utterance-only classifier predicts a label from the current utterance u_t:

$$\hat{y}_t = f(u_t). \tag{2.1}$$

This formulation is reproducible and computationally convenient, but an utterance such as “That’s fine” can express agreement, disappointment, anger, or sarcasm depending on earlier dialogue and delivery. An isolated classifier cannot infer those relationships from the transcript alone. This motivates emotion recognition in conversation (ERC), where the sequence and speakers of the preceding utterances are treated as relevant context.

2.7.2 Speaker and dialogue-context modelling

DialogueRNN, proposed by Majumder et al., models the evolving states of the speakers and uses attention over the conversation for emotion classification [MPH+19]. DialogueGCN, proposed by Ghosal et al., represents utterances and intra- and inter-speaker relations using graph convolution, allowing information to propagate between connected turns [GMP+19]. COSMIC extends context modelling by incorporating commonsense representations of events, mental states, and social interactions [GMG+20]. These studies demonstrate that the relevant unit of analysis is frequently a sequence of social exchanges rather than an isolated text string.

CoMPM, developed by Lee and Lee, combines a context model with representations derived from a speaker's preceding utterances, avoiding dependence on a structured commonsense knowledge base [LL22]. This work is particularly relevant to live interaction because it focuses on information available from conversational history. More sophisticated contextual architectures can represent additional relations, but their extra components may impose larger training, inference, and reproducibility costs. For an application-oriented dissertation, a simpler controlled comparison between context-free and context-aware representations may therefore provide a more interpretable empirical question than an attempt to reproduce every contemporary ERC architecture.

2.7.3 Causal context and deployment compatibility

Not every offline ERC configuration is directly usable in an interactive agent. A model that attends to the full dialogue, including utterances after the target turn, may be valid for retrospective transcript annotation but cannot be deployed unchanged to classify a live user's current message. Similarly, a pipeline that provides the gold emotion labels of neighbouring utterances during testing would rely on information unavailable to the application. For real-time adaptation, the model should use only the current message and permitted history:

$$\hat{y}_t = f(u_t, u_{t-1}, \ldots, u_{t-k}), \tag{2.2}$$

where k is the context-window length and no future user or character turns are visible. Previous utterances can be separated by role markers, preserving the distinction between the user and simulated character. It is important that a bidirectional *token encoder* such as BERT not be confused with bidirectional access to the *conversation timeline*: the encoder can attend to all tokens inside a causally assembled input while the input still excludes future turns.

The literature establishes that context can be useful, but its specific contribution depends on how context is selected, which model is trained, and how the experiment is split [MPH$^+$19, GMP$^+$19, LL22]. For AffectLab, this motivates a matched evaluation of utterance-only and history-conditioned text models with the same session-held-out test items. The aim is to measure a reproducible difference rather than to claim that all contextual methods must outperform simpler approaches.

2.8 Speech Emotion Recognition

2.8.1 Acoustic cues and conventional representations

Speech conveys properties that transcripts discard, including pitch movement, timing, intensity, speaking rate, vocal quality, and pauses. These may provide evidence about expressive delivery, although they are also affected by individual voice characteristics, linguistic content, microphone distance, health, and environment. The survey by El Ayadi et al. identifies feature selection, classification strategies, and database design as core problems in speech emotion recognition (SER) [EAKK11]. This perspective is important because improving the architecture alone cannot remove measurement problems in the recording or weaknesses in the labels.

The Geneva Minimalistic Acoustic Parameter Set (GeMAPS) was proposed by Eyben et al. to standardise a compact group of theoretically motivated descriptors for affective voice research [ESS+16]. Such handcrafted descriptors provide interpretable baseline features and can be suitable for smaller datasets. However, their predefined statistics may not capture all temporal and contextual properties relevant to a classification task, prompting the development of learned representations based on large-scale speech pre-training.

2.8.2 Self-supervised speech representations

Self-supervised learning makes it possible to develop acoustic representations using substantial amounts of unlabelled or weakly labelled speech. wav2vec 2.0 learns speech representations using masking and contrastive prediction in a latent representation space [BZMA20]. HuBERT instead uses masked prediction of hidden units produced by an iterative clustering process [HBT+21]. WavLM extends speech pre-training to support a wider range of tasks through denoising and representation learning that capture multiple properties of the signal [CWC+22]. These papers establish the general value of pre-trained speech representations; their original improvements are not all results on emotion classification, so reported speech-recognition error rates must not be presented as SER accuracy.

For an emotion classifier, learned acoustic representations can be pooled or otherwise summarised before applying a task-specific classification head. The choice to freeze or fine-tune the encoder depends on data quantity, available compute, and overfitting risk. Li et al. provide a more directly relevant affective example by combining pre-trained textual and speech representations on IEMOCAP and examining how feature adaptation changes performance [LWC+22]. Even with strong pre-training, an SER model may learn speaker identity, lexical content, or recording artefacts correlated with benchmark labels. Held-out-speaker evaluation and careful interpretation are therefore essential.

2.8.3 Operational constraints of speech input

Audio has additional application-level costs that may not appear in offline classification results. Microphone quality, permissions, ambient noise, sampling-rate conversion, silence, and short or incomplete recordings all affect the inference pathway. Voice recordings may also contain identifying information. A research application should therefore process audio only when the user explicitly chooses it, minimise retained raw recordings, and make unsuccessful audio inference a recoverable condition. For AffectLab, speech provides a supplementary signal rather than a requirement for completing a text-based rehearsal; the implementation of this behaviour is described in Chapter 4.

2.9 Multimodal Emotion Recognition and Fusion

2.9.1 Complementarity of text and speech

Text and speech are not interchangeable observations. A transcript can clarify what a user explicitly says, while acoustic features may capture emphasis, tension, or delivery. Combining the two is a multimodal learning problem because their representations have different time scales, noise patterns, and failure modes. Baltrušaitis et al. classify multimodal challenges in terms of representation, alignment, fusion, and related processes, showing why the task is more complex than concatenating measurements [BAM19a]. In particular, acoustic frames and word tokens do not automatically correspond one-to-one.

Broadly, *early* or feature-level fusion combines modality representations before classification, while *late* or decision-level fusion combines outputs from separately trained classifiers. Intermediate cross-modal architectures permit richer interactions between modalities. The Multimodal Transformer (MulT) developed by Tsai et al. applies directional cross-modal attention to potentially unaligned multimodal sequences [TBL+19]. Li et al. apply context-aware pre-trained representations and investigate several fusion approaches for IEMOCAP, obtaining reported weighted accuracies of 84.45% in a Session 5 evaluation and 80.36% under their five-fold cross-validation configuration [LWC+22]. These are results under the authors' particular protocols, not performance guarantees for other implementations or datasets.

2.9.2 Decision-level fusion and its trade-offs

A decision-level approach retains two independent class-probability distributions, $p_T(y)$ for the text model and $p_A(y)$ for the audio model. A simple mixture is

$$p_F(y) = \lambda p_T(y) + (1-\lambda)p_A(y), \qquad 0 \leq \lambda \leq 1, \tag{2.3}$$

where λ determines the relative contribution of text. Equal-weight fusion fixes $\lambda = 0.5$; a fitted configuration estimates the weight from appropriate validation data. The resulting prediction can be inspected alongside both modality outputs. This is useful for determining whether a change in classification is attributable to text, audio, or their combination, and for recognising disagreement between the predictors.

Decision-level fusion is less expressive than cross-modal attention because one classifier does not directly inspect low-level features from the other modality. Its advantages are modularity, relatively modest computational coupling, explicit missing-modality handling, and an auditable fusion rule. It also avoids the need for word-to-frame alignment at fusion time. These properties suit an application whose primary model question is whether adding audio improves over a contextual text model, rather than whether a particular complex cross-modal architecture establishes a new benchmark [BAM19a, TBL+19, LWC+22].

The limitations remain important. A fused model may be confidently incorrect if both modalities rely on the same misleading cue. It can also underperform the stronger modality when one source is especially noisy or domain-shifted. Weights chosen on the test set inflate apparent performance, while weights fitted on validation data may not transfer to new microphones or speakers. Therefore, the fusion method and its selection procedure must be predeclared and evaluated separately from the final held-out test data.

2.10 Confidence Calibration, Uncertainty, and Generalisation

2.10.1 Accuracy does not establish trustworthy probabilities

A classifier's predicted probability is not automatically a calibrated statement about correctness. For example, among samples assigned confidence near 0.8, a well-calibrated model should be correct approximately 80% of the time under the evaluated distribution. Guo et al. found that modern neural networks often produce miscalibrated confidence estimates and showed that temperature scaling is a practical post-processing approach [GPSW17]. Temperature scaling transforms classifier logits using a positive validation-fitted temperature before applying softmax; it changes confidence magnitudes while preserving the top-class ranking for a given model under the usual single-temperature formulation.

For a model with logit vector z and temperature $T > 0$, the calibrated class probability is

$$p_T(y = c \mid z) = \frac{\exp(z_c/T)}{\sum_{j=1}^{C} \exp(z_j/T)}. \tag{2.4}$$

Temperature scaling is relatively simple, but other approaches are possible. Kull et al. introduced Dirichlet calibration as a more flexible multiclass mapping and compared it using metrics including log-loss and the Brier score [KPNK$^+$19]. More flexible calibration models may require more validation data and can overfit, so complexity is not necessarily beneficial for a small, speaker-grouped dataset.

2.10.2 Evaluating classification and calibration separately

Classification measures describe the labels predicted; calibration measures describe the probabilities used to express confidence. Accuracy can obscure class imbalance, and a high weighted score can coexist with weak recognition of a less frequent emotion. Macro-averaged F1, per-class precision and recall, confusion matrices, and support counts are therefore useful complements. Probability quality can be examined using negative log-likelihood, the multiclass Brier score, and reliability diagrams. Expected calibration error (ECE) summarises differences between binned confidence and observed accuracy, but depends on the chosen bins and does not completely characterise multiclass calibration [GPSW17, KPNK$^+$19].

The distinction is especially relevant to fusion. Equal weighting may obtain the best classification score while a validation-fitted or post-calibrated configuration produces more meaningful confidence estimates. Selecting whichever configuration scores highest on the final test set would invalidate the comparison. Instead, training, model selection, calibration fitting, and testing should remain separate within each session-held-out experiment. Differences should be accompanied by uncertainty estimates and class-sensitive analysis rather than presented only as a single benchmark number.

2.10.3 Distribution shift and restrained use of predictions

Calibration is a property of a model relative to a data distribution. Ovadia et al. demonstrated that methods that produce useful probabilities under ordinary benchmark conditions can deteriorate when the input distribution changes [OFR$^+$19]. This is directly relevant when moving from studio-recorded English conversations to speech captured through personal devices. Even a calibrated IEMOCAP probability must not be interpreted as a guarantee that a user's private emotional experience has been correctly identified.

For an adaptive application, a threshold on maximum probability should consequently be viewed as an operational policy choice rather than a validated psychological boundary. Agreement between text and audio can be informative but is not proof of correctness. A suitable conservative policy may default to baseline dialogue, offer the user an explicit pacing choice, or suppress automated emotional claims when inference is uncertain, conflicting, or unavailable. Importantly, such policies can be evaluated for predictable behaviour even without establishing that the classifier correctly identifies the user's actual feelings [GPSW17, OFR$^+$19].

2.11 Affective and Empathetic Conversational Agents

2.11.1 Generating emotion-sensitive responses

Prior work on affective conversational agents has studied both the expression of emotion and responses to another speaker's expressed experiences. Zhou et al. proposed an Emotional Chatting Machine that conditions response generation on emotional categories and mechanisms representing emotional state and vocabulary [ZHZ$^+$18]. Rashkin et al. introduced the EmpatheticDialogues dataset and found that models trained with emotionally situated conversations were judged more empathetic than comparison systems trained only on broad conversational data [RSLB19]. These contributions show that affect-related signals can shape dialogue-generation objectives and perceived response quality.

However, generating an apparently empathetic response is not identical to correctly recognising a speaker's emotional state, and neither result establishes safe adaptation in a learning system. Empathy can be perceived through wording even when the model's assumptions are wrong. Furthermore, in a role-play exercise the simulated character's purpose is not always to be maximally agreeable or reassuring: a productive practice scenario may involve scepticism, delay, or refusal. Therefore, an affective mechanism must be subordinated to the scenario's instructional purpose and the user's control over the interaction.

2.11.2 Bounded adaptation rather than emotion-driven control

AffectLab adopts an application-oriented interpretation of emotion awareness. Model predictions can provide context for a small set of allowable adjustments, such as optional acknowledgement or a choice to slow the exchange. They do not define what the user really feels, determine character compliance, or modify success and safety criteria. If modalities disagree or confidence is low, asking the user to choose a pace is less presumptive than asserting an emotional state. If inference fails, the normal dialogue remains available. This division of responsibilities is informed by the uncertainty literature and the distinction between affective response generation and validated educational action [OFR$^+$19, RSLB19, LSR$^+$24].

This approach also limits the meaning of *adaptivity*. A system may be adaptive because it changes an interface decision in response to input, but this does not automatically make the change beneficial. A valid engineering evaluation can establish that the intended branch was selected under specified inputs. Demonstrating educational benefit from affect adaptation would require a controlled user study comparing otherwise similar adaptive and non-adaptive versions; the small supporting pilot cannot, on its own, establish that causal effect.

2.12 Trust, Safety, Privacy, and Human Oversight

2.12.1 Boundaries between rehearsal and psychological support

Difficult interpersonal conversations may contain disappointment, conflict, or serious distress. This makes the interpretation and presentation of automated responses particularly consequential. De Freitas and Cohen describe health risks associated with generative AI wellness applications, including occasions when users bring crisis-related concerns to systems not adequately prepared to respond [DFC24]. AffectLab is not presented as a mental-health intervention, and these risks do not imply that rehearsal and treatment are equivalent. They do, however, justify clear scope limits, emergency-service guidance when appropriate, and independence between safety decisions and optional probabilistic emotion inference.

Separating safety handling from the classifier prevents a four-category benchmark model from becoming an unvalidated crisis detector. Similarly, a generator should not be allowed to override a safety interruption or invent authoritative assessments of a participant. Independently inspectable dialogue controls, deterministic fallbacks, and the ability to pause or finish a scenario are therefore part of responsible system design rather than merely conveniences. The need for such boundaries is reinforced by concerns over long-running generated role-play and context preservation [LJG25, DFC24].

2.12.2 Data minimisation and legal context

Voice recordings and personal conversations can reveal identifying or sensitive information beyond the narrow task of emotion classification. Appropriate safeguards include informed participation, explicit microphone activation, transient audio handling when possible, restricted access to saved conversations, pseudonymous research identifiers, retention limits, and withdrawal procedures. These are design requirements whose adequacy must also be examined against the actual deployment environment; code-level safeguards alone cannot establish regulatory compliance.

In the European context, Regulation (EU) 2024/1689, the Artificial Intelligence Act, includes restrictions on certain uses of emotion inference, notably its prohibition concerning emotion inference in workplace and educational settings subject to the regulation's stated exceptions [Eur24]. This legal provision should not be simplified into a universal ban on all emotion-aware software, nor does a non-clinical label remove obligations relating to personal data. AffectLab is intended for voluntary individual rehearsal rather than employer or institutional assessment, surveillance, or consequential decisions about other people. Its applicable obligations would still require case-specific legal and ethical assessment before real-world deployment.

2.13 Model and Application Evaluation

An integrated application requires evidence at complementary levels. *Model evaluation* asks whether a classifier predicts predefined labels for held-out examples and whether its probabilities are calibrated. *Application evaluation* asks whether dialogue stages, feedback evidence, safety overrides, permissions, persistence, accessibility, recovery, responsive layouts, and performance behave as specified. Limited user feedback can reveal workflow friction or perceived usefulness, but it is supplementary to these technical forms of evidence. Confusing the levels produces exaggerated conclusions: passing application tests does not validate emotion recognition, while model accuracy alone does not establish that the completed product is robust or helpful.

For the modelling component, the surveyed literature motivates five session-held-out IEMOCAP experiments with consistent examples across text-only, audio-only, contextual, and fused configurations. Results should include accuracy, macro-F1, weighted F1, class-level error patterns, and calibration measures. Because utterances from one conversation are dependent, paired comparisons should respect conversational grouping rather than treating every turn as an independent participant. The precise statistical procedure and development-data separation are presented in Chapter 5 [BBL$^+$08, LWC$^+$22, GPSW17].

For the complete application, controlled test cases should include normal progression and failure paths: lack of a required communication feature, an unresolved conflict, model disagreement, low confidence, missing audio, unavailable language-model generation, a safety-triggering phrase, an interrupted session, stale or duplicate requests, persistence across restart, and desktop/mobile interaction. Performance checks should report their transport, persistence, model, provider, and concurrency scope so that local latency is not misrepresented as production capacity. A small user pilot may add feedback about completion, realism, and usefulness, but any observed change must not be interpreted as a causal learning effect without an appropriate comparison group [LKK$^+$20, LSR$^+$24].

2.14 Synthesis of Existing Research

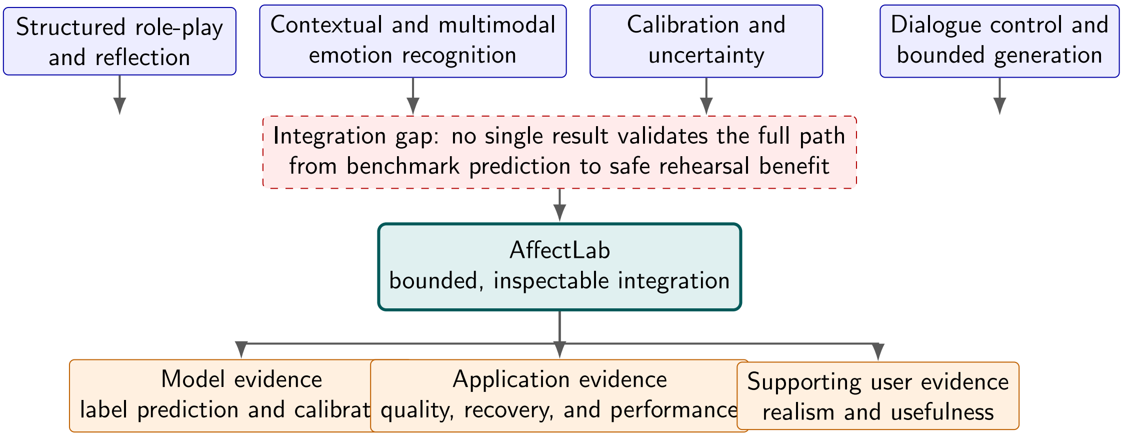
The studies reviewed above address complementary parts of the overall problem. Research on communication training supports structured practice and reflection. ERC research develops contextual models and multimodal features that can improve benchmark predictions. Calibration research warns against treating predictive confidence as self-validating. Dialogue research shows both the flexibility of generated responses and the need to test their consistency over multiple turns. None of these results, considered independently, validates the complete chain from emotion inference to safe rehearsal behaviour and meaningful user feedback.

Table 2.2: Selected research approaches, demonstrated contributions, and limitations for communication rehearsal.

Work	**Main contribution or evidence**	**Limit for AffectLab's problem**
Bosse et al. [BSN+12] and Lee et al. [LKK+20]	Controlled role-play evidence and synthesis of virtual-patient instructional designs.	Mostly clinical training; do not evaluate the proposed multimodal, uncertainty-aware individual rehearsal workflow.
IMBUE [LSR+24]	Personalised LLM-assisted interpersonal simulation with expert-informed feedback and participant evaluation.	Transfer varies across outcomes and situations; it does not establish the correctness of AffectLab's separate text–audio inference policy.
DialogueRNN, DialogueGCN, COSMIC, CoMPM [MPH+19, GMP+19, GMG+20, LL22]	Speaker-aware, contextual, graph-based, and commonsense-informed ERC modelling.	Classification architectures are not themselves instructional or safety-controlled rehearsal systems.
Li et al. [LWC+22]	Context-sensitive text–audio emotion classification with IEMOCAP session-aware experiments.	Benchmark classification does not establish calibrated behaviour or reliable deployment inference in an interactive role-play application.
Tsai et al. [TBL+19] and Baltrušaitis et al. [BAM19a]	Cross-modal modelling and a taxonomy of multimodal learning problems.	Rich fusion architectures do not automatically address missing inputs, transparent decisions, or learning outcomes.
Guo et al. [GPSW17] and Ovadia et al. [OFR+19]	Confidence calibration methods and evidence of degradation under distribution shift.	General findings require task-specific validation and cannot guarantee calibration on AffectLab participants.
Lu et al. [LJG25]	Multi-turn comparison exposing quality degradation in generated professional role-play dialogue.	Evaluates dialogue quality rather than the effect of a bounded hybrid controller on participants' learning.

Three conclusions are particularly important. First, *performance measures are not interchangeable.* Weighted emotion-classification accuracy, perceived empathy, realistic dialogue, acceptability, and communication-skill scores describe different phenomena. Second, *context and multimodality create opportunities as well as vulnerabilities*: adding information can improve a benchmark result but increases dependence on data quality, compatible inputs, and the assumed distribution. Third, *reliable application behaviour requires explicit policies*, especially when machine-learning results are uncertain or generative services are unavailable. A coherent system should therefore preserve provenance from model output to adaptation decision and from observed utterances to feedback.

Figure 2.1 summarises how these research streams motivate an integration problem while retaining separate standards of evidence.


Structured role-play and reflection
Contextual and multimodal emotion recognition
Calibration and uncertainty
Dialogue control and bounded generation
Integration gap: no single result validates the full path from benchmark prediction to safe rehearsal benefit
AffectLab
bounded, inspectable integration
Model evidence
label prediction and calibrat
Application evidence
quality, recovery, and performance
Supporting user evidence
realism and usefulness

The evidence sources answer different questions and are not interchangeable.

Figure 2.1: Relationship between the reviewed research streams, the AffectLab integration gap, and its complementary model, application, and supporting user evidence.

2.15 Research Gap and Positioning of AffectLab

The identified research gap is not that conversational role-play, emotion recognition, multimodal fusion, or feedback generation have never been studied. Each has an extensive literature. Rather, the literature motivates a need for a carefully bounded, reproducibly evaluated integration in which (i) a live-compatible textual context model is compared with an utterance-only baseline; (ii) independent text and speech models are compared with a transparent fusion mechanism; (iii) classification performance is distinguished from calibrated operational confidence; and (iv) affect predictions are allowed to influence only explicit, low-consequence role-play actions. This is an *integration and evaluation gap*, not a claim of universal novelty or proof that no similar application exists.

AffectLab addresses this gap through the following linked decisions. Its machine-learning study uses a four-category IEMOCAP task with five held-out sessions, a causal history window for text, an independent audio branch, and late fusion. Equal-weight and validation-fitted configurations are distinguished, and calibration is examined separately from classification. This is intended to provide controlled evidence for RQ1 and RQ2 without claiming real-world emotion understanding [BBL+08, LWC+22, GPSW17].

Its conversational component uses scenario-specific, deterministic stages and completion rules, while optional language-model generation provides bounded linguistic variation. Confidence, agreement, modality availability, and the user's stated pacing preference are interpreted by a versioned, inspectable adaptation policy. Missing or disputed emotion evidence leads to baseline interaction or an explicit user choice rather than an imposed psychological interpretation. Safety and feedback are independent of the affect classifier, and generation failures have deterministic fallbacks. Together, these choices turn RQ3 into a question about verifiable application behaviour rather than an untestable claim of machine empathy [BR09, LJG25, OFR+19].

Finally, AffectLab treats communication rehearsal as a useful, reflective activity delivered through a fully integrated application. Users practise workload negotiation, boundary maintenance, and the expression of relationship needs, then inspect evidence-linked feedback, repeat or branch an exchange, and record an actionable takeaway. RQ4 is therefore addressed through developer-led verification of the complete product across functional, persistence, recovery, accessibility, responsive-interface, performance, and deployment boundaries. A small pilot contributes supporting perceptions of realism and usefulness but is not asked to prove product readiness or causal communication improvement [LKK+20, LSR+24].

2.16 Chapter Summary

Previous research provides substantial evidence for the value of structured communication practice, the usefulness of instructional feedback and reflection, the contribution of conversational context to emotion classification, the availability of strong pre-trained acoustic and textual representations, and the potential of multimodal fusion. It also reveals important limitations: benchmarks are domain-specific, labels do not directly measure private emotion, model confidence can be unreliable, generated dialogue can lose consistency, and apparently helpful simulations do not necessarily produce transferable learning [LKK+20, LSR+24, LWC+22, OFR+19, LJG25].

The work undertaken in this dissertation is consequently different from simply training another emotion classifier or constructing an unconstrained role-playing chatbot. AffectLab combines controlled context-aware text and speech experiments with a deliberately modest affect-adaptation policy, structured dialogue management, evidence-linked feedback, optional generation, reproducible fallbacks, and layered technical verification. Its contribution is to join these parts into a transparent, production-oriented application while keeping conclusions within the available evidence. Chapter 3 defines the proposed architecture, requirements, scope, ethical boundaries, and limitations; Chapter 4 documents the model and application implementation; and Chapter 5 presents the evaluation methodology and results.

Chapter 3

Proposed System Architecture

3.1 System Overview

AffectLab is designed as a hybrid, full-stack, production-oriented application for rehearsing difficult interpersonal conversations. Its architecture combines deterministic application logic, trained affect-recognition models, optional generative services, and a browser-based interaction interface. The system is not constructed as a single end-to-end conversational model. Instead, responsibilities are separated so that probabilistic components can contribute linguistic flexibility and affective information without controlling safety-critical or evaluative decisions.

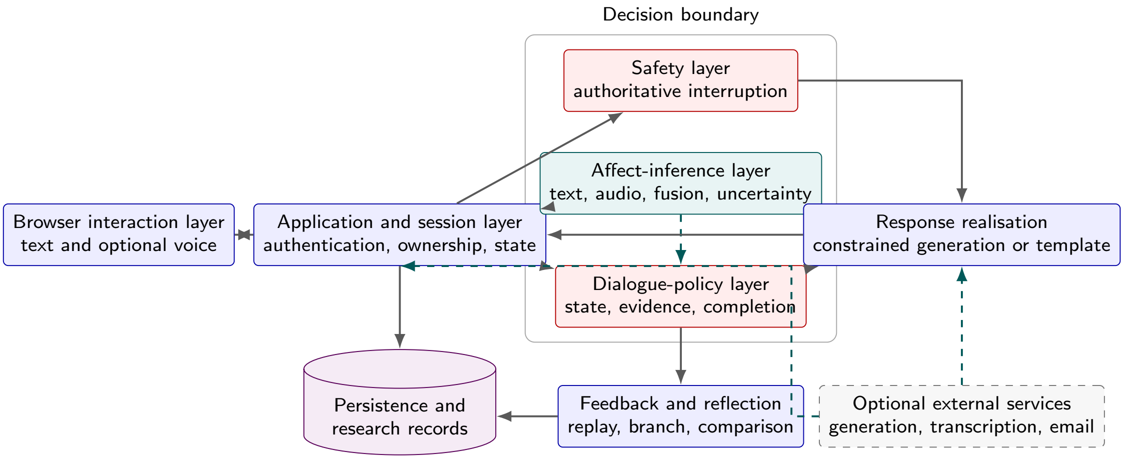
The central architectural principle is the separation of *inference*, *control*, and *presentation*. The affect-recognition subsystem estimates a distribution over a limited set of emotion categories from conversational text and optional speech. The dialogue-control subsystem determines how the scenario progresses, which communication features have been observed, and whether a completion condition has been reached. The response-generation subsystem expresses the selected dialogue action using either constrained generative wording or a deterministic template. The interface presents the conversation, optional affect information, feedback, and reflection tools to the user. This separation makes it possible to inspect and test each decision independently.

The system follows a text-first design. A user can complete the entire rehearsal by typing, while voice input is an optional enhancement. At the beginning of a rehearsal, the user selects a scenario, difficulty level, and, where available, a simulated character profile. The standard scenarios concern workload negotiation, personal boundary-setting, and the expression of a relationship need. Each scenario defines its objective, expected communication features, dialogue stages, difficulty-dependent behaviour, and possible completion outcomes.

During the rehearsal, the user submits a textual utterance and may optionally attach a short voice recording. The text is always available to the dialogue and safety components. When voice input and trained-model inference are enabled, the recording is processed for the current request and is not retained as part of the conversation history. Denial of microphone access, missing audio, or an inference failure therefore does not prevent the rehearsal from continuing.

Table 3.1 summarises the main architectural components and the boundaries placed around their responsibilities.

Figure 3.1 shows the corresponding information and authority flow. Solid paths represent the core workflow, while dashed paths identify optional evidence or services whose absence must not prevent text-based rehearsal.


Decision boundary
Safety layer
authoritative interruption
Affect-inference layer
text, audio, fusion, uncertainty
Browser interaction layer
text and optional voice
Application and session layer
authentication, ownership, state
Response realisation
constrained generation or template
Dialogue-policy layer
state, evidence, completion
Persistence and
research records
Feedback and reflection
replay, branch, comparison
Optional external services
generation, transcription, email

Probabilistic components provide evidence; deterministic components retain authority.

Figure 3.1: Logical architecture of AffectLab and separation between probabilistic evidence and authoritative application control.

For each submitted utterance, processing begins with authentication, session-ownership validation, and input validation. The safety layer then examines the textual message before ordinary dialogue generation occurs. If a safety condition is detected, the normal role-play path is interrupted, a predefined safety response is returned, and the interruption is recorded. This ordering prevents optional affective or generative components from overriding an explicit safety decision.

When no safety interruption is required, the affective and conversational analyses are performed. The contextual text model receives the current message together with a causal window of preceding turns. If an audio sample is available, the audio model analyses the corresponding speech signal independently. The calibrated text and audio probability distributions can then be combined through late fusion. The system retains the individual modality outputs so that disagreement, missing information, and the contribution of each modality remain observable.

The resulting affective estimate is interpreted by a bounded adaptation policy. The policy considers model availability, calibrated confidence, agreement between modalities, the current dialogue state, and the user's selected pacing preference. Where affect adaptation is enabled and the evidence satisfies the declared conditions, the system may make a limited change such as adding a tentative acknowledgement. Low confidence or disagreement may lead to an explicit pacing choice, while unavailable inference produces the standard non-adaptive response. The policy does not describe the estimated category as a verified internal emotion.

In parallel with this bounded adaptation, the deterministic dialogue policy evaluates the user's message against the active scenario. It records observable language features and selects the next dialogue action. Depending on the scenario, relevant features may include a specific request, a concrete detail, an "I" statement, non-blaming language, acknowledgement of another perspective, maintenance of a boundary, or a proposed next step. The policy also determines whether the conversation should advance, continue under resistance, finish successfully, close without resolution, or stop because the maximum number of turns has been reached.

Table 3.1: Principal components of the AffectLab architecture.

Component	**Primary responsibility**	**Architectural boundary**
Interaction layer	Supports scenario selection, text and optional voice input, conversation display, pausing, replay, branching, feedback review, and personal takeaways.	Does not determine dialogue outcomes, affect labels, safety decisions, or feedback scores.
Application and session layer	Coordinates authenticated requests, session state, scenario configuration, conversation history, model calls, and persistence.	Accepts decisions from specialised components but does not treat a generated response as the authoritative dialogue state.
Safety layer	Examines textual input for crisis-related language and applies a predefined interruption response when required.	Operates independently of affect classification and takes precedence over dialogue generation and affect adaptation.
Affect-inference layer	Produces contextual text, audio, and fused probability distributions together with confidence, agreement, and availability information.	Provides bounded evidence rather than a statement of the user's true emotional state. Its output cannot determine safety, scenario success, or feedback scores.
Dialogue-policy layer	Detects observable communication features, advances the scenario state, selects the simulated character's intended action, and determines completion or unresolved outcomes.	Uses explicit, scenario-specific rules. It is authoritative for progression even when generative response wording is enabled.
Response-realisation layer	Transforms the dialogue action into a natural-language response using constrained generation or a deterministic template.	May alter wording but cannot modify the selected action, completion state, recorded evidence, or safety response.
Feedback and reflection layer	Calculates communication metrics, links them to supporting turns, and supports replay, retry, branch comparison, and takeaway creation.	Bases assessment on recorded observable evidence. Generative phrasing cannot introduce unsupported findings.
Persistence and research layer	Stores account-owned sessions, textual turns, dialogue decisions, feedback evidence, model provenance, and permitted research measures.	Does not retain raw voice recordings and separates identifiable application data from pseudonymous research exports.

The selected action is passed to the response-realisation component. When generative services are enabled, a language model receives a constrained representation of the scenario, simulated character, difficulty, current state, and required dialogue action. The generated wording is accepted only as a realisation of that action. If generation is disabled, unavailable, or invalid, the system uses the deterministic response associated with the same action. Consequently, a service failure can reduce linguistic variety but does not remove the core rehearsal workflow.

After the response has been produced, the system persists the textual turns and their associated decision metadata. This metadata includes the dialogue state before and after the exchange, detected communication evidence, selected action, reason codes, affect-policy decision, model or policy versions, and whether generative or fallback wording was used. Recording these elements supports reproducibility and makes it possible to distinguish what the system decided from how that decision was expressed.

At the end of a rehearsal, the feedback component calculates scenario-specific metrics from the evidence accumulated during the conversation. Each metric is linked to the turn or turns that support it. Optional generative processing may improve the readability of the feedback, but the deterministic metrics and recorded evidence remain authoritative. The completed conversation can subsequently be replayed, retried from the latest exchange, or used as the basis for an alternative branch. Branch comparison evaluates the new continuation separately from the conversational context copied from the original attempt.

This architecture supports the research questions at distinct levels. The contextual text, audio, fusion, and calibration components support the model comparisons in RQ1 and RQ2. The separation between affect inference, bounded adaptation, deterministic dialogue control, and safety addresses RQ3. The integrated rehearsal, feedback, persistence, recovery, accessibility, and deployment workflow provides the system evaluated for application quality in RQ4. Maintaining these architectural boundaries also prevents model classification performance, policy conformance, application readiness, and supplementary user feedback from being treated as interchangeable forms of evidence.

3.2 System Requirements and Scope

3.2.1 Overview

The system requirements translate the research aims introduced in Chapter 1 and the design implications identified in Chapter 2 into explicit capabilities and constraints for AffectLab. They define what the application is expected to do, the conditions under which it should operate, and the responsibilities that must remain outside probabilistic affect recognition and generative artificial intelligence.

The requirements are divided into functional and non-functional requirements. Functional requirements describe user-visible capabilities and internal system behaviours. Non-functional requirements describe qualities that apply across those capabilities, including safety, privacy, explainability, reliability, accessibility, and reproducibility. Together, they provide a basis for designing the architecture and for evaluating whether the implemented system conforms to its intended behaviour.

A distinction is maintained between *core* and *optional* functions. Text-based structured rehearsal, deterministic dialogue control, safety handling, feedback, and session management constitute the core workflow. Voice capture, trained multimodal inference, transcription, generative response wording, and generative feedback phrasing are optional enhancements. The absence or failure of an optional component must not make the core rehearsal unusable.

3.2.2 Scope of the System

AffectLab is intended to support adults who wish to prepare for and rehearse ordinary but potentially difficult interpersonal conversations. It provides a private, repeatable environment in which users can practise alternative wording, experience controlled conversational resistance, inspect feedback linked to their own messages, and reflect on possible next steps.

The primary scope comprises three standard communication situations:

- negotiating workload or priorities with a manager;
- stating and maintaining a personal boundary; and
- expressing a relationship need using specific and non-blaming language.

Each standard scenario defines a simulated character, a communication objective, expected observable features, dialogue stages, difficulty-dependent behaviour, and explicit completion conditions. The system may additionally support user-prepared scenarios, provided that they use the communication features and safety constraints supported by the application.

The system supports text-based interaction as the default modality. Optional voice input can be used for transcription and acoustic affect analysis when the relevant services are enabled. Affect analysis is limited to estimates over predefined categories derived from the selected research dataset. These estimates may inform restrained changes in conversational pacing, but they are not presented as direct measurements of the user's internal emotional state.

General conversation outside an active rehearsal uses an additional deterministic routing layer. It may expose tentative labels for requested support, a phrase following a causal connector, conversational pushback, or a matched cognitive-pattern vocabulary and use them to select a supportive response strategy. These labels describe which authored lexical rule fired on the current message. They are not latent-state estimates, clinical constructs, or user attributes, and the dissertation evaluates them only as application rules. Cognitive assessment, diagnosis, and treatment remain outside scope.

The scope also includes post-rehearsal reflection. Users can inspect evidence-grounded feedback, replay an attempt, retry an exchange, create an alternative conversational branch, compare compatible attempts, record a personal takeaway, and prepare an action card. These functions support deliberate practice rather than unrestricted social conversation.

For optional research use, AffectLab includes versioned consent and eligibility procedures, pre- and post-rehearsal measures, pseudonymous research records, technical-event recording, and restricted aggregate export. These functions support supplementary study workflows. They do not transform the application into a clinical intervention or establish that practice transfers to real-world behaviour.

3.2.3 Functional Requirements

The functional requirements of AffectLab are as follows.

FR1. **User authentication and ownership** The system shall provide account-based access and shall associate every conversation, rehearsal, feedback record, branch, preparation record, and personal takeaway with its owning user. A user shall not be able to access another user's resources.

FR2. **Onboarding and disclosure** The system shall explain its purpose, limitations, data handling, optional use of voice, and non-clinical status before the user begins the main workflow. It shall not present itself as a therapist, diagnostic instrument, mediator, or emergency service.

FR3. **User preferences and practice goals** The system shall allow users to select and update practice goals and relevant interaction preferences. Optional microphone use and affect adaptation shall remain under user control.

FR4. **Scenario preparation** The system shall allow the user to prepare for a conversation by recording its context, the intended outcome, relevant concerns, and a possible objection or source of resistance. Preparation information shall be available to the corresponding rehearsal without becoming an authoritative assessment of the situation.

FR5. **Scenario selection** The system shall provide standard scenarios for workload negotiation, personal boundary-setting, and expressing a relationship need. It may also allow account-owned custom scenarios constructed from the supported communication objectives and observable features.

FR6. **Difficulty and character configuration** The system shall support multiple difficulty levels and character profiles. These settings may vary cooperation, resistance, scepticism, time pressure, or willingness to clarify, but shall not modify the safety rules or the meaning of the scenario's communication objectives.

FR7. **Rehearsal lifecycle** The user shall be able to start, pause, resume, manually finish, and leave a rehearsal. The system shall preserve the current state when continuation is permitted and shall record the reason for completion or interruption.

FR8. **Text-first interaction** The system shall accept a textual message for each conversational turn. All core scenario, safety, feedback, and reflection functions shall remain available without microphone access or audio input.

FR9. **Optional voice input** Where voice functions are enabled, the system shall allow the user to record a short audio sample. A transcript produced from the recording shall be presented for review and editing before it is submitted as a conversational turn.

FR10. Safety precedence The system shall examine textual input for supported crisis-related indicators before applying ordinary affect adaptation or response generation. When a safety condition is detected, it shall interrupt the ordinary role-play path, return the predefined safety response, and record the rehearsal as safety-interrupted.

FR11. Contextual text inference Where trained inference is enabled, the system shall construct the text-model input from the current utterance and a fixed causal window of preceding conversational turns. It shall not use future dialogue turns or unavailable annotation information.

FR12. Acoustic inference and multimodal fusion When valid audio is supplied, the system shall produce separate textual and acoustic probability distributions and may combine them using the declared late-fusion configuration. It shall retain modality availability, confidence, agreement, calibration, and model-version information needed to interpret the result.

FR13. Missing-modality handling The system shall handle missing, rejected, invalid, or failed audio without terminating the text-based rehearsal. The absence of audio shall not be interpreted as negative evidence about the user's affect.

FR14. Bounded affect adaptation Where affect adaptation is enabled and permitted, the system shall translate model confidence, modality agreement, inference availability, user preference, and dialogue state into a limited set of predefined actions. Affect predictions shall not determine crisis handling, scenario success, feedback scores, participant eligibility, or research inclusion.

FR15. Deterministic dialogue progression The system shall use scenario-specific policies to detect observable communication features, update the dialogue stage, select the simulated character's intended action, and determine whether a completion condition has been reached.

FR16. Evidence recording For every relevant user turn, the system shall record the observable features detected by the active scenario policy and link those features to the corresponding conversation turn. The system shall preserve the policy and scoring versions used to produce the evidence.

FR17. Constrained response realisation The system shall transform the selected dialogue action into a simulated-character response. A configured language model may phrase the response, but it shall not alter the authoritative dialogue action, completion decision, safety result, or recorded evidence.

FR18. Deterministic fallback If generative response production is disabled, unavailable, rejected, times out, or produces an invalid result, the system shall return a deterministic response associated with the same dialogue action. The source of the response and any fallback reason shall be recorded.

FR19. Explicit rehearsal outcomes The system shall distinguish between relevant completion outcomes, including achievement of a practical next step, maintenance of a boundary, unresolved closure, manual completion, maximum-turn completion, and safety interruption. Scenario success shall not always require agreement from the simulated character.

FR20. Evidence-grounded feedback At the end of a rehearsal, the system shall calculate scenario-specific communication metrics from recorded evidence. Feedback shall identify supported strengths and areas for further practice without making claims about personality, diagnosis, or general communication competence.

FR21. Controlled feedback phrasing A language model may improve the wording of deterministic feedback when configured, but it shall not introduce a strength, weakness, score, or supporting example that is absent from the authoritative feedback record.

FR22. Replay and evidence inspection The system shall allow the user to review the completed conversation and identify the turns associated with feedback evidence, dialogue decisions, and permitted affect-adaptation actions.

FR23. Retry and branching The system shall support retrying an eligible exchange and creating an alternative branch from a supported point in the conversation. Branching shall preserve the copied context while distinguishing it from newly generated continuation turns.

FR24. Attempt comparison The system shall allow comparison only between compatible attempts, such as rehearsals using the same scenario and scoring version. For a branch, feedback shall evaluate the new continuation separately from the copied context.

FR25. Personal reflection The system shall allow the user to save a personal takeaway and create an action-oriented summary of wording or next steps they wish to remember. User-authored reflections shall remain distinct from system-generated feedback.

FR26. Session persistence and deletion The system shall retain the textual and structural information required for session recovery, replay, branching, and feedback for the configured retention period. Users shall be able to delete eligible sessions and their account-owned data.

FR27. Research participation controls When the research workflow is enabled, the system shall record versioned consent, eligibility confirmation, questionnaire status, required-task progress, and withdrawal. Research collection shall begin only after the applicable consent and eligibility requirements have been satisfied.

FR28. Pseudonymous research access The system shall provide authorised researchers with aggregate monitoring and pseudonymous exports containing only the fields required by the study protocol. Research exports shall exclude names, email addresses, passwords, raw conversation text, raw audio, custom scenario wording, feedback prose, and personal takeaways.

FR29. Bounded general support routing Outside active role-play, the system may use deterministic lexical rules to select among a declared set of general support strategies. Any intent, cognitive-pattern, possible-context, readiness, or resistance output shall be presented as a tentative rule match rather than a psychological measurement. The subsystem shall not determine crisis detection, eligibility, research inclusion, scenario progression, completion, or feedback scores, and its rule-priority values shall not be presented as probabilities.

3.2.4 Non-Functional Requirements

The non-functional requirements apply across the complete system.

NFR1. Safety Safety handling shall remain independent of the trained affect classifier and shall take precedence over affect adaptation and generative response production. Failure of a probabilistic component shall not suppress a deterministic safety response.

NFR2. Graceful degradation The core text-based rehearsal shall remain operational when microphone permission is denied, audio is unavailable, trained inference cannot be loaded, transcription fails, or an external generative service is disabled or unreachable.

NFR3. User control Users shall be able to decline optional voice processing, disable eligible adaptation, select an available pacing preference, pause a rehearsal, finish it manually, or leave it without being required to accept an inferred emotional interpretation.

NFR4. Privacy and data minimisation The system shall collect and retain only information required for the declared application and research functions. Raw audio shall be processed transiently and shall not be stored as part of the conversation record.

NFR5. Security The system shall protect accounts and sessions through secure password storage, authenticated access, resource-ownership checks, restricted configuration secrets, session expiry, and deployment-appropriate transport and host controls.

NFR6. Explainability and auditability Consequential system behaviour shall be traceable to an explicit policy, evidence record, reason code, configuration, or safety rule. The system shall distinguish model predictions, dialogue decisions, generated wording, fallback responses, and feedback metrics.

NFR7. Reproducibility Model, policy, scenario, scoring, consent, study-protocol, and export-schema versions shall be recorded where they affect interpretation. Evaluation configurations shall be preserved so that reported results can be reproduced without selecting settings from final test outcomes.

NFR8. Reliability and consistency Repeated execution with the same deterministic state and input shall produce the same dialogue transition, completion decision, evidence record, and feedback score. Concurrent or duplicate submissions shall not silently produce contradictory session states.

NFR9. Responsiveness and failure recovery Model-dependent operations shall use bounded execution and expose appropriate busy, error, or fallback states. Failure of an optional operation shall return control to the user rather than leaving the session in an indeterminate state. The locally controlled text-first path shall be measurable independently of provider and model latency so that responsiveness regressions can be detected reproducibly.

NFR10. Accessibility Core workflows shall support keyboard navigation, visible focus, accessible form labels, understandable status announcements, sufficient contrast, reduced-motion preferences, and layouts suitable for desktop and mobile interaction.

NFR11. Maintainability and modularity Persistence, affect inference, dialogue policy, safety, generation, feedback, and research-data functions shall be separated through explicit interfaces. It shall be possible to replace an optional model or external provider without rewriting the authoritative scenario logic.

NFR12. Testability The architecture shall permit automated tests of dialogue transitions, safety precedence, affect-policy decisions, missing-modality behaviour, fallback generation, feedback evidence, access control, persistence, replay, branching, and research export.

NFR13. Data integrity Stored sessions shall preserve the relationship between turns, dialogue decisions, evidence, feedback, branches, and version information. Invalid or stale updates shall be rejected rather than silently overwriting newer session state.

NFR14. Appropriate presentation of uncertainty Affect outputs shall be presented as model estimates rather than verified facts. Low-confidence, conflicting, or unavailable results shall be distinguishable from supported predictions and shall not be converted into stronger psychological claims.

3.2.5 Assumptions

The proposed architecture is based on the following assumptions:

- Users are adults practising ordinary interpersonal communication rather than seeking diagnosis, treatment, crisis intervention, or professional mediation.
- The primary interaction language is English because the scenarios, deterministic language features, safety phrases, and emotion-recognition datasets are designed for English-language input.
- Every conversational turn contains text, either typed directly or reviewed after optional transcription. The system does not rely exclusively on acoustic input.
- Users have access to a modern web browser and a sufficiently stable connection to the application. Voice functions additionally require a compatible microphone and browser permission.

- The predefined communication features are limited proxies for observable wording. Their detection does not provide a complete assessment of communication quality or interpersonal effectiveness.

- Predictions derived from IEMOCAP may not generalise to spontaneous speech, different languages, cultures, accents, devices, or recording environments. Calibration measured on the benchmark may also deteriorate under deployment conditions.

- Agreement between textual and acoustic models increases the consistency of available model evidence but does not prove that the predicted category is correct.

- Optional external services are available only when valid configuration and credentials are provided. Their outputs, availability, latency, and data-handling conditions are not fully controlled by AffectLab.

- Replay, feedback, and branching require the system to retain conversation text for a declared period. Users are expected to receive clear information about this retention before using the application.

- Any participant study or remote deployment will occur only after the required institutional, ethical, security, and data-protection reviews have been completed.

3.2.6 System Boundaries

The system boundaries define both the technical responsibility of AffectLab and the limits of the claims that may be made from its outputs.

External systems may include the user's browser and microphone, an email delivery service, an optional transcription or language-model provider, the database service, and the deployment infrastructure. AffectLab controls the information sent through its defined interfaces but cannot guarantee the internal availability, behaviour, or data-processing practices of an independently operated provider. Consequently, externally dependent functions must remain optional or provide an explicit failure path.

The system boundary also separates participant-facing data from research-facing data. Participants may access their conversation content, feedback, branches, and takeaways. Researchers receive only the pseudonymous measures authorised by the study protocol. Operational access to a deployed database or server is an administrative responsibility and is not equivalent to permission to use participant data for research.

3.2.7 Summary

AffectLab is scoped as a text-first, structured communication-rehearsal application with optional voice, affective, generative, and research-support capabilities. Its functional requirements cover the complete path from onboarding and scenario

Table 3.2: Principal boundaries of the proposed AffectLab system.

Boundary	**Within the system**	**Outside the system**
Intended use	Preparation, structured rehearsal, evidence-grounded feedback, repetition, and reflection for difficult interpersonal conversations.	Therapy, diagnosis, clinical treatment, emergency response, professional mediation, employment assessment, or institutional surveillance.
Affect recognition	Prediction of predefined dataset labels from causal text context and optional speech, including confidence and modality agreement.	Direct measurement of private emotion, intention, truthfulness, personality, mental health, or future behaviour.
Dialogue control	Scenario-specific stages, resistance, observable features, response actions, completion rules, and deterministic fallbacks.	Unrestricted open-domain conversation or autonomous alteration of learning objectives by a language model.
Generative artificial intelligence	Optional wording of an already selected response action and optional phrasing of evidence-grounded feedback.	Authority over safety, scenario progression, success, scoring, eligibility, research outcomes, or unsupported advice.
Safety	Conservative text-first checks, interruption of the rehearsal, predefined guidance, and separation from affect inference.	Comprehensive crisis detection, clinical risk assessment, monitoring of the user's environment, or contact with emergency services.
Data handling	Authenticated session storage, limited conversation retention, transient audio processing, account controls, and pseudonymous research records.	Guarantees about infrastructure or external-provider processing beyond the configured deployment and applicable agreements.
Evaluation	Emotion-classification evaluation, calibration analysis, engineering conformance, accessibility, recovery, and performance tests, with a small supplementary user pilot.	Proof of therapeutic effectiveness, causal learning benefit, reliable real-world emotion understanding, or transfer to future conversations.

preparation to controlled role-play, multimodal inference, feedback, replay, branching, reflection, and optional research-data collection. Its non-functional requirements emphasise safety precedence, graceful degradation, privacy, security, accessibility, explainability, reproducibility, responsiveness, and user control.

The defining constraint is that probabilistic components provide bounded support rather than authoritative control. Emotion predictions cannot determine safety, success, or assessment, while generative models cannot change dialogue decisions or invent feedback evidence. These requirements establish the basis for the component architecture and interaction flow described in the following sections.

3.3 Ethical Considerations and Limitations

3.3.1 Ethical Approach and Intended Use

AffectLab operates in a sensitive domain because users may enter personal information, discuss conflict, provide voice recordings, and receive system-generated interpretations or feedback. Ethical considerations are therefore treated as architectural requirements rather than as matters to be addressed only after implementation. The principal concerns are the possible misinterpretation of affect predictions, inappropriate reliance on generated responses, loss of user autonomy, exposure of sensitive information, unequal model performance, and confusion between communication rehearsal and psychological support.

The intended use of AffectLab is limited to voluntary preparation for and rehearsal of ordinary interpersonal conversations by adults. It is not intended for diagnosis, treatment, clinical triage, crisis monitoring, employment assessment, educational assessment, surveillance, deception, or the evaluation of another person without their knowledge. It must not be used to infer protected characteristics, stable personality traits, truthfulness, or psychological disorders.

The system is intended to support reflection rather than prescribe a correct way to conduct a relationship or workplace conversation. Its scenarios simplify complex interpersonal situations into structured practice tasks. Completing a scenario therefore means satisfying the declared communication conditions within the simulation; it does not establish that the same response is appropriate in every real-world context.

3.3.2 Ethical Use of Affect Recognition

The most significant ethical risk associated with AffectLab is that a predicted emotion could be mistaken for knowledge of the user's internal state. The trained models do not directly observe emotion. They classify textual and acoustic patterns according to categories derived from annotations in IEMOCAP. These categories represent an operational benchmark rather than a complete theory of human affect [BBL$^+$08].

The primary four-class task contains anger, happiness, neutrality, and sadness, with excitement incorporated into the happiness category. It does not adequately represent anxiety, frustration, embarrassment, fear, shame, mixed affect, deliberate emotional masking, or uncertainty. A model can consequently be statistically correct according to the benchmark while still producing a description that is inappropriate for an individual user.

To reduce this risk, affect outputs are treated as uncertain evidence rather than facts. The interface and dialogue policy must avoid statements such as “you are angry” or “the system knows that you are sad”. Where an affective response is permitted, the wording should remain tentative and should not require the user to accept the interpretation. The user may continue normally, select a preferred pace, or disable an eligible adaptive function.

Confidence does not eliminate this ethical concern. Neural classifiers may produce high-confidence errors, particularly when deployment data differ from their training distribution [GPSW17, OFR$^+$19]. Similarly, agreement between the textual and acoustic models does not prove correctness because both may respond to the same misleading cue. The low-confidence threshold and agreement rules used by AffectLab are therefore operational safeguards rather than scientifically validated boundaries between correct and incorrect emotional interpretation.

The architecture restricts affect predictions to low-consequence and reversible actions. They cannot determine crisis escalation, scenario success, communication-skill scores, study eligibility, or research inclusion. If prediction is unavailable, uncertain, or conflicting, the system returns to baseline dialogue or offers an explicit user choice. This limits the harm that can result from an incorrect classification.

3.3.3 Ethical Limits of Cognitive-Pattern and Intent Heuristics

The rule-based component named `RuleBasedCognitiveAnalyzer` in the source code creates a particular risk of category confusion. Terms such as “cognitive distortion”, “intent”, “cause”, “readiness”, and “resistance” have psychological or interpersonal meanings beyond what a phrase matcher can support. In AffectLab, they are implementation labels for lexical routing: the system does not interview the user, examine enduring beliefs, consider a clinical history, or compare observations with a validated instrument or professional judgement. The ethical justification for retaining the component is therefore narrow. It can make a low-consequence template choice reproducible and inspectable, but it cannot assess cognition.

The user-facing explanation presents a matched item as a tentative pattern that may be wrong and presents the extracted clause only as possible context. Nevertheless, tentative wording does not make an invalid inference valid. False positives may pathologise ordinary emphasis, quotation, sarcasm, or culturally specific language; false negatives may omit indirect or unfamiliar wording; the “cause” field may merely copy a grammatically adjacent clause; and the intent, readiness, and resistance values may misrepresent what the user wants. Naming a familiar therapeutic concept may also lend the output more authority than its evidence warrants.

Several authority limits reduce, but do not remove, these risks. The fields are not used for eligibility, study inclusion, feedback scores, scenario success, or clinical and safety decisions. During active role-play, the scenario controller rather than the support-strategy selector determines the required action. A crisis flag takes precedence, and the generator is instructed not to diagnose. The labels and strategy reasons remain visible so that the routing is contestable rather than hidden. These controls justify treating the subsystem as a transparent prototype heuristic; they do not justify a claim of psychological validity or user benefit.

Accordingly, the evaluation standard for this dissertation is implementation conformance: whether declared strings produce declared tags and whether priority rules produce the specified response category. Any stronger use would require a separately justified construct definition, expert and target-user review, representative annotated data, inter-rater procedures, error and subgroup analysis, comparison with a neutral baseline, and prospective evaluation of helpfulness and harm. Until such evidence exists, the outputs must not be presented or interpreted as clinical assessment, and the safest future refinement may be to replace psychological terminology with descriptions of directly observed language.

3.3.4 User Autonomy, Transparency, and Avoidance of Dependency

Users should understand when they are interacting with deterministic application logic, a trained classifier, or a generative model. AffectLab therefore distinguishes dialogue decisions from their linguistic realisation and records whether a response was generated or produced from a deterministic template. Feedback metrics are similarly separated from any optional generative phrasing.

Voice input is optional and requires an explicit user action. A transcript created from a recording is placed in the message composer for review and editing rather than being submitted automatically. This allows the user to correct transcription errors and decide whether the content should become part of the conversation.

The user retains the ability to pause, resume, finish, or leave a rehearsal. Affect adaptation must not be used to pressure the user into continuing, accepting an interpretation, disclosing more information, or selecting a particular response. Manual completion is not treated as misconduct, and a user is not penalised for refusing voice input or declining an adaptive feature.

The system must also avoid encouraging emotional dependency. Generated language should not imply consciousness, exclusive understanding, personal attachment, or an ability to replace human relationships and professional services. AffectLab may support rehearsal and reflection, but it must not position itself as the user's primary source of emotional support or claim authority over consequential interpersonal decisions.

3.3.5 Safety and Non-Clinical Boundaries

Difficult-conversation practice may elicit or contain language associated with distress, self-harm, abuse, or immediate danger. AffectLab is not designed to assess the severity of such circumstances. Its safety mechanism is a conservative text-first interruption layer whose purpose is to prevent ordinary role-play from continuing when supported crisis-related expressions are detected.

Safety handling remains independent of multimodal affect inference. A four-category emotion classifier is not an appropriate crisis detector, and an apparently neutral prediction must not override explicit crisis-related language. Safety handling also takes precedence over the language model so that a generated response cannot suppress or replace the predefined interruption.

The safety layer nevertheless has important limitations. Lexical matching may fail when risk is expressed indirectly, metaphorically, sarcastically, with spelling variation, or in an unsupported language. Optional provider moderation may identify additional cases, but it is also probabilistic and cannot guarantee detection. Conversely, conservative rules may interrupt a conversation when crisis-related wording is quoted or discussed hypothetically.

The application is not continuously monitored, does not establish the user's physical location, and cannot independently contact emergency services or another trusted person. Its safety response can provide appropriate signposting, but it cannot perform clinical triage. These limitations must be disclosed before use and repeated when a safety interruption occurs. The risks of presenting generative wellness systems as substitutes for professional care further support this restrained scope [DFC24].

3.3.6 Privacy, Data Protection, and Data Governance

Conversation text may contain names, workplace details, relationship information, and other sensitive personal content. Voice recordings may additionally reveal identity-related characteristics, accent, health information, background conversation, or environmental details. The fact that users choose to enter this information does not remove the system's responsibility to minimise its collection and exposure.

The design follows the principles of purpose limitation, data minimisation, storage limitation, transparency, integrity, and confidentiality described by the General Data Protection Regulation [Eur16]. These principles are reflected in several architectural decisions:

- text is collected only when required for conversation, feedback, replay, or authorised research functions;
- microphone access requires an explicit user action;
- audio duration and accepted file size are restricted;
- raw audio is processed transiently and is not intentionally retained in the conversation record;
- account-owned conversations expire according to a declared retention policy;
- users can delete individual sessions and request account deletion;
- research records exclude raw conversation text, audio, custom scenario wording, feedback prose, and personal takeaways; and
- access to participant and researcher functions is separated and authenticated.

Pseudonymisation reduces direct identifiability but does not make active research records anonymous while they remain linkable for withdrawal or data management. This distinction must be explained accurately. Participants may withdraw from research and have identifiable study records removed where possible. If data have already been irreversibly de-identified and included in a frozen aggregate dataset, removal may no longer be technically possible; this limitation must be stated before consent rather than introduced only after withdrawal.

Research consent and the lawful basis for processing personal data are related but distinct matters. Participation consent should describe the study, its risks, voluntary nature, and withdrawal procedure. The data controller must separately determine and document the appropriate legal basis for each processing purpose. The existence of a consent screen alone does not demonstrate full data-protection compliance.

When an external provider is configured for transcription, generation, feedback phrasing, or moderation, relevant content may leave the locally controlled application boundary. Users must be told which functions involve external processing and whether a text-only or offline alternative is available. Provider credentials must remain outside source code and logs, while ordinary application logs should avoid storing prompts, generated responses, access tokens, or raw audio.

Technical controls such as password hashing, authenticated ownership checks, restricted research exports, transport encryption, secure configuration, rate limiting, and backup protection reduce risk but cannot guarantee absolute confidentiality. A remote pilot additionally requires deployment-specific security review, appropriate institutional approval, an assessed lawful basis, and consideration of whether a data-protection impact assessment is necessary.

3.3.7 Fairness, Representation, and Accessibility

IEMOCAP contains a small number of English-speaking actors recorded in controlled dyadic sessions. Speaker-independent evaluation prevents direct reuse of test speakers during training, but it does not demonstrate equitable performance across cultures, accents, age groups, disabilities, gender identities, socioeconomic backgrounds, communication styles, recording devices, or spontaneous emotional situations [BBL+08].

The available dataset is insufficient for strong demographic fairness conclusions. Reporting only aggregate accuracy could conceal systematically weaker behaviour for particular classes or speakers. The model evaluation should therefore include per-class results, confusion matrices, fold-level variation, and failure analysis. These analyses reveal some forms of uneven performance, but they do not establish demographic fairness when the required demographic coverage and statistical power are absent.

Bias can also arise from deterministic rules. Feature detectors may favour direct, standardised English phrasing and fail to recognise indirect requests, culturally specific politeness, neurodivergent communication patterns, quoted speech, or valid paraphrases. The detected features are therefore proxies for the scenario's declared objectives rather than universal measures of respectful or effective communication.

The system should not penalise a user for an accent, unavailable microphone, transcription error, disability, or preference for text. Core interaction must remain keyboard accessible and usable without audio. Accessible labels, visible focus, status announcements, reduced-motion support, and responsive layouts are required to reduce avoidable participation barriers. Nevertheless, compliance checks cannot establish accessibility for every user, and evaluation with diverse participants remains necessary.

3.3.8 Generative Artificial Intelligence and Feedback Integrity

A generative model may produce fluent language while failing to follow the intended scenario. It may become overly agreeable, reinforce an unrealistic assumption, introduce unsupported content, or express advice with excessive confidence. Multi-turn role-play can also lose consistency as the conversation grows [LJG25]. Natural wording must therefore not be interpreted as evidence that the response is educationally valid or emotionally accurate.

AffectLab limits the authority of generation by requiring the deterministic dialogue policy to select the response action first. The generator may express that action in varied language, but it cannot determine scenario completion, change the evidence record, or override safety. When generation is unavailable or rejected, the associated template response is used.

The same separation applies to feedback. Observable features and scores are computed deterministically. A generative model may phrase strengths and suggestions only from this structured record. It must not create a supporting example, skill observation, or criticism that is absent from the recorded evidence.

These safeguards reduce but do not eliminate generative risk. Prompt constraints cannot guarantee that every output is appropriate, culturally sensitive, or factually correct. Provider models and policies may also change after deployment. Generation source, model configuration, validation outcome, and fallback reason should consequently be retained as operational provenance.

3.3.9 Research Ethics and Participant Protection

The participant evaluation is designed as a feasibility and usability pilot rather than an effectiveness trial. Participation is restricted to adults who can understand the English-language information and provide informed consent. Individuals seeking crisis care, diagnosis, treatment, or emergency support are outside the intended study population. Eligibility confirmation is self-reported and must not be described as a clinical screening assessment.

Before participation, users must receive understandable information about:

- the research purpose and study procedure;
- the non-clinical status of the system;
- the use and limitations of affect recognition;
- optional voice and external-provider processing;
- the information collected and its retention period;
- foreseeable privacy, emotional, and technical risks;
- the voluntary nature of participation;
- the right to pause, omit optional input, stop, or withdraw; and
- the circumstances under which withdrawn or irreversibly de-identified data can no longer be removed.

Required research tasks use standardised scenarios and intermediate difficulty to reduce unnecessary variation. Optional voice and multimodal affect inference are exploratory and are not required for study completion. Safety interruptions and technical failures are recorded as feasibility outcomes and must not be interpreted as communication-skill failures.

Pre- and post-rehearsal ratings are study-specific feasibility measures rather than validated clinical instruments. Participants may answer or explicitly skip them. Missing values must remain missing rather than being replaced with fabricated neutral scores. Research exports use pseudonymous identifiers and exclude conversation content. Only authorised researchers should access the research dashboard and exports.

Recruitment must not begin until the applicable institutional ethics approval, participant-facing materials, researcher contact details, retention schedule, deployment review, and data-management responsibilities have been confirmed. Material changes after recruitment begins require a new protocol version and, where applicable, renewed approval and participant consent.

3.3.10 Legal and Regulatory Context

The ethical safeguards described above do not by themselves establish legal compliance. Applicable obligations depend on the purpose of processing, the data used, the deployment context, the organisations involved, and the jurisdiction in which the system operates.

The European Union Artificial Intelligence Act prohibits certain emotion-inference uses in workplace and educational-institution contexts, subject to its stated exceptions [Eur24]. AffectLab is designed for voluntary individual rehearsal and not for employers, teachers, or institutions to assess workers or students. No employer or educational authority should receive individual affect predictions or use them for selection, discipline, grading, access, or performance evaluation. However, describing the application as voluntary research does not automatically determine its legal classification. The applicability of the prohibition and other obligations requires formal assessment before institutional or public deployment.

The AI Act does not replace the General Data Protection Regulation. Conversation text, account information, research measurements, and voice recordings may constitute personal data even where an AI Act prohibition does not apply. Whether a particular voice-processing operation involves special-category biometric data depends on how and why the data are processed and should not be assumed solely from the presence of an audio recording. Applicable lawful bases, transparency duties, data-subject rights, processor agreements, international transfers, retention rules, security measures, and impact-assessment obligations require deployment-specific review [Eur16].

Accordingly, the dissertation evaluates technical and ethical safeguards but does not claim to provide a definitive legal determination or compliance certification.

3.3.11 Limitations of the Proposed System

Construct and model limitations

AffectLab predicts benchmark categories rather than directly measuring emotion. Its labels simplify affective experience, and its confidence values are conditional on the model, dataset, and calibration procedure. Even a well-calibrated classifier can be wrong for an individual example. Performance on IEMOCAP does not demonstrate equivalent performance during real application use.

The contextual model uses a fixed number of preceding turns and cannot represent every relevant feature of a relationship or situation. The audio model may be influenced by microphone quality, background noise, speaking style, health, accent, or recording distance. Late fusion cannot correct an error when both modalities are misleading, and it may perform worse than the stronger individual modality when one input is degraded.

Dialogue and feedback limitations

Deterministic policies improve predictability but can respond only to anticipated stages and detectable language patterns. They may fail to recognise a valid paraphrase, sarcasm, indirect communication, or a creative solution. Template responses may become repetitive, while generated responses remain probabilistic despite their constraints.

Feedback metrics capture only the observable features defined for each scenario. They cannot evaluate sincerity, appropriateness for the user's actual relationship, non-verbal behaviour, long-term consequences, or the other person's interpretation. Users may also learn to optimise their wording for the detectors without developing a broader communication skill.

Safety limitations

The safety layer cannot identify every dangerous or abusive situation. It is strongest for explicit supported English phrases and weaker for indirect, coded, multilingual, or context-dependent expressions. It cannot verify whether a user is currently safe, determine their location, or provide a monitored emergency response. False positives and false negatives remain possible.

Privacy and security limitations

Although raw audio is not intentionally retained, transient processing and transmission still create exposure. Stored conversation text may contain sensitive information, and pseudonymous research data may remain linkable while withdrawal is supported. Software controls cannot eliminate risks arising from compromised devices, incorrect deployment configuration, administrator access, provider-side processing, unpatched dependencies, or unauthorised disclosure.

Evaluation limitations

The engineering tests use authored and synthetic cases. Passing them demonstrates conformance to declared policies but not performance across the full diversity of human conversation. Local response-time checks do not establish concurrent-user capacity or external-provider latency. The small user pilot uses a single-arm design and does not include a randomised non-adaptive comparison; it can provide product feedback but cannot establish that AffectLab causes communication improvement.

Self-reported confidence can be affected by familiarity, repetition, demand characteristics, or a desire to satisfy the researcher. Deterministic skill scores can improve because participants learn which phrases the system recognises. Neither result establishes transfer to real conversations, improved relationships, or therapeutic benefit.

Operational and regulatory limitations

Optional services depend on network availability, provider behaviour, credentials, and external data-handling conditions. Model or provider changes can affect outputs even when the local application remains unchanged. Local tests cannot establish the security, reliability, accessibility, or legal compliance of every deployment environment.

The regulatory discussion in this dissertation is necessarily general. It does not replace advice from the responsible institution, ethics body, data-protection officer, or qualified legal professional. A future change in population, purpose, language, deployment setting, or recipient of the affect predictions may materially change the associated obligations.

3.3.12 Summary

The ethical design of AffectLab is based on restrained claims, separation of responsibilities, informed user choice, data minimisation, and explicit limits on probabilistic components. Affect predictions are treated as uncertain contextual signals; generative models are restricted to response and feedback wording; deterministic policies retain authority over safety, dialogue progression, evidence, and scoring. Users retain control over voice input, pacing, continuation, deletion, and research participation.

These measures reduce foreseeable risks but do not remove them. Affect recognition may be inaccurate or culturally inappropriate, safety mechanisms may miss indirect risk, stored conversations remain sensitive, generated language may be unsuitable, and a small user pilot cannot establish effectiveness. AffectLab should therefore be understood as a bounded, production-oriented application whose responsible use depends on transparent presentation, controlled deployment, appropriate oversight, and conclusions that remain proportionate to the available evidence.

Chapter 4

Implementation

4.1 Chapter Overview

This chapter describes the implementation of AffectLab, translating the architecture, requirements, and safeguards defined in Chapter 3 into a production-oriented application. The implementation combines a React and TypeScript browser interface, a Python and FastAPI backend, MongoDB persistence, trained text and speech emotion-recognition models, and optional external services for language generation, transcription, moderation, and email delivery. These elements are integrated as replaceable components rather than as a single end-to-end model, preserving the separation between affect inference, deterministic application control, response realisation, and presentation established in the system design.

The application follows a text-first and failure-tolerant implementation strategy. Structured role-play, dialogue progression, safety handling, evidence collection, feedback, and session management remain available without voice input or network-dependent artificial-intelligence services. When configured, the trained models add contextual textual and acoustic affect estimates, which are calibrated and combined through late fusion. Their outputs are interpreted by a bounded pacing policy and cannot determine crisis handling, dialogue success, or feedback scores. Similarly, a language model may improve the wording of a response or feedback item, but deterministic policies and recorded evidence remain authoritative. This arrangement makes component failures explicit and allows the core rehearsal workflow to fall back to reproducible local behaviour.

The chapter first presents the technology stack, repository organisation, and configuration approach. It then explains the emotion-recognition pipeline, including dataset preparation, contextual text modelling, speech modelling, multimodal fusion, calibration, and model provenance. The subsequent sections describe the backend services and persistence layer, the scenario-based role-play engine, affect-aware pacing, safety precedence, and the feedback and reflection mechanisms. The browser interface is then considered from the perspective of the participant and researcher workflows, including conversation preparation, voice interaction, uncertainty presentation, replay, branching, questionnaires, and research export.

The final sections describe the security and privacy controls, deployment modes, and verification strategy used during development. This chapter is concerned with how the system was constructed and how its implementation was checked through unit, integration, browser, accessibility, performance, end-to-end, and smoke tests.

Quantitative model performance, the consolidated application-verification results, and a short supplementary user-pilot summary are reported separately in Chapter 5.

4.2 Technology Stack and Project Organisation

AffectLab is implemented as a modular full-stack application rather than as a single machine-learning program. The implementation separates the participant-facing interface, application and dialogue logic, persistence, optional external integrations, and model-development code. This separation reflects the architectural boundaries introduced in Chapter 3: a trained classifier supplies evidence to the application, but it is not responsible for authentication, dialogue progression, safety handling, or feedback scoring. It also permits the core text-based rehearsal workflow to run when model artefacts, a graphics-processing unit, or a network-backed provider are unavailable.

Table 4.1 summarises the principal technologies in the implemented system. Exact Python application dependencies are pinned in the backend requirements file, while the frontend lock file records the resolved JavaScript dependency graph. Machine-learning dependencies use bounded version ranges because training and inference may be executed in different managed environments, including local installations and notebook runtimes.

Table 4.1: Principal technologies used in the AffectLab implementation.

Area	**Technologies**	**Implementation role**
Browser application	React 19, TypeScript, Vite, HTML, and CSS	Implements account, rehearsal, feedback, reflection, and research interfaces as a responsive single-page application.
Application programming interface	Python 3.12, FastAPI, Uvicorn, and Pydantic	Validates requests, coordinates services, exposes authenticated HTTP endpoints, and serialises typed response objects.
Persistence	MongoDB 8 and PyMongo	Stores users, account-owned sessions, research records, token digests, lifecycle state, and frozen research exports.
Machine learning	PyTorch, Hugging Face Transformers, DeBERTa-v3, and wav2vec 2.0	Supports text and speech model training, calibration, evaluation, and optional local multimodal inference.
Optional integrations	OpenAI Python client and SMTP	Provide constrained response wording, feedback phrasing, moderation, transcription, and transactional email when configured.
Verification	pytest, Ruff, Playwright, axe-core, ESLint, and the TypeScript compiler	Checks backend behaviour, code quality, browser workflows, accessibility, type consistency, and integrated operation.
Packaging and serving	Docker Compose, Docker, and Nginx	Builds the services, coordinates local deployment, serves the compiled frontend, and provides service health checks.

4.2.1 Frontend Technologies

The participant and researcher interfaces are implemented in React with TypeScript. At the time of the documented build, the lock file resolves React and

React DOM to version 19.2.8, TypeScript to version 6.0.3, and Vite to version 8.2.0. The TypeScript configuration enables strict checking and targets the ES2022 and browser document-object-model libraries. This provides compile-time checking across component properties, application state, and the HTTP data structures shared by the interface and backend schemas.

The frontend is a single-page application with a small application shell in `App.tsx`. It uses the browser History API to select public, authentication, onboarding, user, settings, practice, and optional researcher views. Feature code is divided into components such as the role-play workspace, conversation preparation, voice capture, emotion panel, replay view, branch comparison, study checklist, and researcher dashboard. State that belongs to an individual workflow remains close to the corresponding component, avoiding an unnecessary global state-management dependency.

Communication with the backend is centralised in a typed client under `frontend/src/services`. The client defines the API base address through `VITE_API_URL`, attaches the current access token, includes the refresh cookie, performs one controlled refresh after an authentication failure, and converts non-successful responses into interface errors. Request and response types are declared separately under `frontend/src/types`; this makes mismatches between components and backend response shapes visible during the TypeScript build. Browser session storage contains the short-lived access token, whereas refresh-token handling remains behind the backend-issued cookie boundary.

Vite provides the local development server and the production bundling step. The production container performs the TypeScript build before creating the optimised browser bundle, so type errors prevent an invalid image from being produced. The compiled static files are served by Nginx. Its configuration supports client-side routes by returning `index.html` when no static resource matches, prevents stale caching of the application entry point, and assigns long-lived immutable caching to content-hashed assets. Styling is implemented in the repository's CSS rather than through a component framework, allowing the responsive layouts, keyboard focus states, reduced-motion behaviour, status announcements, and touch-oriented controls to remain directly inspectable.

4.2.2 Backend Technologies

The backend is implemented in Python and is packaged with a Python 3.12 base image. FastAPI 0.141.1 provides the HTTP application and automatic request routing, Uvicorn 0.52.1 supplies the application server, and Pydantic-based schemas validate incoming data and serialise responses. The API is exposed below the `/api` prefix. Its endpoints cover authentication, user settings, session management, role-play actions, feedback, branching, multimodal inference, transcription, participant-study operations, and restricted researcher functions. FastAPI dependency injection supplies authenticated users and the configured services to these endpoint functions.

Backend code is divided by responsibility. The `api` package contains the HTTP boundary; `schemas` defines transport objects; `models` contains domain entities and enumerations; `services` implements application, dialogue, feedback, research, and external-provider behaviour; `repositories` defines persistence operations; `safety` contains the independent crisis-text checks; and `core` contains configuration, security middleware, and service construction. Consequently, an HTTP route coordinates validated inputs and outputs without also containing the complete dialogue or persistence implementation.

Explicit interfaces further reduce coupling between these packages. The repository abstraction defines account, session, research-record, token, and dataset-freeze operations, with separate in-memory and MongoDB implementations. Interfaces for emotion analysis, cognitive analysis, strategy selection, and response generation similarly allow deterministic implementations, trained models, and provider-backed generators to be substituted without changing the conversation service's public contract. The application container selects the repository from configuration and constructs the authentication, conversation, multimodal, and transcription services once for the process.

The backend uses separate dependency files for the core runtime, optional local inference, and development tooling. The main `requirements.txt` file contains the web application, database, authentication, validation, and provider clients. The larger PyTorch, Transformers, audio, and numerical packages are added only through `requirements-inference.txt` when local trained-model inference is required. Test and static-analysis tools are isolated in `requirements-dev.txt`. This prevents a normal text-only backend installation from acquiring the substantially larger machine-learning runtime and makes the optional nature of multimodal inference concrete at the packaging level.

4.2.3 Persistence and External Services

MongoDB is the configured durable store for the integrated application. Its document representation accommodates a session containing ordered conversational turns together with dialogue states, policy decisions, feedback evidence, model metadata, and questionnaire records. The PyMongo-based repository adds ownership conditions to session operations so that a session identifier alone is insufficient to access another user's data. It also creates the indexes required for uniqueness, expiry, and research operations. Session and study-record retention are represented by separate expiry fields because the application workflow and the consented research dataset have different lifecycles.

An in-memory repository implements the same abstract interface. It is used by the normal automated backend suite and can support isolated development without a database process. The purpose of this implementation is behavioural substitution rather than durable storage: tests can exercise service logic quickly, while MongoDB-specific tests separately verify persistence across process restarts, index creation, expiry behaviour, cascade deletion, and concurrent-update protection. Selecting between the implementations through `PERSISTENCE_BACKEND` avoids conditional database logic throughout the services.

Network-backed functionality is accessed through narrow service boundaries. The OpenAI client may be configured for response realisation, feedback phrasing, moderation, and audio transcription. These calls are subject to timeouts and are not required for the deterministic role-play path. If credentials are absent, a service is disabled, a request fails, or an output does not satisfy the expected constraints, the relevant service reports the failure and the application uses its local response path. The email service follows the same configuration approach: SMTP enables account verification and password-recovery messages, while credentials and transport settings remain outside the source code.

The browser and API communicate through JSON over HTTP, with short voice samples represented only for the request that requires them. Local model inference loads private text and audio artefacts from configured directories rather than packaging those artefacts into the public repository. This arrangement keeps licensed data, model weights, credentials, and transient user audio outside version-controlled application code. The detailed persistence, privacy, and failure-handling controls are discussed later in this chapter.

4.2.4 Repository Structure and Configuration

The repository is organised around independently executable concerns. The `frontend` directory contains the React application, browser tests, and production web-server configuration. The `backend` directory contains the FastAPI application, dependency specifications, container definition, seed utilities, and backend tests. Model work is located under `ml`, with distinct packages for preprocessing, training, evaluation, and reproducible notebooks. Declarative experiment, fusion, baseline, and study-protocol settings are stored under `configs`. Supporting design records and data dictionaries are maintained under `docs`, repeatable command-line workflows are placed under `scripts`, and the dissertation sources are isolated under `thesis`.

This organisation separates source code from generated and restricted material. Raw datasets, checkpoints, experiment runs, local database material, virtual environments, Node modules, compiled frontend files, and environment files are excluded from version control. In contrast, model hyperparameters, label order, context length, audio limits, calibration parameters, protocol versions, and evaluation procedures are retained as configuration or documentation. A developer can therefore inspect the declared experimental and application settings without the repository distributing licensed datasets or private trained weights.

Application configuration is read through a typed Pydantic settings object. Values may be supplied through the process environment or through repository-level and backend-level `.env` files. The settings cover database selection, permitted origins and hosts, token lifetimes, retention periods, optional model paths, inference thresholds, provider timeouts, email delivery, study versions, researcher access, offline demonstration, and rate limits. Production-like environments additionally reject a weak signing secret, wildcard or non-HTTPS origins, wildcard hosts, disabled HTTPS enforcement, and disabled rate limiting. These checks cause unsafe deployment configuration to fail during application start-up rather than silently weakening the intended controls.

Two execution arrangements are supported. During local development, the frontend development server, backend server, and an existing MongoDB installation can be run independently. Alternatively, Docker Compose builds the frontend and backend and starts them with MongoDB 8, named database storage, dependency ordering, and health checks. The frontend image uses a Node.js 22 build stage followed by a small Nginx runtime image; the backend image exposes a readiness-based container health check. An offline demonstration configuration disables network-backed functions and provides deterministic behaviour for environments in which external credentials or connectivity are inappropriate.

Together, the directory boundaries, dependency separation, lock files, declarative configuration, and container definitions make the implemented application inspectable and repeatable. They also preserve a practical distinction between the core application, optional probabilistic services, research-only model development, and deployment-specific secrets. The following section describes the emotion-recognition pipeline that produces the local model artefacts consumed through this structure.

4.3 Emotion-Recognition Pipeline

The emotion-recognition implementation transforms the private IEMOCAP release into separate text and audio classifiers. It calibrates their outputs without using held-out test labels and packages the final artefacts for optional local inference. The pipeline is isolated from the conversational application under the `ml` directory. Preprocessing, fold training, calibration, fusion, statistical comparison, final-model packaging, and integrity checks can therefore be executed without involving participant accounts or application sessions.

Two stages must be distinguished. First, model choices are assessed through five speaker-independent folds. Each fold produces validation and held-out test predictions, and only these fold results are suitable for reporting performance. Second, after the experimental configuration and calibration parameters have been frozen, one text model and one audio model are trained on all eligible examples for application inference. These final full-data artefacts have no independent test split and their training metrics are not treated as evaluation evidence.

Figure 4.1 makes this separation explicit by placing held-out evaluation and final-model packaging on distinct paths.

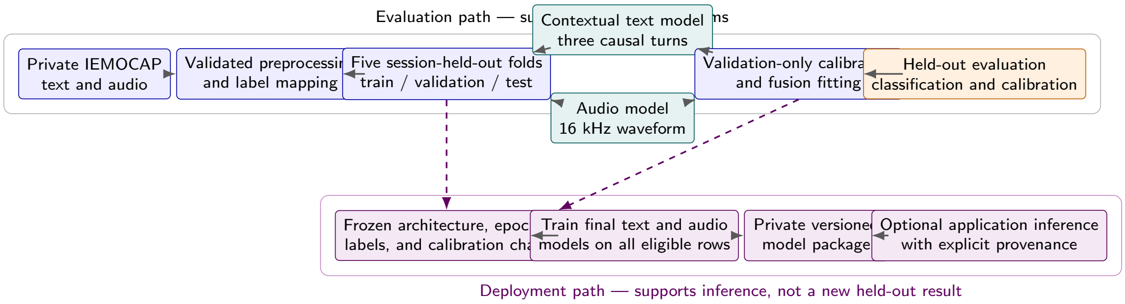

Evaluation path — su
ns
Contextual text model
three causal turns
Private IEMOCAP
text and audio
Validated preprocessi
and label mapping
Five session-held-out folds
train / validation / test
Audio model
16 kHz waveform
Validation-only calibra
and fusion fitting
Held-out evaluation
classification and calibration
Frozen architecture, epoc
labels, and calibration ch
Train final text and audio
models on all eligible rows
Private versione
model package
Optional application inference
with explicit provenance
Deployment path — supports inference, not a new held-out result

Figure 4.1: Emotion-recognition evaluation and deployment pipeline. The lower path packages a frozen configuration for inference but does not create a new held-out performance result.

4.3.1 Dataset Preparation and Label Mapping

IEMOCAP contains dyadic scripted and improvised sessions with aligned transcripts, sentence-level audio, categorical annotations, and dimensional ratings [BBL+08]. Access to the release is restricted, so the preprocessing code operates on a privately obtained archive and does not place raw media or row-level derived data in the repository. The text preparation command can extract only annotation and transcription files from the archive. It rejects absolute paths and parent-directory traversal before extraction, and the audio preparation command applies equivalent path validation when creating the private task-specific audio bundle.

Annotation and transcript lines are joined by the IEMOCAP utterance identifier. For every usable row, preprocessing records the session, speaker, dialogue, timing, original categorical label, valence, arousal, dominance, transcript, and the expected location of the corresponding sentence-level waveform. Duplicate categorical annotations and missing transcripts cause preprocessing to stop. The resulting manifest records the source archive metadata, source-label counts, task definitions, split composition, per-class counts, and a SHA-256 digest of the manifest contents.

Table 4.2 shows the implemented label mappings. The deployed pipeline uses the conventional four-class task: happiness and excitement are merged, while anger, neutrality, and sadness remain separate. This produces 5,531 unique paired utterances. The code also retains an exploratory six-class task aligned more closely with the application vocabulary. That task preserves frustration, maps IEMOCAP fear to the operational label *anxiety*, and renames the merged happiness/excitement class as *joy*. It is not used by the deployed multimodal service, in part because the source corpus contains too few fear examples to support a stable application-facing anxiety model.

Table 4.2: IEMOCAP source-label mappings implemented for the two experimental tasks.

IEMOCAP label	**Four-class benchmark**	**Exploratory six-class task**
`ang` (anger)	anger	anger
`hap` (happiness)	happiness	joy
`exc` (excitement)	happiness	joy
`neu` (neutral)	neutral	neutral
`sad` (sadness)	sadness	sadness
`fea` (fear)	excluded	anxiety
`fru` (frustration)	excluded	frustration

The fold construction uses complete recording sessions rather than randomly partitioning utterances. For test session $s \in \{1, \ldots, 5\}$, the preceding session is used for validation, with Session 5 preceding Session 1 cyclically, and the remaining three sessions form the training split. Because each of the ten actors belongs to exactly one IEMOCAP session, this arrangement keeps speakers disjoint across training, validation, and test data. Preprocessing explicitly computes the speaker sets and aborts if any overlap is detected. Across the five folds, each session appears once as held-out test data.

The private audio bundle contains exactly the sentence-level waveforms referenced by the four-class rows and names them only by utterance identifier. During creation, every expected file must be found. The script audits sample rates, channel counts, sample widths, durations, byte totals, source-archive identity, and the final bundle hash. The recorded corpus audit confirms that all 5,531 packaged files are mono 16 kHz, 16-bit PCM. Raw and processed data remain outside Git and the OneDrive-backed repository, and the bundle manifest is retained beside the private archive for integrity checking.

4.3.2 Contextual Text-Emotion Model

The textual classifier uses `microsoft/deberta-v3-small`, a compact DeBERTaV3 encoder whose pre-training combines disentangled representations with replaced-token detection and gradient-disentangled embedding sharing [HGC21]. A classification head maps the encoded input to logits in the fixed order anger, happiness, neutral, and sadness. The same training program also supports the exploratory six-class label order through configuration rather than a separate model implementation.

Two input formats are implemented. The utterance-only baseline receives the target transcript with a maximum token length of 128. The contextual model receives at most the preceding three turns from the same dialogue followed by the target utterance, with a maximum length of 256 tokens. Previous turns are ordered by their annotated start time and are marked relative to the target speaker:

$$x_i = [r_{i-k}]\, u_{i-k} \parallel \cdots \parallel [r_{i-1}]\, u_{i-1} \parallel [\text{target}]\, u_i, \tag{4.1}$$

where $k = \min(3, i-1)$, u_i is the target utterance, r_j is either `same speaker` or `other speaker`, and $\parallel$ denotes concatenation with a line separator. Future turns, affect labels, dimensional annotations, and dialogue outcomes are never included. The backend reproduces this format from the last three available session turns so that training and deployment use the same causal information boundary.

For each fold, the encoder and classification head are fine-tuned with seed 42. The primary configuration uses a learning rate of 2×10^{-5}, weight decay of 0.01, a 10% warm-up period, training batches of 16, evaluation batches of 32, and at most eight epochs. Validation occurs after each epoch, the checkpoint with the highest validation macro F1 is restored, and training stops after two consecutive epochs without improvement. Trainable weights begin in 32-bit floating point; mixed 16-bit computation is enabled when a compatible graphics processor is available. The four-class task uses ordinary cross-entropy, whereas the optional six-class configuration supports inverse-frequency weights clipped to a declared maximum of 10 to constrain the effect of rare categories.

Each run stores the selected model and tokenizer together with its task, label order, seed, split sessions, class weights, training history, software environment, validation and test metrics, calibrated probabilities, per-class measurements, row-level prediction files, and confusion matrix. The utterance-only and contextual experiments use the same fold membership and identifiers, permitting paired comparison by complete dialogue rather than treating correlated turns as independent samples. Quantitative comparisons between these configurations are reported in Chapter 5.

After the fold experiments were frozen, the final contextual text artefact was trained for seven epochs on all 5,531 unique benchmark rows. The epoch count was derived from the completed cross-validation training behaviour rather than selected from the final full-data loss. Before training, the packaging code reconstructs the complete dataset from one fold, rejects duplicate identifiers, and requires the expected row count. The resulting artefact is intended only for inference; its training metrics do not replace the speaker-independent fold evaluation.

4.3.3 Speech-Emotion Model

The acoustic classifier fine-tunes `facebook/wav2vec2-base`, which learns speech representations from raw waveform input through self-supervised pre-training [BZMA20]. The same four-class output order is used as for text so that probability vectors can be paired without remapping. Training and inference load audio from memory or private local storage; the application database never provides audio as model-training data.

Waveforms are read as 32-bit floating-point samples. A multi-channel recording, if encountered, is averaged to mono. Audio with a different sampling rate is converted to 16 kHz by polyphase resampling, and only the first 20 seconds are supplied to the model. Shorter examples are padded dynamically within each batch by the model's feature extractor. Before a fold begins, an audio audit verifies that every metadata row has a matching waveform and records the observed sample rates, maximum duration, and number of examples requiring truncation.

The convolutional feature encoder is frozen during fine-tuning, while the transformer and classification layers remain trainable. The configured learning rate is 1×10^{-5} with weight decay 0.01, 10% warm-up, a per-device batch size of four, and four gradient-accumulation steps, producing an effective batch of 16 before distributed effects. Gradient checkpointing reduces memory use. Training runs for at most ten epochs, evaluates once per epoch, restores the checkpoint with the highest validation macro F1, and applies the same two-epoch early-stopping patience as the text experiment. GPU execution uses mixed precision when available.

Audio runs write the trained feature extractor and classifier along with the fold, label order, seed, split sessions, audio audit, environment versions, training history, validation-calibration parameters, test distributions, per-class measurements, and confusion matrix. Validation and test identifiers must match those of the contextual text run before fusion can proceed. Following the fold experiments, the deployable audio artefact is trained for ten epochs on the same 5,531 unique examples used by the final text model. As with the final text artefact, this is a packaging run rather than a new evaluation.

4.3.4 Multimodal Late Fusion

Text and speech are combined through decision-level late fusion rather than by joining their hidden representations. This choice keeps each modality independently trainable, inspectable, and usable in ablation analysis, and it avoids retraining a joint network when the fusion policy changes [BAM19a]. Before any distributions are combined, the fusion program verifies the label order, exact utterance-identifier set, expected label, recorded fold number, and reconstruction of the saved single-modality metrics. It also rejects an utterance that appears in the test output of more than one fold.

Let z_t and z_a denote text and audio logits, and let T_t and T_a be their validation-fitted temperatures. The calibrated modality distributions are

$$p_t = \mathrm{softmax}\left(\frac{z_t}{T_t}\right), \qquad p_a = \mathrm{softmax}\left(\frac{z_a}{T_a}\right). \tag{4.2}$$

Given text weight w, the intermediate fusion is a convex combination,

$$p_m = wp_t + (1 - w)p_a. \tag{4.3}$$

The research comparison uses a predeclared equal-weight setting, $w = 0.5$, so that the hypothesis that audio contributes complementary information is not assessed using a weight selected from the held-out test predictions. A separate deployment-calibration path fits w over the fixed grid $\{0.00, 0.01, \ldots, 1.00\}$ using paired validation predictions and validation negative log-likelihood only. After choosing the weight, it fits a final scalar temperature T_f and calculates

$$p_f = \mathrm{softmax}\left(\frac{\log(\max(p_m, 10^{-12}))}{T_f}\right). \tag{4.4}$$

For the packaged full-data models, the deployable parameters were fitted from 5,531 unique pooled out-of-fold validation predictions. The frozen configuration contains $T_t = 1.8268$, $T_a = 1.3357$, $w = 0.59$ for text, $1 - w = 0.41$ for audio, and $T_f = 0.5989$. These values are applied unchanged by the backend. The held-out fold predictions are used to evaluate the frozen chain, not to choose its parameters. This preserves a distinction between the equal-weight confirmatory experiment and the calibrated configuration used when the application exposes or thresholds confidence.

4.3.5 Confidence Calibration and Uncertainty Handling

For each single-modality fold, temperature scaling fits one positive scalar on validation logits by minimising negative log-likelihood [GPSW17]. The implementation searches in log-temperature space and never uses the test session during fitting. Dividing every class logit by the same temperature changes the probability concentration without changing the class with maximum logit. Calibration is consequently evaluated separately from classification performance.

The evaluation utilities compute accuracy, macro F1, weighted F1, per-class precision, recall and F1, negative log-likelihood, multiclass Brier score, and expected calibration error. Expected calibration error partitions maximum confidence into fixed bins and accumulates the sample-weighted difference between mean confidence and empirical accuracy. Row-level outputs retain the complete probability vector rather than only the winning label, enabling later reliability analysis and exact pairing between modalities. Statistical comparisons use paired percentile bootstrap samples at dialogue level so that multiple turns from one conversation remain in the same resampled unit; those results belong to the evaluation chapter rather than the implementation account.

At inference time, the backend returns the calibrated text, audio, and fused distributions, each modality's selected label and confidence, and an explicit indicator of whether the two modalities agree. Fused confidence below 0.55 is labelled *low*, confidence from 0.55 up to 0.75 is labelled *moderate*, and confidence at or above 0.75 is labelled *high*. The adaptation policy uses the configurable 0.55 low-confidence boundary for the fused, textual, and acoustic confidences; the 0.75 boundary names the high presentation band but does not impose a further adaptation gate. These boundaries do not modify the model probabilities, turn a prediction into a verified emotion, or establish that estimates above them are correct.

Missing, invalid, conflicting, or low-confidence evidence is represented explicitly rather than converted into a stronger claim. The role-play workflow remains usable without audio or trained inference, and the dialogue policy and crisis checks do not depend on the predicted category. Calibration measured on IEMOCAP may deteriorate under changes in speakers, microphones, language, interaction style, or context [OFR+19]. The application therefore treats confidence and cross-modal agreement as bounded operational signals, not as guarantees of correctness.

4.3.6 Model Artefacts, Provenance, and Inference Loading

The final private package contains one contextual-text model directory and one audio-model directory. Each directory includes the saved classifier and the associated tokenizer or feature extractor. The versioned multimodal JSON configuration stores the label order, input field, maximum text length, sample rate, waveform-duration limit, model names, optimisation settings, and complete calibration chain. Model weights and licensed data are excluded from Git; a synchronisation script retrieves the final directories from private cloud storage into a user-selected local destination and prints the corresponding environment settings.

Provenance is captured after the private models are downloaded. The provenance command records the current source-control commit, SHA-256 of the multimodal configuration, Python platform, relevant library versions, and, for every file in both model directories, its relative path, size, and SHA-256 digest. It also calculates an aggregate directory digest from the ordered file entries. This manifest binds an inference result to specific model contents and code configuration without publishing the restricted artefacts themselves.

The backend keeps multimodal inference disabled by default. When enabled, it checks that both model directories and the configuration exist, then loads the tokenizer, feature extractor, and classifiers lazily on the first request. Device selection uses an available CUDA device when configured as automatic and otherwise uses the requested device or CPU. Models are placed in evaluation mode and executed under the framework's inference-only context. A process-level asynchronous lock serialises multimodal requests so that simultaneous users do not compete unpredictably for model memory; model execution is moved to a worker thread so that the asynchronous web loop remains responsive. Queue and inference times are returned separately.

Before inference, the service checks the configured byte limit, parses the in-memory WAV payload, requires a duration between 0.25 and 35 seconds, converts multi-channel audio to mono, resamples when required, and truncates model input to the configured 20-second maximum. Text context is constructed from the latest three causal turns using the same speaker-relative markers as training. The service then applies the stored modality temperatures, weighted fusion, and fusion temperature exactly as defined in Equations 4.2–4.4. Its response includes the distributions, agreement, confidence level, model version, latency, queue time, and an explicit statement that audio was not persisted.

Two forms of validation protect the packaged artefacts. Unit tests check causal context construction, confidence boundaries, disabled-service behaviour, and serialisation of concurrent requests without importing machine-learning libraries when inference is disabled. A private integrity smoke test selects an equal number of examples from each class, loads both final models, verifies finite logits and probabilities, runs the frozen fusion chain, and requires each component to predict more than one class. Any accuracy calculated on this balanced training sample is labelled as an integrity diagnostic and is not reported as evaluation performance. If loading or inference fails in the application, the failure remains contained within the optional service and the text-first rehearsal workflow continues through its deterministic path.

4.4 Backend Application

The backend application is the authoritative coordination layer between the browser, persistence store, dialogue policies, safety checks, trained models, and optional external providers. It exposes a JSON API below the `/api` prefix, but keeps transport concerns separate from the rules that govern a rehearsal. Consequently, neither the frontend nor a language-model response can directly alter dialogue state, feedback evidence, study status, or resource ownership. These changes are performed by typed backend services and persisted through a repository interface.

This section describes the general backend infrastructure and the lifecycle of a conversation request. The implementation of the role-play policy, feedback calculations, research workflow, and detailed security controls is developed in the subsequent sections.

4.4.1 Application and Service Architecture

The FastAPI application is assembled from four principal layers. API routes form the HTTP boundary and translate expected service failures into suitable status codes. Pydantic schemas validate transport data and constrain the response shape. Application services implement authentication, conversation coordination, transcription, multimodal analysis, preparation, branching, and research operations. Finally, repositories encapsulate all mutable storage. The domain models are shared between the service and repository layers, while external-provider clients remain behind dedicated service classes. This arrangement prevents route functions from becoming an alternative location for dialogue or persistence logic.

Service construction is centralised in a small application container. At process start, configuration selects either the MongoDB repository or its in-memory substitute, after which one authentication service, conversation service, multimodal service, transcription service, and email service is constructed. FastAPI dependencies provide these instances and the authenticated user to individual routes. The conversation service itself depends on interfaces for emotion analysis, cognitive analysis, strategy selection, response generation, and persistence. Default deterministic implementations can therefore be replaced without changing the route contract or the stored session representation.

The application lifespan initialises repository indexes before accepting normal work. In offline demonstration mode it also creates the local demonstration account if required; this mode is rejected by production configuration. Existing eligible sessions are then used to backfill any missing study records. A separate liveness endpoint confirms that the process is running, whereas the readiness endpoint validates configuration, checks the persistence connection and required indexes, and reports the availability of optional transcription and multimodal services. A missing optional model does not make the text-first application unready.

Cross-cutting HTTP behaviour is installed as middleware. Configured origins control credentialled cross-origin requests, trusted-host validation rejects unexpected host headers, and production deployments redirect HTTP traffic to HTTPS. Response middleware adds restrictive framing, content-type, referrer, permissions, caching, and content-security headers, together with HTTP Strict Transport Security in production. A configurable rate limiter applies separate windows to authentication, email, transcription, and model-backed operations. It is intentionally a single-instance implementation; a horizontally scaled deployment would require a shared limiter at a gateway or distributed store.

4.4.2 Domain Models and API Schemas

The domain layer represents application state as nested Pydantic models rather than unstructured dictionaries. Its enumerations constrain concepts such as speaker role, difficulty, practice goal, role-play status, support strategy, and user intent. A `Session` owns its ordered turns, current emotion state, optional role-play state, feedback, action card, questionnaires, research events, version, and expiry time. Each `ConversationTurn` may retain the emotion state observed at that point, selected strategy, response-generation metadata, and the bounded affect-pacing decision. More detailed role-play models preserve dialogue snapshots, evidence linked to source turns, reason codes, policy versions, branch lineage, and feedback comparisons. This makes the recorded session an auditable account of state transitions rather than only a transcript.

Transport schemas are deliberately distinct from these persistent entities. Request models impose limits before data reaches a service: messages are non-empty and limited to 5,000 characters, session titles to 80 characters, takeaways to 500 characters, and encoded audio to 7,000,000 characters. Enumerated fields constrain pacing preferences and difficulty settings, strict Boolean fields prevent ambiguous coercion for decisions such as eligibility confirmation, and selected request objects reject unknown properties. Responses expose only the fields needed by the relevant interface; for example, the public user response omits the password hash and token records held by the domain and repository layers.

Validation also protects relationships between fields. An enhanced chat exchange that requests audio analysis, affect adaptation, or non-default pacing must supply both a unique request identifier and the version of the session displayed by the client. Questionnaire schemas restrict ratings to their declared scales, and service validation checks that all required answers are submitted together or explicitly skipped. A multimodal estimate is accepted only when every probability is finite and within the unit interval, each distribution sums to approximately one, and the declared label and confidence agree with its maximum entry. Agreement and confidence-band fields are then derived again on the backend instead of being trusted from an external payload.

The domain models also distinguish operational metadata from user-facing content. Generated responses record their source, model, latency, token counts, and any fallback reason. Affect decisions record availability, user preference, inference timing, confidence, and the selected pacing action. Study records contain pseudonymous participant and protocol identifiers rather than copying the complete account. These explicit fields allow later analysis to separate deterministic and provider-backed behaviour, while preventing absent measurements from being interpreted as observed zero values.

4.4.3 Authentication and Account Management

Registration requires acceptance of the current privacy terms. Email addresses are normalised before storage and a unique database index prevents duplicate accounts. Passwords are processed by the password library's recommended one-way hashing scheme; plaintext passwords are never stored. A cryptographically random verification value is sent by email, while only its SHA-256 digest and expiry time are retained. Normal login and protected routes require a verified email address. Registration, verification resend, and password-recovery endpoints return deliberately generic messages so that their responses do not reveal whether a supplied address belongs to an account.

Successful authentication issues two tokens with different handling. The short-lived JSON Web Token used for API authorisation is returned to the browser and presented as a bearer credential. The longer-lived refresh token is placed in an HTTP-only, same-site cookie scoped to the authentication routes and marked secure in production. Its identifier and digest are stored server-side. Refreshing atomically removes the presented record and creates a replacement, so a token cannot be successfully rotated twice. Logout, password change, password reset, and account deletion revoke the user's outstanding refresh tokens; password-reset links are likewise digest-only, expiring, and consumed atomically after the replacement password has passed validation.

The dependency that resolves the current user decodes the access token, reloads the account, and checks email verification on every protected request. This means deletion or loss of eligibility is observed by later calls rather than being determined solely by claims in an old access token. Researcher routes add a second dependency that compares the verified account email with the configured researcher allow-list. Participant study enrolment is a separate state transition and does not implicitly grant researcher privileges.

Profile and lifecycle operations are handled through the same authenticated service boundary. Users can update the constrained profile fields, choose between one and three distinct practice goals during onboarding, change their password, sign out, or delete their account. Account deletion cascades through sessions, study records, refresh tokens, and outstanding verification and recovery records in both repository implementations. Study withdrawal has a narrower, protocol-specific meaning and is therefore implemented independently from deletion; the later research-workflow section describes this distinction in detail.

4.4.4 Session Persistence and Ownership Controls

The repository contract supplies identical account, session, token, study-record, lifecycle, and frozen-export operations for the in-memory and MongoDB implementations. The in-memory implementation supports isolated development and fast unit tests; the MongoDB implementation supplies durable storage for the integrated application. Services depend only on this contract, so choosing a persistence backend does not introduce database-specific branches into conversation processing.

Session ownership is enforced as part of repository queries. Reading or deleting a session requires both its identifier and the authenticated user's identifier, and session listings are filtered by the same user key. Expired sessions are excluded from reads, while a MongoDB time-to-live index removes them according to their `expires_at` value. Thus, knowing another session's universally unique identifier is insufficient to retrieve or mutate it. Service methods translate a failed owned lookup into the same not-found result, avoiding a distinction between a missing resource and one owned by somebody else.

Every session also carries a monotonically increasing version. When the enhanced chat interface submits an exchange, it includes the displayed version and a request identifier. The conversation service rejects a repeated request identifier or stale version before modifying a deep copy of the session. At the storage boundary, MongoDB replaces a document only when its stored version equals the service's expected version, then increments it; an insert collision or failed conditional replacement becomes a concurrent-update error. The in-memory repository implements the same comparison. The global exception handler returns HTTP status 409 with instructions to reload, preventing two tabs or overlapping requests from silently overwriting one another.

MongoDB indexes additionally enforce unique email, session, study-record, protocol, and frozen-export identifiers, support per-user chronological queries, and expire session, token, and study records at their respective deadlines. Readiness checks verify these indexes rather than assuming they were created. Conversation and consented research data remain separate: a session contains the complete interaction needed by the application, whereas the synchronised study record contains protocol-bounded events, measures, generation counts, and feedback metrics under its own retention rule. The later research-workflow and data-protection sections describe the detailed minimisation and withdrawal behaviour.

4.4.5 Conversation Request Processing

The `/api/chat` endpoint is intentionally a thin coordinator. After schema validation and authentication, it delegates the message, account identity, optional audio, affect preference, request identifier, and expected session version to the conversation service. The service first performs the owned session lookup and the duplicate and stale-version checks described above. If necessary, the first message supplies a shortened default title; beginning a new exchange also closes an unused immediate post-questionnaire opportunity so that questionnaire timing cannot drift into later conversation.

Safety evaluation precedes affect adaptation and response generation. A local crisis-language check always runs, and provider moderation may add a flag when the provider is configured. If a crisis is detected, optional multimodal inference is skipped, the fixed safety response takes precedence, and an active or paused rehearsal is marked as interrupted. The trained affect model is therefore neither a prerequisite nor an authority for crisis handling.

For a non-crisis exchange, multimodal inference is attempted only when the request is bound to a version and request identifier, audio is present, a configured model is available, and the rehearsal is eligible for experimental adaptation. It is deliberately excluded from required study tasks so that the required protocol path is not altered by this optional condition. Decoding, validation, or inference failure is caught and recorded as unavailable rather than aborting the message. Independently, the local text analyser updates the smoothed emotion state, the cognitive analyser derives tentative intent and readiness features, and the scored selector chooses a support strategy with its component scores and reason codes.

The user turn is appended before reply selection. During an active rehearsal, the deterministic role-play engine advances the dialogue state, links observed evidence to that turn, and supplies both a required character action and safe fallback wording. Outside role-play, the selected support strategy guides the general response generator. An affect-pacing prefix may be added only through the bounded pacing policy and only under its eligibility conditions. The resulting assistant turn stores generation and affect-decision metadata; role-play decisions additionally store before-and-after snapshots, evidence identifiers, policy versions, and reason codes.

Finally, the service records a minimised completion event, updates measurement boundaries when a rehearsal ends, persists the incremented session, and synchronises an eligible study record. The response returns the committed version together with the user and assistant turns, decision summary, current role-play state, any completed feedback, and the one-use opportunity token required for an immediate post-rehearsal questionnaire. This single service path keeps the dialogue shown by the client aligned with the state and evidence retained by the backend.

The ordering and alternative paths are summarised in Figure 4.2. In particular, the safety branch terminates ordinary processing before optional affect inference and response generation.

4.4.6 Optional Services and Deterministic Fallbacks

Network-dependent and model-dependent functions are implemented as enhancements rather than foundations of the rehearsal workflow. When an OpenAI API key is present, the response generator can realise a selected support strategy in natural language, moderate text, phrase evidence-based feedback, or render a role-play action. Requests use a configured timeout and disable provider-side response storage. If credentials are absent, the service is disabled, a call raises an exception, or a response is empty, refused, or structurally invalid, generation returns the corresponding deterministic template and records the fallback reason.

Provider output is constrained differently according to its purpose. General support generation receives the authoritative emotion state and selected strategy together with instructions against diagnosis, medication advice, certainty claims, and dependency. Role-play realisation receives the authoritative scenario, difficulty behaviour, state, and required action, and must return a typed action and a short in-character utterance. Outputs that expose coaching language or disagree with the required action are discarded. Where the enhanced dialogue policy contains a concrete negotiated constraint or proposal, exact deterministic wording remains in force until semantic validation can reliably establish that a rephrasing preserves it. Feedback generation is limited to rewording strengths and suggestions that have already been calculated from recorded evidence; it cannot create new observations or scores.

Transcription and trained multimodal inference use separate authenticated endpoints and report explicit unavailability when invoked directly without configuration. In the integrated chat path, multimodal failure is contained and the deterministic exchange continues. Audio is size-checked and decoded in memory for the immediate operation, and service responses state that it was not persisted. Email delivery is similarly isolated behind its service boundary; generic account responses remain stable when delivery fails, without falsely granting verification or recovery access.

Each generated turn records whether its content came from a template, safety response, deterministic role-play path, or configured provider, together with latency and fallback information where available. Health and model-information endpoints expose capability status without treating an optional dependency as a core outage. These choices preserve an operational baseline in offline demonstrations, tests, and provider failures, while making the provenance of enhanced behaviour visible for debugging and later evaluation.

4.4.7 Rule-Based Cognitive Features and Support Routing

The class named `RuleBasedCognitiveAnalyzer` is a deterministic phrase matcher applied to the current text; "cognitive" is an implementation name rather than a claim that the component performs cognitive or clinical assessment. It checks ten authored regular-expression families labelled mind-reading, catastrophising, overgeneralisation, all-or-nothing thinking, should statements, emotional reasoning, fortune-telling, personalisation, discounting positives, and labelling. A matching family receives a fixed rule weight and the highest-weight family is returned as a "possible" pattern. The number is neither a learned probability nor a severity score. The rules have no access to clinical history, speaker identity, longitudinal belief patterns, or professional annotations.

The remaining fields are equally mechanical. Intent follows a fixed precedence: explicit rehearsal terms, practical-help terms, emotion or validation terms, a question mark, and otherwise unclear. "Readiness for advice" is assigned one of four constants according to those same matches; it is not estimated from observed acceptance of advice. "Resistance" is 0.7 for four explicit pushback phrases, 0.15 for any other occurrence of "but", and zero otherwise. "Possible cause" copies up to 100 characters following *because*, *after*, *when*, or *since*. Boolean request flags repeat the rehearsal, practical-help, and validation matches. These definitions make the output reproducible but also expose obvious failure modes involving negation, quotation, sarcasm, polysemy, indirect requests, multiple clauses, and dialectal or multilingual wording.

Outside active role-play, `ScoredStrategySelector` converts the heuristic record and lexical affect state into additive priorities for validation, reflection, clarification, validate–reframe, practical suggestion, rehearsal invitation, de-escalation, and safety escalation. For example, an explicit rehearsal phrase favours an invitation to practise; a practical-help match with its associated fixed readiness value favours a suggestion; a cognitive-pattern match favours validate–reframe; and one of the four strong pushback phrases favours reflection while reducing the suggestion score. The maximum total selects the strategy supplied to the response generator. These strategy scores express rule priority, not the probability that a response is suitable or effective.

The subsystem has deliberately limited authority. Crisis handling is evaluated first and forces the safety strategy. During active role-play, the scenario controller selects the character action, so cognitive tags cannot advance a stage, declare success, or alter feedback scores. The subsystem does affect general supportive responses and its intent, possible pattern, possible context, selected strategy, and reasons can be displayed in the "Why this response?" panel. That visibility is why the feature is documented here even though it is secondary to the dissertation's emotion-recognition and rehearsal contributions. Its evidence base is limited to rule and integration tests reported in Section 5.6.2; no corpus-level predictive validity, clinical validity, or outcome benefit has been established.

4.5 Role-Play and Adaptation Engine

The role-play engine implements the central distinction established in the system architecture: deterministic code selects the permitted conversational action, while a generator may only realise that action in words. It therefore treats scenario progress, evidence, completion, safety, and feedback as application state rather than delegating them to a language model. The same state can be serialised, restored, replayed, rewound, and tested without a network connection.

4.5.1 Scenario Representation and Configuration

A role-play scenario is represented by an identifier, title, character, situation, user objective, opening line, expected skills, difficulty-specific behaviour, success conditions, and a maximum number of turns. The repository contains six standard scenarios: workload prioritisation, maintaining a personal boundary, expressing a relationship need, giving feedback to a colleague, negotiating a deadline, and sharing household responsibilities. The first three form the standardised study sequence; the remaining scenarios broaden ordinary practice without changing the study protocol.

Difficulty is an explicit enumeration with beginner, intermediate, and difficult levels. At initialisation these levels map to increasing challenge and decreasing cooperation values, but the selected level does not change the meaning of a recorded communication feature. For standard unprofiled practice, difficulty primarily controls how much resistance the character provides and, for the boundary scenario, how many clear refusals must be maintained. Every new attempt stores its scenario, policy, and scoring versions so that later feedback and comparisons are interpreted under the rules that produced them.

Additional practice in the workload, boundary, and relationship scenarios may also use one of three authored character profiles. The cooperative profile asks for a clear practical response, the rushed profile prioritises concise decisions and a workable time, and the sceptical profile questions the proposal and may repeat pressure. These profiles select versioned deterministic policies rather than psychological personalities. They are unavailable for required study tasks and custom scenarios, preventing an optional experimental path from modifying the frozen protocol.

Users can create account-owned custom scenarios directly or through the preparation workflow. Custom inputs are normalised and limited in length, and their target skills must come from the supported observable set: clear request, specific detail, boundary maintenance, I-statements, and non-blaming language. A prepared scenario additionally stores the difficult part identified by the user and a tentative likely objection produced by a bounded local template. The template is presented for review and editing; it is not claimed to predict another person's response.

4.5.2 Dialogue State and Transition Logic

The role-play state stores status, current stage, turn count, difficulty, cooperation, accumulated evidence, decisions, outcome, completion reason, and timestamps. It supports active, paused, completed, and interrupted states. Pausing permits reflection without advancing the rehearsal, resuming returns to the existing controller state, and manual completion remains distinct from automatic success. The maximum of eight user turns prevents an unresolved simulation from continuing indefinitely.

The enhanced scenario policy represents each supported conversation as a small state graph. Workload practice advances from explaining the problem, through addressing constraints, to agreeing a plan. Boundary practice progresses from an initial refusal, through pressure, to respectful closure. Relationship practice moves from describing the situation and need, through acknowledging the other perspective, to selecting a practical next step. A transition occurs only when the observable evidence required at the current stage has appeared; unsupported or contradictory input leaves the controller in place and produces a targeted follow-up.

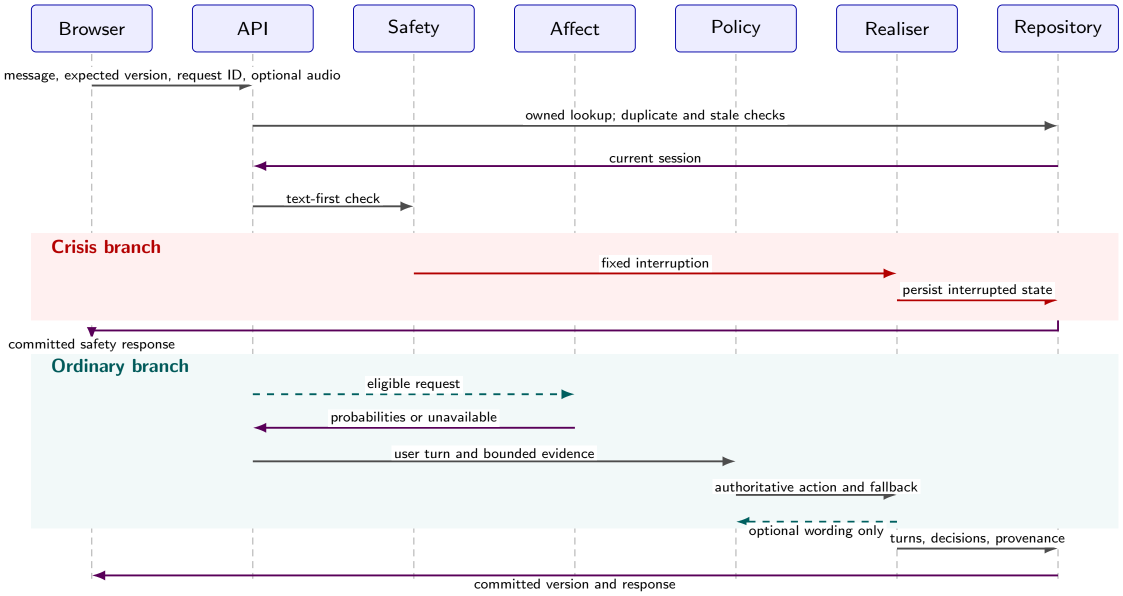
The three graphs contain scenario-specific checks. For example, the workload path requires the user to acknowledge the stated delivery constraint while proposing a supported trade-off before agreement can be confirmed. The boundary path distinguishes a maintained refusal from a reversal or conditional acceptance and varies the permitted pressure rounds by difficulty and profile. The relationship path requires a specific personal need, acknowledgement of perspective, and a feasible request; unsupported or explicitly unresolved endings remain recorded as such. These rules model the authored practice exercise, not the probable behaviour of a real manager, friend, or partner.

Figure 4.3 presents the principal state progression for the three standardised scenarios and the explicit non-success exits shared by the controllers.

Each processed rehearsal message produces a decision record containing the before-and-after dialogue snapshots, selected character action, reason codes, linked user and assistant turns, supporting evidence identifiers, generation metadata, character profile, and scenario, policy, and scoring versions. Completion reasons distinguish success, the turn limit, manual completion, and safety interruption. Persisting these transitions makes recovery exact: a restored session does not need to infer its stage again from the transcript.

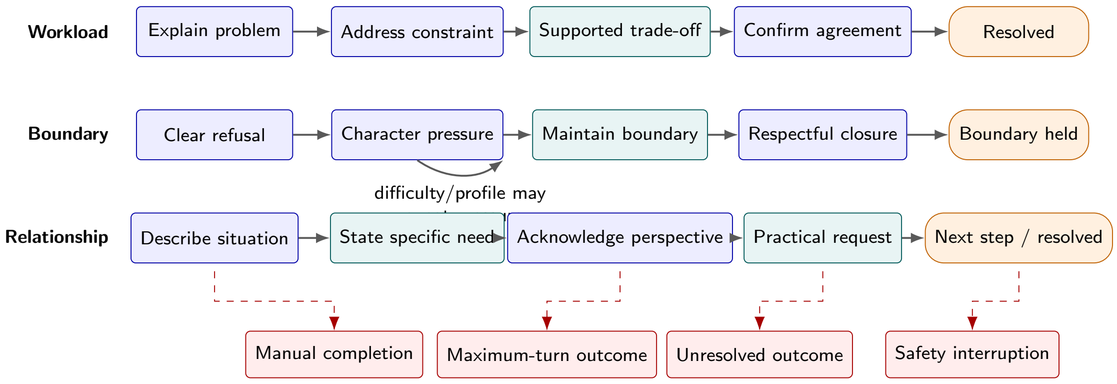
4.5.3 Communication-Evidence Extraction

Evidence extraction operates on the submitted text and records bounded, inspectable language features. The common extractor identifies patterns associated with a concrete request, repeated apology, maintained boundary, blame phrasing,


Browser
API
Safety
Affect
Policy
Realiser
Repository
message, expected version, request ID, optional audio
owned lookup; duplicate and stale checks
current session
text-first check
Crisis branch
fixed interruption
persist interrupted state
committed safety response
Ordinary branch
eligible request
probabilities or unavailable
user turn and bounded evidence
authoritative action and fallback
optional wording only
turns, decisions, provenance
committed version and response

Inference and generation fail softly; authority, ownership, and safety fail closed.

Figure 4.2: Sequence of a version-bound conversation request, including safety precedence and failure-tolerant optional processing.


Workload
Explain problem
Address constraint
Supported trade-off
Confirm agreement
Resolved
Boundary
Clear refusal
Character pressure
Maintain boundary
Respectful closure
Boundary held
difficulty/profile may
Relationship
Describe situation
State specific need
Acknowledge perspective
Practical request
Next step / resolved
Manual completion
Maximum-turn outcome
Unresolved outcome
Safety interruption

Transitions require stage-appropriate observable evidence; unsupported input remains in the current state.

Figure 4.3: Principal dialogue-state progression for workload, boundary, and relationship rehearsal.

specific detail, and I-statements. Versioned scenario extractors add features such as a problem description, workable option, check-in, clear refusal after pressure, respectful closure, acknowledgement of perspective, and practical request. Negation, conditional language, reported speech, and contradictory clauses are handled for the specific patterns required by the authored dialogue paths.

Each evidence item records its rehearsal number and source-turn identifier. It also contains the extracted feature names and the text analyser's arousal value. It does not store an invented quotation span. The original utterance remains the authoritative source, and replay highlights the complete linked turn when exact spans were not recorded. Features may accumulate across exchanges where a scenario permits this, but completion is still controlled by stage order; mentioning an agreement before the character has raised its constraint cannot skip the intervening dialogue.

The rules intentionally favour auditability over unrestricted semantic coverage. Indirect refusals, sarcasm, unusual paraphrases, and complex multi-clause statements may be missed. A detected phrase is therefore reported as observable evidence in this rehearsal, not as proof of communication competence, intention, personality, or likely real-world success. The deterministic test fixtures include supported, negated, reversed, irrelevant, and ambiguous examples to keep this boundary visible.

4.5.4 Affect-Aware Pacing Policy

The affect-pacing policy is a separate versioned layer applied after the dialogue controller has selected its response. It never changes the scenario stage, profile, difficulty, success conditions, evidence, or feedback score. Automatic use is disabled by default, and the participant may instead request that the exchange keep going unchanged, use a gentler pace, or add more challenge. An explicit choice takes precedence over automatic inference and can operate without audio.

Automatic adaptation is restricted to active, enhanced, non-required rehearsals. Required study tasks, associated retries, ordinary reflection, legacy sessions, paused or completed attempts, and ineligible scenarios use the baseline response. Safety and completed-exchange conditions take precedence. The policy also refuses to translate model categories into anxiety, readiness, resistance, or other psychological constructs; the existing lexical state tracker remains independent.

When automatic adaptation is eligible, missing audio, unavailable models, inference failure, or unsupported categories produce no addition. If any fused or component confidence is below the stored low-confidence threshold, or if the fused, text, and audio labels do not all agree, the response neutrally offers a pacing choice. When all three confidences meet that threshold and all three labels agree, anger or sadness adds only the phrase "I hear you."; agreement on happiness or neutral produces no addition. Thus, a matching estimate in either the moderate or high presentation band can reach the category branch. The user-selected gentler and more-challenge modes use fixed prompts. The full decision, reason, source, model version, timings, preference, and probability summaries are stored with the assistant turn, allowing the interface to explain the exact action without reconstructing it later.

Figure 4.4 expresses the same policy as a decision tree and highlights the precedence of safety, eligibility, and explicit user choice over automatic inference.

4.5.5 Response Realisation

Response realisation begins with the authoritative action and deterministic wording returned by the controller. In offline operation this wording is sent directly. When provider generation is enabled for a supported unbranched practice flow, the language model receives the saved scenario, difficulty behaviour, current role-play state, affect estimate, required action, and fallback wording. Its structured response must repeat the required action exactly, remain in character, avoid coaching and hidden-system language, and stay within a short response limit.

Failure to parse the response, an action mismatch, disallowed coaching language, an exception, or unavailable credentials returns the deterministic wording. For enhanced dialogue states containing a concrete constraint or proposal, even a plausible paraphrase is rejected unless it matches the authoritative wording; this conservative rule avoids treating a correct action label as evidence that a generated sentence preserved the negotiation facts. Branches likewise use deterministic wording so their differences arise from the participant's alternative response rather than an uncontrolled regeneration.

Outside role-play, the support-routing component described in Subsection 4.4.7 supplies one of the permitted general-response strategies. The response generator receives the selected strategy but cannot select it. Generation metadata records whether wording came from a template, deterministic role play, the provider, or a safety response, ensuring that later study summaries can count response sources and fallbacks without retaining provider internals.

4.5.6 Safety Precedence and Crisis Interruption

A local crisis-language detector executes before model inference and response generation. Its bounded patterns cover direct self-harm and immediate-danger statements while applying exclusions for common negations, hypothetical or quoted material, and known false-positive phrases. When an external moderation service is configured, it can add a safety flag but cannot suppress a local match. This remains a conservative engineering safeguard rather than a validated clinical risk assessment.

On a detected safety condition, the backend skips multimodal inference and pacing, emits the fixed crisis response, and changes an active or paused role play to interrupted with the completion reason `safety_interruption`. The decision trail preserves the preceding dialogue state and records the interruption action. Safety-interrupted wording is never reused as a suggested action-card phrase, and the research workflow does not count the attempt as a completed protocol task.

The response states the application's limitations and directs the user to emergency or crisis support without claiming to know the user's location or contacting services on their behalf. Because the deterministic safety path precedes provider and classifier output, failure of either optional component cannot remove the interruption. False negatives and ambiguous statements remain possible, which is why the interface consistently states that AffectLab is not monitored care, diagnosis, or emergency support.

4.6 Feedback and Reflection Features

Feedback features are constructed from the evidence and decisions already stored during a rehearsal. They do not rerun an affect model, infer hidden reasoning, or ask a generator to judge the participant after completion. This design permits the user to move from a summary into the exact conversation evidence, try a bounded alternative, and prepare personal wording while preserving the original attempt.

4.6.1 Evidence-Grounded Feedback Generation

At completion, the role-play service calculates scenario-specific metrics from Boolean evidence attached to the participant's turns. Examples include clear request, specific evidence, maintained boundary, concise delivery, I-statements, and non-blaming language. A metric score is the proportion of relevant turns in which its condition was observed, and its evidence list contains the corresponding turn numbers. Versioned dialogue policies add stage-specific feedback while retaining the same link to stored evidence.

Deterministic templates turn these measurements into observed statements, strengths, and suggestions. A feature at or above the configured descriptive boundary becomes a strength; an absent or infrequent feature becomes a next-attempt suggestion. The completion reason and controller outcome remain separate from individual language metrics: reaching the turn limit is not relabelled as success, and a safety interruption is not scored as a skill failure. Custom scenarios receive metrics only for their selected supported skills.

If provider phrasing is enabled, it receives only the completed evidence-based feedback object and may reword its strengths and suggestions. The numeric scores, evidence links, and observations remain authoritative, and generation metadata distinguishes deterministic feedback from provider wording. Comparative feedback is added only when an earlier ordinary attempt uses the same scenario and scoring version; changes are described as differences in observed language rather than personal or clinical improvement.

4.6.2 Conversation Replay

Replay is a read-only projection of the stored session rather than a second analysis pass. Its timeline is bounded by the role-play start and measurement-end timestamps, with a compatibility rule for older sessions that recorded only completion time. Later reflection is excluded, while reflection entered during a pause remains visible but has no controller decision and is not shown as a scored exchange. Missing boundaries, legacy decisions, unavailable feedback, and inference failures are displayed as gaps instead of being reconstructed.

Selecting an exchange highlights the paired user and assistant turns. Separate panels show observable language features, controller action and stage transition, generation source, and any saved affect-pacing decision. Technical details expose reason codes, before-and-after state, policy versions, modality distributions, confidence and agreement, availability, and timings. Feedback evidence buttons resolve stored evidence numbers back to their conversation-turn identifiers. Since exact phrase spans are unavailable, the complete source utterance is highlighted rather than presenting a guessed substring.

The browser constructs this view from the normal ownership-protected session response; replay has no new endpoint and makes no writes. It does not reload audio, regenerate explanations, or mutate recorded decisions. Navigation away from an unanswered immediate post-questionnaire may close that questionnaire window under the study rules, but inspection of the replay itself remains read-only.

4.6.3 Rewind, Branching, and Comparison

Rewind removes only the most recent linked user–assistant rehearsal exchange. For versioned dialogue it restores the exact preceding snapshot, truncates the matching evidence and decision lists, clears completion and feedback, restores the previous affect state, and removes later research events. It is refused when the latest messages cannot be linked safely, when they belong to copied branch context, or after post-ratings or an explicit skip have fixed the measured attempt. The removed user text is returned to the composer for editing.

Branching provides a copy-on-write alternative without modifying the original. It is available only for completed or interrupted attempts whose stored policy and pre-turn snapshot are supported. The server validates ownership, source version, branch-point turn, linked decision, policy versions, and intact evidence. It then creates a deterministic child identifier from the user and request identifiers, copies only the context before the selected response, restores the saved pre-turn state, and records lineage including the parent version, copied turn identifiers, branch group, and SHA-256 digest of the source rehearsal.

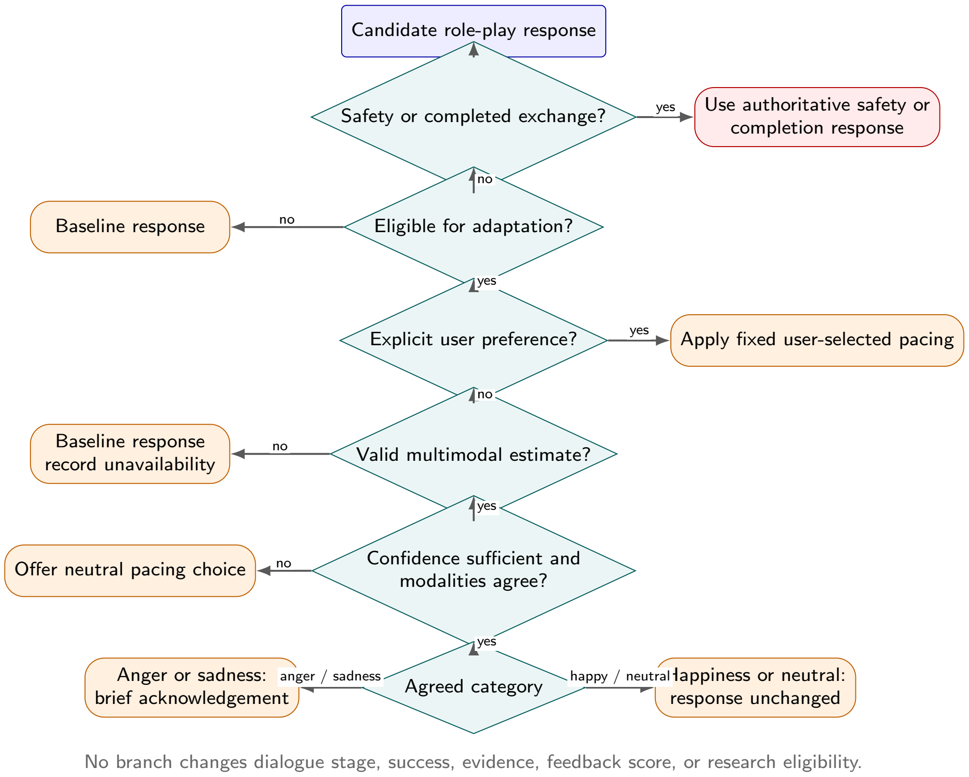
The child is labelled as a retry with no required-study association. Its feedback covers only newly authored continuation evidence; copied context is not counted again in turn totals, events, or generation statistics. Comparison presents the shared context once and the original and alternative continuations side by side, including actions, evidence, sources, and outcomes. If the parent has been deleted, expired, or changed since the branch was created, the saved alternative remains usable but the original continuation is explicitly marked unavailable. This prevents a silently altered source from being presented as a controlled comparison.

Figure 4.5 illustrates the copy-on-write branch and the distinction between shared context and newly assessed continuation turns.

4.6.4 Takeaways and Action Cards

The feedback screen provides a 500-character personal takeaway that is stored with the session and surfaced on the home dashboard. It is entirely user-authored and remains distinct from measured feedback. The dashboard also aggregates prior feedback metrics as a practice history, accompanied by language that these are averages of observed rehearsal features rather than a clinical assessment.

After a completed or interrupted rehearsal, the application can draft an action card containing an opening sentence, main request, boundary or fallback, and reminder. The draft draws only from eligible participant turns within the rehearsal measurement boundary and stores the identifiers of any reused source turns. It uses a maintained-boundary utterance where available, otherwise supplies neutral


Candidate role-play response
Safety or completed exchange?
yes
Use authoritative safety or completion response
no
Eligible for adaptation?
no
Baseline response
yes
Explicit user preference?
yes
Apply fixed user-selected pacing
no
Valid multimodal estimate?
no
Baseline response record unavailability
yes
Confidence sufficient and modalities agree?
no
Offer neutral pacing choice
yes
Agreed category
anger / sadness
Anger or sadness: brief acknowledgement
happy / neutral
Happiness or neutral: response unchanged

No branch changes dialogue stage, success, evidence, feedback score, or research eligibility.

Figure 4.4: Decision tree for bounded affect-aware pacing. Every terminal action preserves the authoritative dialogue state and evidence.

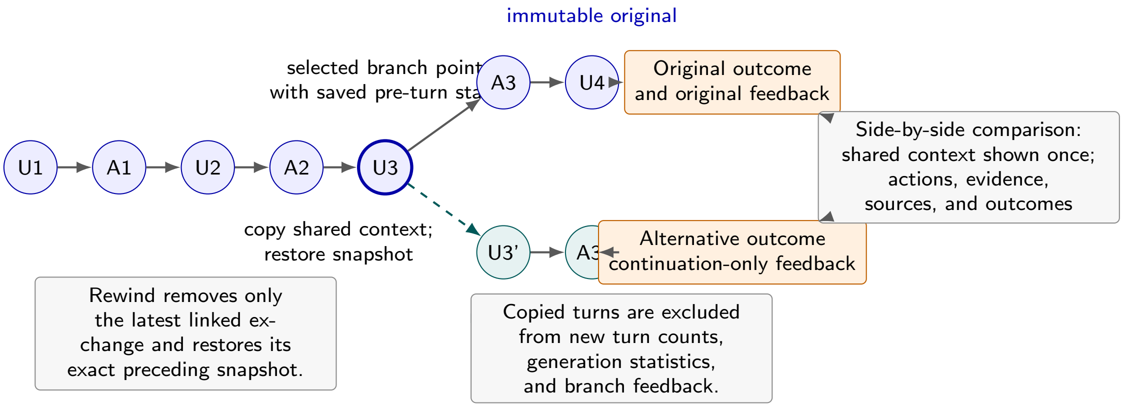

immutable original
selected branch point with saved pre-turn sta
U1
A1
U2
A2
U3
A3
U4
Original outcome and original feedback
Side-by-side comparison: shared context shown once; actions, evidence, sources, and outcomes
copy shared context; restore snapshot
U3'
A3
Alternative outcome continuation-only feedback
Rewind removes only the latest linked exchange and restores its exact preceding snapshot.
Copied turns are excluded from new turn counts, generation statistics, and branch feedback.

Figure 4.5: Replay, rewind, and copy-on-write branching from a stored pre-turn dialogue snapshot.

fallback wording. Safety-interrupted utterances and later reflection are excluded from suggested real-world phrasing.

The user can edit every card field before saving. Saving requires the expected session version, marks the card as user-edited, and updates neither feedback, questionnaires, research events, nor measurement boundaries. Cards can be copied or downloaded by the browser and survive normal session reloads. In a branch they use new continuation turns rather than silently recycling shared context, preserving the distinction between the original and alternative plans.

4.7 Frontend Application

The frontend presents the backend's typed state as a sequence of participant and researcher workflows. It does not calculate authoritative dialogue transitions, affect decisions, or study completion in the browser. Instead, components retain editable local state, submit constrained requests, and replace their view with the version returned by the server. This keeps recovery behaviour and uncertainty disclosures as important as the successful path.

4.7.1 Application Interface Overview

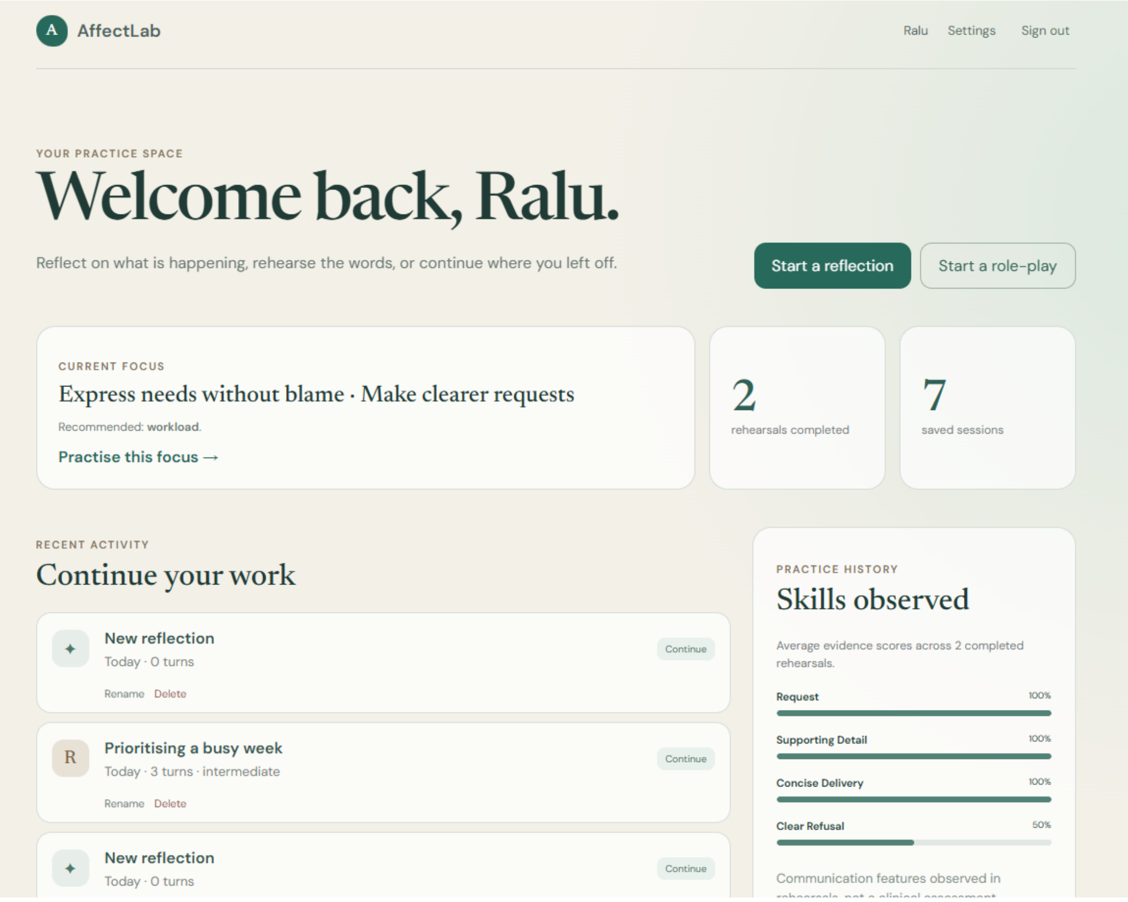
Figures 4.6, 4.7, and 4.8 provide a compact visual overview of the participant-facing application. They follow the main user journey from choosing a practice activity, through an active rehearsal, to reviewing the resulting feedback. The screenshots should be captured with synthetic demonstration content and without personal or participant data. Only these three representative views are included because the purpose is to show the appearance and principal interactions of the implemented system, while the following subsections describe its behaviour in detail.

4.7.2 Application Navigation and User Workflow

The React application uses the browser History API to provide public landing and information pages, registration, sign-in, email verification and recovery, onboarding, the authenticated home dashboard, the practice workspace, settings, and the restricted researcher dashboard. On initial load it attempts to restore the access session and then performs one refresh-token exchange if necessary. Unauthenticated access to a protected route returns to sign-in, while a verified user who has not completed the current onboarding version is directed through onboarding before reaching the workspace.

Onboarding explains that the application is not therapy or diagnosis, that affect estimates can be wrong, that voice is optional, and that emergency support is outside the service. The user selects between one and three practice goals, which are stored on the account and used to recommend a starting scenario without restricting the available scenarios. The home dashboard then offers reflection, guided role play, recent sessions, saved takeaways, and a descriptive history of observed skills. Sessions can be resumed, renamed, or deleted.

The application shell includes explicit loading, empty, success, and failure states. An error boundary provides a recoverable sign-in path if a component cannot render. Network operations disable only the relevant control, expose errors through alert


A
AffectLab
Ralu
Settings
Sign out

YOUR PRACTICE SPACE

Welcome back, Ralu.

Reflect on what is happening, rehearse the words, or continue where you left off.

Start a reflection
Start a role-play

CURRENT FOCUS

Express needs without blame · Make clearer requests

Recommended: **workload**.

Practise this focus →

2 rehearsals completed

7 saved sessions

RECENT ACTIVITY

Continue your work

New reflection
Today · 0 turns
Continue
Rename
Delete

Prioritising a busy week
Today · 3 turns · intermediate
R
Continue
Rename
Delete

New reflection
Today · 0 turns

PRACTICE HISTORY

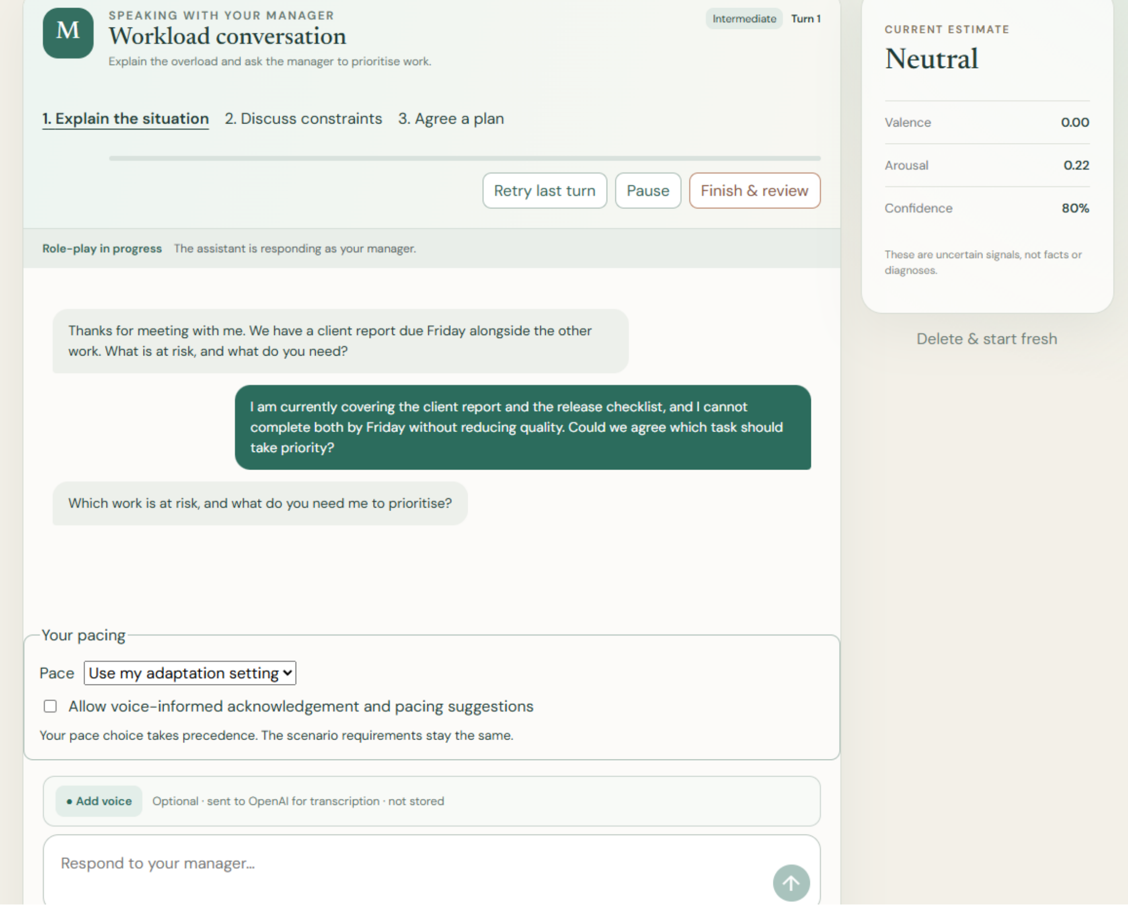
Skills observed

Average evidence scores across 2 completed rehearsals.

Request	100%
Supporting Detail	100%
Concise Delivery	100%
Clear Refusal	50%

Communication features observed in

Figure 4.6: AffectLab home dashboard, showing the available practice paths, recommended starting point, and access to recent sessions and saved reflections.



M
SPEAKING WITH YOUR MANAGER
Intermediate
Turn 1
1. Explain the situation
2. Discuss constraints
3. Agree a plan
Retry last turn
Pause
Finish & review
Add voice
CURRENT ESTIMATE
Delete & start fresh

Workload conversation

Explain the overload and ask the manager to prioritise work.

Role-play in progress The assistant is responding as your manager.

Thanks for meeting with me. We have a client report due Friday alongside the other work. What is at risk, and what do you need?

I am currently covering the client report and the release checklist, and I cannot complete both by Friday without reducing quality. Could we agree which task should take priority?

Which work is at risk, and what do you need me to prioritise?

Your pacing

Pace: Use my adaptation setting

☐ Allow voice-informed acknowledgement and pacing suggestions

Your pace choice takes precedence. The scenario requirements stay the same.

Optional · sent to OpenAI for transcription · not stored

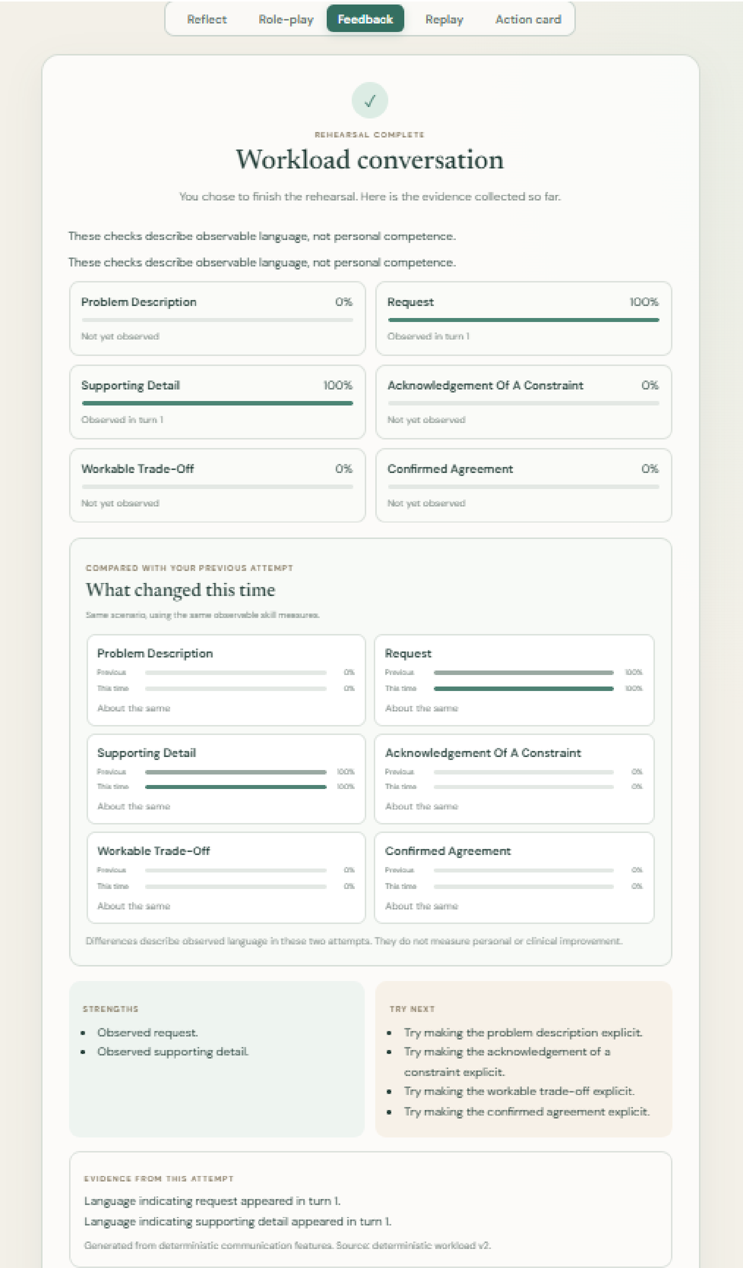
Respond to your manager...

Neutral

Valence	0.00
Arousal	0.22
Confidence	80%

These are uncertain signals, not facts or diagnoses.

Figure 4.7: Active role-play workspace, combining the conversation transcript and message composer with scenario progress, session controls, and optional voice and affect-aware functions. The standard affect panel shown here is the lexical estimate; a trained voice–text result, when available, appears in a separate panel.



REHEARSAL COMPLETE

Workload conversation

You chose to finish the rehearsal. Here is the evidence collected so far.

These checks describe observable language, not personal competence.

These checks describe observable language, not personal competence.

Check	Score	Status
Problem Description	0%	Not yet observed
Request	100%	Observed in turn 1
Supporting Detail	100%	Observed in turn 1
Acknowledgement Of A Constraint	0%	Not yet observed
Workable Trade-Off	0%	Not yet observed
Confirmed Agreement	0%	Not yet observed

COMPARED WITH YOUR PREVIOUS ATTEMPT

What changed this time

Same scenario, using the same observable skill measures.

	Previous	This time	
Problem Description	0%	0%	About the same
Request	100%	100%	About the same
Supporting Detail	100%	100%	About the same
Acknowledgement Of A Constraint	0%	0%	About the same
Workable Trade-Off	0%	0%	About the same
Confirmed Agreement	0%	0%	About the same

Differences describe observed language in these two attempts. They do not measure personal or clinical improvement.

STRENGTHS

- Observed request.
- Observed supporting detail.

TRY NEXT

- Try making the problem description explicit.
- Try making the acknowledgement of a constraint explicit.
- Try making the workable trade-off explicit.
- Try making the confirmed agreement explicit.

EVIDENCE FROM THIS ATTEMPT

Language indicating request appeared in turn 1.

Language indicating supporting detail appeared in turn 1.

Generated from deterministic communication features. Source: deterministic workload v2.

Figure 4.8: Post-rehearsal feedback and review interface, presenting evidence-linked observations and access to replay, alternative branches, personal takeaways, and the editable action card.

or status regions, and generally retain the user's input for retry. Responsive CSS changes the multi-column workspace and comparisons into single-column layouts, while visible keyboard focus, semantic controls, reduced-motion rules, live regions, and touch-sized actions support accessible desktop and mobile use.

4.7.3 Conversation Preparation

Participants can begin with a standard scenario or prepare a conversation in their own terms. The preparation form asks who the conversation is with, what happened, the desired outcome, and the difficult part. These answers are sent to the bounded preparation endpoint, which returns an editable scenario draft with an opening line, supported skills, and a tentative objection. The interface labels this output as a rehearsal plan rather than a prediction.

The participant reviews and may edit the title, character, situation, objective, opening line, expected skills, difficult part, and likely objection before saving the custom scenario. Validation errors remain attached to the draft so an outage or rejected save does not discard their work. Previously stored custom scenarios can be edited or deleted only through the account that owns them; older scenarios without preparation metadata remain compatible with the editor.

Before every attempt, the setup screen displays the selected character, objective, supported skill targets, difficulty description, and, where eligible, character profile. Pre-rehearsal confidence and anxiety ratings require an explicit complete response or an explicit skip. Starting from a populated session creates a new attempt rather than replacing prior reflection, feedback, or research history.

4.7.4 Role-Play Workspace

The main workspace contains modes for reflection, active role play, feedback, replay, comparison, and the action card. A transcript panel presents ordered user and assistant turns, while the composer holds typed or reviewed transcribed text. During role play, a header shows the scenario, character, stage, progress, difficulty or profile, and controls for pause, resume, finish, and eligible rewind. Moving to reflection pauses an active rehearsal after confirmation; text entered while paused does not advance the role-play controller.

The browser includes the current session version and a newly generated request identifier with enhanced chat submissions. It clears the composer only after the server confirms the saved exchange. If a request fails, the typed draft and any voice attachment remain available; if the server reports a conflict, the interface offers to reload the committed conversation while retaining the draft. This avoids displaying a message bubble that was never persisted and lets the participant decide whether an uncertain submission should be sent again.

The workspace replaces its state from each response, including the saved user turn, assistant turn, emotion state, role-play status, feedback, and post-questionnaire opportunity. Busy indicators and an `aria-live` transcript communicate progress without inventing an optimistic response. Deleting and starting afresh requires a deliberate action and uses the backend deletion path rather than only clearing local display state.

4.7.5 Voice Capture and Multimodal Interaction

Voice input is opt-in at both the settings and exchange levels. The browser requests microphone permission only after the participant selects the voice control, records for at most 20 seconds, and exposes recording and stop states. The captured signal is decoded through the browser audio context, mixed to mono, resampled to 16 kHz, encoded as PCM WAV, and kept only in component memory. Denied permission, unsupported recording, and conversion failure leave the text composer available.

When transcription is configured, stopping a recording sends the transient WAV to the authenticated transcription endpoint. The returned text is inserted into the composer for explicit review and editing; it does not become a conversation turn until Send is selected. A new recording or text edit clears any stale display associated with the previous draft. The reviewed text and optional WAV are then submitted together in one version-bound chat request so the affect estimate, pacing decision, and stored wording refer to the same exchange.

The interface permits a user to disable the microphone preference locally, enable automatic voice-informed pacing, or select keep going, gentler pace, or more challenge. It does not calculate predictions or send client-authored confidence values. After completion or failure, the component releases its media stream and object references. Raw audio is not added to browser session history or backend session objects.

4.7.6 Presentation of Affect and Uncertainty

The standard emotion panel displays the backend's tentative lexical state using a distribution, valence, arousal, intensity, trend, and confidence. This is the panel shown in Figure 4.7; it is not an output of the trained multimodal classifier. Its confidence is a deterministic heuristic derived from lexical evidence and temporal smoothing, so, for example, a displayed value of 80% does not mean that the estimate has an empirically calibrated 80% probability of being correct. Its language consistently presents these values as estimates. The panel is secondary to the conversation and is accompanied by the uncertainty disclaimer rather than being used as an authoritative description of how the user feels.

When a valid multimodal result is available, a separate panel presents the fused confidence, text and voice leaders, modality agreement, full distribution, latency, queue time, and the statement that audio was not stored. A fused label below the configured display threshold is headed as an uncertain estimate rather than as the predicted category. Disagreement receives an explicit explanation, and the full probabilities remain visible so that the leading label is not separated from its uncertainty.

For exchange-linked pacing, the "Why this response?" view uses the persisted `AffectDecision`. It shows whether analysis was requested and available, the selected action and reason, user preference, fallback code, model and policy versions, timings, and distributions. Reloading displays the same record and does not run inference again. Missing or failed inference is represented as an availability gap, not silently replaced by the lexical estimate.

Table 4.3 makes explicit the different inputs, vocabularies, score semantics, operational effects, and supporting evidence of the four analysis and decision components. In particular, only the trained multimodal classifier has corpus-level classification and calibration evidence; the remaining scores expose deterministic rule strength or priority and must not be interpreted as probabilities.

4.7.7 Feedback, Replay, and Comparison Interfaces

The feedback screen distinguishes the controller outcome from the individual metrics. It renders evidence scores, supporting turn numbers, strengths, suggestions, observed features, generation source, and any same-version comparison with a prior attempt. Difference labels explicitly refer to observed language in two rehearsals and avoid claims of personal improvement. The same screen hosts the personal takeaway and the immediate post-rehearsal ratings when their server-issued opportunity token is still valid.

Replay presents a selectable exchange timeline with linked evidence, controller state, generation provenance, and affect-pacing details. Buttons move focus to the source turn for keyboard as well as pointer users. Missing legacy data and unavailable measurement boundaries have dedicated explanations. From a supported user turn, the participant can create a branch; the original remains unchanged and the new workspace opens with copied context and the old response as an editable draft.

The comparison view separates shared context from two labelled continuation regions and stacks them on narrow screens. It reports utterances, evidence, actions, sources, outcomes, and agreements without ranking one branch as globally better. The action-card view provides editable, character-limited fields with save, retry, copy, and download controls. Save failures preserve edits, and version conflicts ask the user to refresh rather than overwriting a newer card.

4.7.8 Optional Research Interfaces

Participant research controls live in settings and the authenticated dashboard rather than in public registration. The settings page fetches the current server-provided information sheet, displays eligibility criteria, data categories, processors, audio handling, retention, risks, withdrawal terms, and contacts, and requires separate confirmations plus the access code. Enrolled participants can see their pseudonymous identifier and consent versions, download a text-free personal research JSON file, or enter a two-step withdrawal flow without deleting the ordinary account.

An enrolled participant also receives a three-task checklist. It obtains progress from durable study records, labels tasks as not started, in progress, awaiting ratings, incomplete, or complete, and directs the next eligible action to workload, boundary, and relationship practice in order at intermediate difficulty. Required-task selection is fixed by the server; additional practice remains visibly separate. Closing or skipping a post-questionnaire advances the interface without calling the task complete.

The allow-listed researcher dashboard presents descriptive recruitment and completion counts, protocol status, scenario and difficulty summaries, questionnaire averages, skill summaries, generation sources, and pseudonymous participant activity. Researchers may set the collection dates, record a bounded exclusion reason and data-quality note, download the versioned CSV, and initiate the final dataset freeze when eligible. Names, email addresses, transcripts, audio, custom wording, and takeaways are not rendered. A non-researcher cannot reach this view merely by

Table 4.3: Distinction between the trained affect model and the application's deterministic analysis and decision components.

Component	**Inputs**	**Labels or outputs**	**Meaning of scores**	**Affected behaviour**	**Evaluation evidence**
Trained multimodal classifier	Target text, up to three preceding causal turns, and up to 20 seconds of 16 kHz speech	Anger, happiness, neutral, and sadness	Temperature-scaled text and audio class probabilities followed by calibrated late fusion; confidence is the maximum fused probability	Separate voice–text panel and optional bounded pacing. It does not determine crisis handling and is excluded from required study tasks	Five speaker-independent IEMOCAP folds with classification, calibration, confusion-matrix, and dialogue-bootstrap analyses (Section 5.5)
Lexical affect estimator (`lexical-v2`)	Current user text and the preceding lexical state used for smoothing	Anxiety, sadness, anger, frustration, shame, guilt, joy, and neutral; also valence, arousal, intensity, and trend	Normalised term weights, a neutral prior, punctuation adjustments, and temporal smoothing. Confidence is heuristic rule strength, not a calibrated probability	Standard affect panel; input to support-strategy selection and to bounded role-play state and evidence handling	Unit and authored-case tests of rule behaviour only; no labelled-corpus accuracy or calibration evaluation
Rule-based cognitive analyser	Current user text and the independently determined crisis flag	Five intent categories, ten possible cognitive-pattern matches, a possible cause, readiness, resistance, and request flags	Fixed values and pattern-derived weights indicating which rules matched; neither probabilities nor validated psychological measurements	Supplies tentative features to the strategy selector; it does not override crisis precedence or the role-play controller	Unit and integration cases described in Section 5.6.2; no predictive- or construct-validity evaluation
Scored strategy selector	Smoothed lexical state and cognitive-analysis output	Validation, reflection, clarification, validate–reframe, practical suggestion, rehearsal invitation, de-escalation, or safety escalation	Additive rule priorities used to rank strategies; scores are not probabilities	Selects the general-response strategy outside active role play. Safety and active role-play actions remain independently authoritative	Deterministic policy and integration tests (Section 5.6.3); no evidence that the selected strategy is clinically effective or optimal

navigating to its route because the API applies the independent researcher dependency.

4.8 Optional Research Support and Data Export

The research workflow implements the frozen feasibility protocol as versioned application state. Account privacy acceptance and study participation are separate, and ordinary use of AffectLab does not imply research consent. The protocol identifier, consent version, eligibility version, attempt purpose, task association, and measurement timestamps are carried into the records needed to reconstruct the study flow.

4.8.1 Participant Enrolment and Consent

The participant information endpoint supplies the current study title, protocol and consent versions, eligibility scope, collected data, external processors, audio handling, retention statement, risks, limitations, withdrawal terms, and research contacts. Enrolment requires an authenticated, verified account, a configured access code, an open collection window, and explicit confirmation that the information was read, participation is voluntary, and the stated processing is accepted. The access code is compared using a constant-time operation.

Eligibility is confirmed separately for minimum age, approved geography, English use, and the additional frozen inclusion and exclusion criteria. The backend stores the criteria fingerprint, protocol version, and confirmation time, but not date of birth, precise address, health details, or a reason for ineligibility. If the configured criteria, consent version, or protocol changes, the old confirmation is not silently treated as current. Withdrawn or researcher-excluded accounts cannot re-enrol, and enrolment closes once dataset freezing begins.

Successful enrolment creates a pseudonymous participant identifier independent of email and records the exact consent and eligibility versions. These fields are evaluated again when starting a required task or accessing progress; possession of an access token from an earlier state is insufficient. The protocol is a single-arm feasibility and usability pilot, and the application does not expose efficacy or clinical-effectiveness claims.

4.8.2 Study Progress and Questionnaires

The required procedure contains workload, boundary, and relationship scenarios in that order, each at intermediate difficulty. Only an attempt explicitly marked required and associated with the matching scenario can become the primary attempt for a task. The first such attempt is selected deterministically by start time and session identifier; an additional practice, retry, custom scenario, branch, or later successful attempt cannot replace it. This prevents outcome-driven reassignment of the protocol record.

Before an attempt begins, confidence and anxiety use integer ratings from 1 to 7. After an eligible completed rehearsal, confidence, realism, and usefulness use the same scale. Each phase requires either all declared items or an explicit skip. Pre-ratings cannot be added after the role play starts, and post-ratings require the one-use token issued at completion. Leaving the immediate feedback opportunity closes it; a saved response or skip cannot be edited, and the attempt cannot then be rewound.

A required task is complete only when the required scenario and difficulty match, its completion reason is success, maximum turns, or manual finish, and a complete post-questionnaire exists. A safety interruption, omitted post-ratings, closed window, or abandoned task remains incomplete. Progress nevertheless permits the participant to continue to later tasks, preserving observable missingness rather than forcing or fabricating a score. Collection dates and dataset-freeze state can disable new required activity while leaving ordinary practice available.

4.8.3 Pseudonymous Research Records

Eligible post-consent session activity is projected into a separate `StudyRecord`. It contains participant and session pseudonyms, protocol and consent versions, enrolment and activity times, retention deadline, turn count, attempt and branch classification, scenario, difficulty, completion reason, measurement times, numeric feedback metrics, questionnaire values, generation and fallback counts, and text-minimised event records. Pre-consent events are excluded, and branch copies are not counted as new participant activity.

The record excludes account names, email, password material, conversation text, raw audio, custom scenario wording, feedback prose, saved takeaways, action cards, and dialogue snapshots. Event export further reduces properties to counts because even a nominally structured property could contain custom wording. A participant with no retained sessions still receives a participant-level export row so the enrolment denominator is not silently reduced.

Study-record retention is independent of the shorter application-session lifecycle. Each eligible activity refreshes the configured research deadline, and MongoDB applies a separate time-to-live index. Startup backfilling projects missing records only for currently eligible, post-consent material. A protocol freeze blocks further durable study-record changes, preserving the final analysis boundary.

4.8.4 Research Dashboard and Data Export

The researcher dashboard constructs aggregates from the current protocol cohort while excluding withdrawn and researcher-excluded participants from analysis. It reports enrolment and completer targets, protocol completion, completed rehearsals, scenario and difficulty counts, questionnaire averages, deterministic skill averages, generation sources, and participant-level lifecycle status. These are descriptive monitoring values; the inferential analyses and substantive results belong to Chapter 5.

The live CSV uses the versioned `affectlab-frozen-dataset-v4` schema. Its rows include pseudonymous identifiers, consent and eligibility versions, timestamps, task classification, branch and feedback scope, questionnaire values, numeric feedback, generation and fallback counts, and event counts. Missing values remain empty rather than imputed. Branch rows explicitly report copied-context counts and continuation-only feedback so that they are not pooled unknowingly with full attempts. The participant's personal JSON export follows the same exclusion of identity and conversation text while retaining their own participant identifier.

Once the configured study end date is reached, an authorised researcher may start the freeze operation. The repository uses a freeze token to prevent concurrent freezes, generates a de-identified CSV with new sequential participant and session labels, stores its schema, protocol, time, row and participant counts, and SHA-256 digest, and then marks the lifecycle frozen. Subsequent CSV downloads return this immutable stored content with its checksum. Date changes, participant-review edits, enrolment, and research-record updates are blocked after freezing; withdrawal remains available, although an irreversibly de-identified frozen row can no longer be linked for removal.

Figure 4.9 summarises the minimising projection, distinct retention domains, and boundary introduced by irreversible dataset freezing.

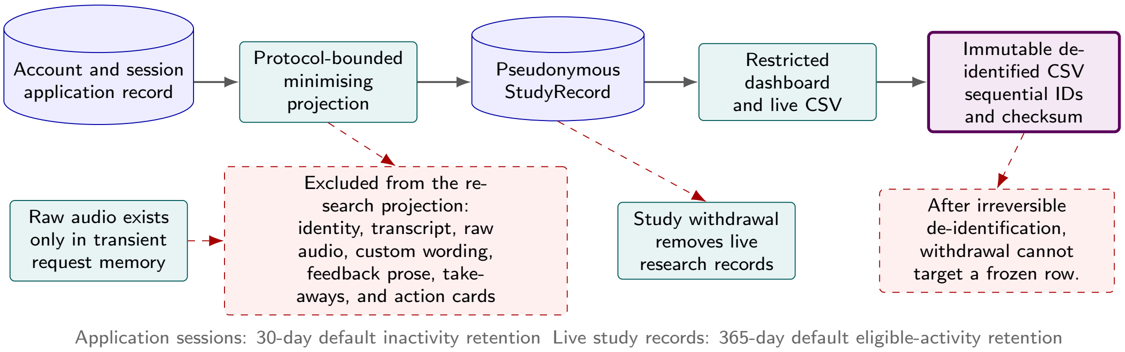

Account and session application record
Protocol-bounded minimising projection
Pseudonymous StudyRecord
Restricted dashboard and live CSV
Immutable de-identified CSV sequential IDs and checksum
Raw audio exists only in transient request memory
Excluded from the research projection: identity, transcript, raw audio, custom wording, feedback prose, takeaways, and action cards
Study withdrawal removes live research records
After irreversible de-identification, withdrawal cannot target a frozen row.
Application sessions: 30-day default inactivity retention
Live study records: 365-day default eligible-activity retention

Figure 4.9: Research-data lifecycle from the richer account-owned session to the pseudonymous live record and immutable de-identified export.

4.9 Security, Privacy, and Data Protection

Security and privacy controls are distributed across configuration validation, middleware, authentication services, ownership-aware repositories, data minimisation, and user-facing lifecycle operations. They establish a production-oriented security baseline rather than a guarantee of confidentiality. The repository includes a separate remote-deployment baseline because a local demonstration configuration is not sufficient for internet-facing use.

4.9.1 Authentication and Token Management

Account passwords are accepted only over the API's protected transport boundary in a deployed environment and are converted immediately to recommended adaptive hashes. Verification and password-reset values are generated with a cryptographic random source; the database receives only SHA-256 digests and expiry times. Password reset validates the new password before atomically consuming the stored record, so an invalid new password does not destroy the recovery opportunity while two successful requests cannot reuse it.

Access tokens are signed, typed JSON Web Tokens with configurable short lifetimes. Refresh tokens have separate type claims and unique identifiers, and their digests permit server-side rotation and revocation. The browser keeps the access token in session storage, whereas the refresh credential is an HTTP-only, same-site cookie restricted to the authentication path and marked secure in production. Changing or resetting a password revokes all refresh records and signs the current browser out.

Generic registration, verification-resend, and password-recovery responses reduce account enumeration. Email verification is checked after reloading the user on protected requests. Authentication and email endpoints have dedicated rate-limit categories, and production start-up rejects a disabled limiter. These controls reduce common automated misuse but do not replace monitoring, secret rotation, or infrastructure-level abuse protection.

4.9.2 Authorisation and Resource Isolation

Authorisation begins with the authenticated account identifier and is repeated at the persistence boundary. Session read, list, update, delete, multimodal analysis, branching, comparison, questionnaire, feedback, and action-card operations all obtain the session through an identifier-and-owner query. An unauthorised identifier produces the same not-found behaviour as a missing session. Custom scenarios are nested in the owning user, and branch creation verifies both parent and child lineage against that owner.

Researcher authorisation is not inferred from a route, client flag, or pilot enrolment. It requires a verified account whose normalised email is present in the server-side allow-list. Study operations additionally check current consent, eligibility version, protocol version, withdrawal and exclusion state, collection dates, and freeze state. Dataset lifecycle mutations, participant review, and exports therefore remain protected even if a participant manually invokes their URLs.

Optimistic versions and idempotency identifiers add integrity protection to authorisation. They do not merely identify who may write, but also which state that write was based on. Conditional MongoDB replacements and deterministic branch identifiers prevent lost updates and duplicate alternatives across processes. The API returns conflicts for recovery instead of choosing a winner silently.

4.9.3 Audio and Conversation Data Handling

Conversation text is stored because session continuation, replay, evidence links, and deletion controls require it. When provider-backed generation is enabled, the relevant session context may be sent to the configured provider with response storage disabled. The application does not write raw prompts or generated replies to its own operational logs. The participant information and settings interfaces disclose these boundaries rather than treating local and provider processing as equivalent.

Audio requires an explicit recording action for every exchange. The browser holds the WAV transiently, and the backend checks encoded and decoded size, file structure, duration, channels, and sample rate before transcription or local inference. Transcription may send it to the configured provider, whereas multimodal inference processes it in memory. Neither path adds the recording to the session, study record, or export. Only an approved transcript, bounded prediction summary, provenance, and pacing decision may persist.

Research projections minimise the richer session document. They omit identity, transcripts, free-text feedback, custom scenario content, takeaways, action cards, and raw model inputs. Exported event information is counted by event name rather than exposing arbitrary properties. Source-control exclusions separately keep environment files, local databases, raw datasets, checkpoints, model artefacts, generated runs, and backup material outside the repository.

4.9.4 Retention, Withdrawal, and Account Deletion

Application sessions and research records have distinct retention fields. By default, an active session expires after 30 days of inactivity, while a pseudonymous study record expires 365 days after its latest eligible activity. MongoDB time-to-live indexes perform eventual physical deletion, and repository reads also apply the deadlines logically so expired material is not returned while background deletion is pending. Token collections use their own expiry indexes.

A participant may delete an individual session without deleting the account. Account deletion removes the user, owned sessions, live study records, refresh tokens, and outstanding verification and reset records. The settings interface requires browser confirmation and retains the signed-in state if the request fails, allowing a safe retry. Backups remain a separate operational responsibility and must follow the approved encrypted retention schedule.

Study withdrawal is intentionally narrower. It removes questionnaires, questionnaire skips, post-rating tokens, research-event telemetry, and live study records, and marks the account as withdrawn so it is excluded from later collection and export. Ordinary account access, conversations, feedback, and takeaways remain available. The interface explains that rows already irreversibly de-identified in a frozen export or incorporated into aggregate analysis may no longer be linkable for removal. Re-enrolment after withdrawal is blocked.

4.9.5 Deployment Security Controls

Setting the environment to production, pilot, or staging activates fail-fast configuration validation. The signing secret must be unique and at least 32 bytes; allowed origins must be explicit HTTPS addresses; trusted hosts cannot contain a wildcard; HTTPS enforcement and rate limiting must be enabled; and offline demonstration mode is prohibited. Refresh cookies become secure and responses add HTTP Strict Transport Security. CORS still permits credentials only from the declared origins.

The API and Nginx layers add complementary content-security, framing, referrer, content-type, permissions, and caching headers. Host validation and HTTPS redirection occur in the backend, while the frontend server restricts executable and embedded sources and grants media access only to local blob content. The intended remote arrangement terminates TLS at a maintained reverse proxy and prevents direct public access to MongoDB and the application server.

The bundled limiter is explicitly scoped to a single backend process and does not trust forwarded client-address headers. A multi-worker deployment must use a shared gateway or distributed limiter. The operational baseline also requires patched hosts, dependency auditing, protected secrets, encrypted access-controlled backups, and a restore drill. The backup utility preserves BSON types and indexes, records per-collection checksums and counts, and will restore by default only into an empty database whose name ends in `_restore_test`, reducing the chance of overwriting the live store.

4.10 Deployment and Configuration

The implementation supports three related execution arrangements: direct local development, a containerised integrated stack, and a deliberately offline demonstration. These arrangements reuse the same application code but select different repositories and optional services through configuration. None embeds production credentials or private model files in an image.

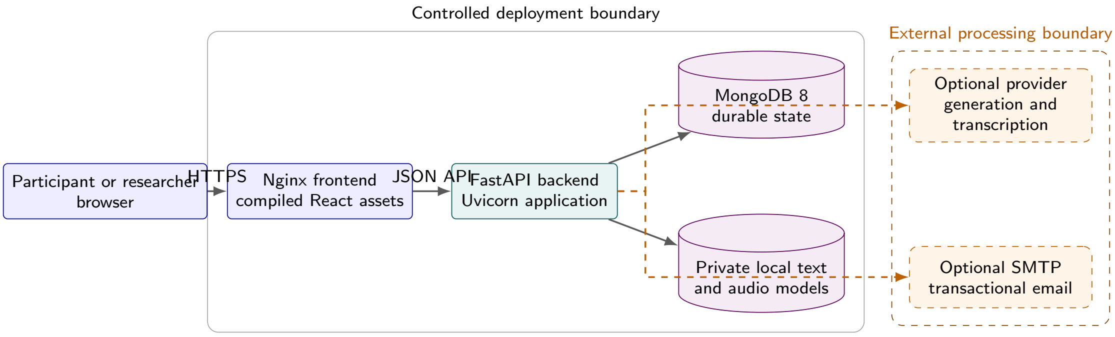
4.10.1 Local and Containerised Deployment

For local development, MongoDB can run as an existing service while Uvicorn serves the FastAPI application and Vite serves the frontend with hot reloading. The API address is provided to the frontend at build or development time. Developers may instead select the in-memory repository for isolated service work, accepting that its contents disappear when the process stops.

Docker Compose defines MongoDB 8, the backend, and the production frontend. A named volume stores database contents, the backend waits for the MongoDB health check, and the frontend waits for backend readiness. The backend image installs the core requirements into a Python 3.12 slim base and runs Uvicorn. Optional PyTorch and Transformers packages and private model directories are deliberately absent unless a separate inference-capable deployment supplies them.

The frontend uses a Node.js 22 build stage to run the TypeScript and Vite production build, then copies only compiled assets into an Nginx image. Nginx serves the single-page-application fallback, prevents caching of `index.html`, applies immutable caching to content-hashed assets, and exposes a small health endpoint. Compose's default values are for local use; a remote deployment must replace the development signing secret and satisfy all production validation.

Figure 4.10 shows the resulting service topology and distinguishes locally controlled components from optional external processing.


Controlled deployment boundary
External processing boundary
Participant or researcher browser
HTTPS
Nginx frontend
compiled React assets
JSON API
FastAPI backend
Uvicorn application
MongoDB 8
durable state
Private local text
and audio models
Optional provider
generation and
transcription
Optional SMTP
transactional email

Offline demonstration replaces MongoDB with memory and disables every external and trained-model service.

Figure 4.10: Containerised deployment topology and external-service boundary.

4.10.2 Offline Demonstration Mode

The offline configuration selects the in-memory repository, creates a verified demonstration account at application startup, and disables OpenAI generation, role-play generation, transcription, multimodal inference, SMTP, and rate limiting. Reflection and rehearsal therefore use only deterministic local templates. No MongoDB, cloud storage, private model, email server, or internet connection is required.

This mode makes the application's graceful-degradation claims demonstrable without pretending that unavailable capabilities ran. Model and transcription status remain unavailable, and data disappear on backend restart. The configuration validator rejects offline demonstration in production-like environments, so convenience credentials and automatic account creation cannot be enabled accidentally for a remote pilot.

A synthetic demonstration dataset and runbook keep example conversation content separate from participant data. A stronger engineering demonstration uses real MongoDB and a compiled frontend but still disables external providers and trained inference. The two modes serve different purposes: the memory-backed mode is portable for presentation, while the isolated MongoDB smoke run verifies persistence and restart behaviour.

4.10.3 Environment-Based Configuration

Settings can be supplied through process variables or repository- and backend-level environment files. They cover API origins and hosts, repository choice and database name, token and retention lifetimes, provider models and timeouts, SMTP, multimodal paths and thresholds, consent and protocol versions, study targets and contacts, eligibility scope, researcher allow-list, access code, offline credentials, and rate limits. Defaults support development but do not silently qualify as production settings.

The checked-in example file documents required names without containing live secrets. The real `.env` file is ignored by version control. Private model weights are synchronised into a user-selected local directory and referenced by path; raw IEMOCAP data and derived row-level artefacts remain outside the application repository. Separate dependency sets allow a deployment to omit local machine-learning libraries entirely.

Configuration values with scientific meaning are also retained in versioned JSON rather than only in mutable environment variables. These include the final multimodal label order, input limits and calibration chain, as well as the study protocol task order, measures, missing-data policy, target sizes, and prohibition on efficacy claims. This separates deployer-specific secrets from inspectable experimental decisions.

4.10.4 Health Checks and Operational Failure Handling

The liveness endpoint reports only that the API process can respond. Readiness revalidates settings, pings the repository, and verifies every required index; configuration, persistence, and index failures receive distinct states and HTTP 503. Optional transcription and multimodal capabilities are reported separately and do not make the text-first service unready. Container health checks consume these endpoints for dependency ordering and restart decisions.

Expected failures are translated into bounded client responses: unauthenticated and forbidden requests, missing resources, validation failures, unavailable optional services, rate limits with `Retry-After`, and concurrency conflicts have distinct status codes. Provider exceptions and inference failures do not expose stack traces or raw exception content in conversation records. The frontend preserves drafts and presents retry or reload actions according to whether the operation may have committed.

The system is designed to fail closed where authority is involved and fail soft where an enhancement is involved. Missing ownership, stale state, invalid consent, a closed study window, or a dataset freeze prevents the write. Missing generation, transcription, or emotion-model capability preserves the deterministic text workflow. This distinction keeps availability from weakening access, protocol, or safety constraints.

4.11 Implementation Verification

Verification is organised by boundary rather than by a single undifferentiated test suite. Deterministic unit tests check policies and transformations, API tests check contracts and ownership, MongoDB tests check real persistence semantics, browser tests check user workflows and accessibility, performance tests detect latency regressions, and smoke tests join the compiled components. These developer-led checks provide the principal evidence for application quality; model accuracy and supplementary user perceptions remain separate questions.

4.11.1 Backend Unit and Integration Testing

The default pytest suite forces the in-memory repository and disables SMTP, provider, transcription, and trained-model inference. It tests authentication and recovery, generic anti-enumeration responses, token rotation and revocation, ownership, optimistic versions, session lifecycle, study consent and questionnaires, exports and freezing, preparation, feedback, replay boundaries, rewind, branching, crisis precedence, dialogue transitions, affect pacing, and optional-service failure. Parameterised cases exercise profiles, difficulty levels, negations, reversals, confidence combinations, missing modalities, duplicate requests, and legacy compatibility.

Machine-learning support tests separately validate IEMOCAP parsing, safe archive paths, causal context, speaker-independent folds, class weights, calibration, probability metrics, utterance pairing, fusion, bootstrap procedures, and balanced final-model smoke selection. The disabled multimodal service is tested without importing heavy machine-learning dependencies; concurrency tests verify serial inference access when enabled. These fixtures confirm implementation contracts, not the accuracy of a trained model on new users.

MongoDB integration tests are opt-in and create uniquely named disposable databases. They recreate repository and service clients to check persistence across restart, required indexes and logical expiry, separate study retention, cascade deletion, optimistic conflict detection, single-use recovery, versioned dialogue restoration, branch idempotency, and saved affect provenance. Backup tests create an archive, restore it into a protected disposable target, and compare checksums and document counts.

Ruff checks imports, syntax-level errors, modern Python usage, and selected bug patterns across application, test, and script code. Dependency security is checked separately because style and behavioural tests cannot identify published package vulnerabilities.

4.11.2 Frontend End-to-End Testing

The regular Playwright suite runs the browser against a deterministic mocked API boundary. This makes UI state and failure injection repeatable while testing registration and onboarding, required and ordinary practice, preparation, voice-review submission, feedback, replay, branching, comparison, action cards, research enrolment, questionnaire timing, researcher access, settings, and account recovery. Dedicated cases abort or reject network calls to verify that drafts, recordings, selections, and stored rows are not falsely discarded.

Projects run in desktop Chromium and a mobile Chromium profile. Representative pages are scanned with axe-core against WCAG 2.0 and 2.1 A/AA rule sets, alongside assertions for keyboard focus, accessible names, live status messages, and responsive layouts. Production compilation runs the strict TypeScript project build before Vite bundling, and ESLint checks React hook and TypeScript rules.

Because the normal browser suite mocks the backend, its passing result demonstrates frontend behaviour against declared contracts rather than integrated persistence. Assertions that depend on real authentication, FastAPI serialisation, MongoDB indexes, or process restart are reserved for the full-stack suite.

4.11.3 Performance Regression Testing

The repeatable performance script exercises the deterministic text-first workflow through FastAPI's in-process HTTP test transport. After warming the application, it records repeated liveness, authenticated scenario-catalogue, session-creation, role-play-start, and conversation-turn requests. Each summary includes the sample count, median, 95th percentile, and maximum, while the evidence file records the execution environment and SHA-256 hashes of the relevant application and benchmark sources.

The benchmark deliberately uses the memory repository and disables transcription, trained multimodal inference, and provider generation. This isolates locally controlled routing, authentication, validation, dialogue, persistence-interface, and serialisation overhead and makes the check stable enough for regression use. Its scope statement explicitly excludes network transit, MongoDB, external providers, concurrent users, and deployment capacity. The performance results are reported in Section 5.6.5; production staging must repeat the measurement with the selected database, models, providers, network, and expected concurrency.

4.11.4 Full-Stack Smoke Testing

The container smoke runner overlays a dedicated Compose configuration on the normal stack. It builds the production frontend and backend, starts an isolated

MongoDB database, creates a verified offline researcher, and waits for service health. Playwright then signs in through the rendered application, creates and reloads a pre-enrolment reflection, enrols with versioned eligibility and consent, completes a required rehearsal and questionnaire, verifies progress and researcher aggregates, checks that the official export excludes the pre-consent session, restarts the backend, and confirms that the completed session persists.

A second journey exercises an enhanced deterministic rehearsal, simulated network failure and draft recovery, replay, branch creation, comparison, action-card persistence, another backend restart, and preservation of the original required-task progress. The runner always removes its isolated containers and volume, and uploads traces or screenshots on continuous-integration failure. A GitHub Actions workflow runs this smoke path on pushes and pull requests.

The native engineering smoke script provides a complementary local path. It builds the compiled frontend, serves it from a small local server, launches and restarts actual Uvicorn child processes, uses a uniquely named real MongoDB database, and writes a synthetic evidence manifest with environment details, source hashes, elapsed time, and restart identifiers before deleting the database. Neither smoke arrangement calls a live language provider or claims trained-model performance.

4.11.5 Reproducibility and Model-Provenance Checks

Reproducibility checks combine declarative configuration, frozen folds, stored predictions, code-level evaluation scripts, and immutable hashes. Model preprocessing records source and manifest digests; training configurations fix seeds and hyperparameters; calibration consumes validation predictions; fusion verifies exact utterance identifiers; and dialogue-clustered bootstrap procedures retain speaker and conversation dependence. Full-data packaging is kept distinct from held-out evaluation.

After private final models are downloaded, the provenance script records the Git commit, multimodal-configuration hash, platform and relevant package versions, and the size and SHA-256 digest of every text and audio model file. Ordered file entries produce aggregate directory digests. The private manifest can therefore bind an inference installation to exact artefacts without publishing licensed weights or local storage paths in the repository.

The deterministic engineering evaluator uses authored synthetic conversations to compare expected state trajectories, evidence decisions, and affect-pacing actions. Its JSON output records scope, source hashes, environment, individual cases, and in-process timings, with an explicit statement that it is neither participant nor model evaluation. The evidence package similarly distinguishes reproducible engineering demonstrations from the cross-validated results reported in Chapter 5.

Continuous integration supplements behavioural checks with weekly and change-triggered Python and npm dependency audits, while Dependabot proposes updates for Python, npm, and workflow dependencies. Reproducing a result still requires recording the exact commit, lock files, private dataset and model provenance, export checksum, and applicable protocol version; a successful build alone is not treated as sufficient provenance.

4.12 Chapter Summary

This chapter described the implementation of AffectLab as a modular, text-first, production-oriented application. The React and TypeScript frontend communicates with a typed FastAPI backend whose services separate authentication, persistence, dialogue control, affect analysis, response realisation, research collection, and optional external providers. MongoDB stores ownership-bound, versioned sessions and separately retained pseudonymous study records, while an in-memory repository and deterministic templates support isolated testing and offline demonstration.

The emotion-recognition subsystem prepares speaker-independent IEMOCAP folds, trains contextual DeBERTa-v3 text and wav2vec 2.0 speech classifiers, calibrates their distributions, and combines them through late fusion. Private final artefacts are loaded only when configured and are tied to explicit calibration and provenance manifests. Within the application, these predictions remain optional uncertainty-bearing inputs to a bounded pacing policy; they do not control crisis handling, dialogue success, study completion, or feedback scores.

Role-play is governed by versioned scenarios, observable evidence rules, deterministic state transitions, and stored before-and-after decisions. Provider generation may phrase an authorised action but cannot select it. Feedback remains linked to source turns, and replay, rewind, branching, comparison, takeaways, and action cards preserve the distinction between recorded evidence and user-authored reflection. Safety checks precede inference and generation and interrupt a rehearsal through a fixed local path.

The optional research workflow separates account use from versioned study consent and eligibility. Required tasks, immediate questionnaires, pseudonymous research projection, text-free exports, lifecycle controls, and immutable dataset freezing implement the frozen pilot protocol without affecting ordinary practice. Authentication, ownership filtering, optimistic concurrency, retention, withdrawal, account deletion, transient audio handling, production configuration checks, and backup verification provide layered security and privacy controls appropriate to controlled deployment.

Finally, unit, API, database, browser, accessibility, performance, full-stack, reproducibility, and provenance checks verify the implementation at complementary boundaries. These checks establish that the coded components follow their declared contracts and that the local deterministic path remains responsive under the measured conditions. Model validity, deployment-scale capacity, and the limited supporting user evidence are reported with their separate boundaries in Chapter 5.

Chapter 5

Evaluation and Results

5.1 Chapter Overview

This chapter evaluates AffectLab primarily as a complete technical product. It first reports the speaker-independent emotion-recognition experiments concerning conversational context, acoustic information, multimodal fusion, and calibration. It then evaluates the integrated application through authored engineering cases, automated unit and integration suites, production builds, browser and accessibility checks, persistence and recovery exercises, full-stack smoke journeys, and a reproducible response-time benchmark. A short final subsection reports the supplementary user-pilot results without treating that small study as the principal validation of the application.

These sources support different claims. IEMOCAP results measure classification and probability quality on a controlled corpus. Engineering evidence establishes that the application follows its specified transitions, fallbacks, ownership rules, safety precedence, persistence contracts, responsive layouts, and recovery paths under the tested conditions. Local timings characterise the deterministic core but are not a substitute for deployment-specific load testing. The small user pilot contributes limited contextual evidence about perceived realism, usefulness, and workflow completion; it does not carry the claim that the product is technically ready for controlled deployment.

The chapter first maps the research questions to their appropriate evidence. It then presents the common model-evaluation protocol, the contextual-text comparison, the audio and multimodal results, and the classification–calibration trade-off. Application verification, performance results, and the concise pilot summary follow. The final discussion answers each research question only to the extent supported by the available evidence and identifies threats to validity.

5.2 Evaluation Scope and Evidence Hierarchy

5.2.1 Research Questions and Evaluation Levels

Table 5.1 maps the four research questions introduced in Section 1.5 to an evaluation unit and an admissible conclusion. This prevents model accuracy, policy conformance, and integrated application quality from being combined into one undifferentiated claim.

Table 5.1: Mapping between research questions, evidence, and permitted conclusions.

RQ	Evaluation unit	Principal evidence	Permitted conclusion
RQ1	IEMOCAP utterance with causal dialogue history	Utterance-only and three-turn contextual text predictions on identical held-out sessions	Whether causal context improves benchmark text classification.
RQ2	Paired textual and acoustic IEMOCAP samples	Audio baseline, equal-weight fusion, validation-fitted fusion, and calibration metrics	Whether audio adds benchmark classification information and what calibration trade-off follows.
RQ3	Versioned application decision and complete workflow	Authored policy fixtures, automated tests, saved provenance, and full-stack smoke journeys	Whether uncertain estimates are translated into the specified bounded, explainable, and failure-tolerant actions.
RQ4	Integrated application and its principal workflows	Automated suites, desktop/mobile browser checks, production builds, persistence and recovery exercises, full-stack smoke journeys, and local response-time measurements	Whether the completed system satisfies its declared functional and non-functional requirements under reproducible developer-led verification.

RQ1 and RQ2 are empirical machine-learning questions with completed results. RQ3 concerns the bounded integration policy: because the policy is deterministic, controlled input cases can establish whether it implements its specification. RQ4 concerns the quality of the complete implementation and is evaluated through complementary developer-led and automated checks at service, API, persistence, browser, accessibility, recovery, and full-stack boundaries. The small user pilot is reported separately as supporting product feedback rather than as the main answer to RQ4.

5.2.2 Separation of Development, Evaluation, and Deployment Evidence

The model study separates training, validation, and test roles by complete IEMOCAP sessions. Validation predictions select temperatures and, where applicable, fusion parameters; the held-out session supplies evaluation predictions. The predeclared equal-weight fusion is retained as the principal classification comparison because its weight was not selected from the fused test results. Fold-specific validation-fitted fusion is reported as a calibration analysis rather than substituted retrospectively as the headline classifier.

The final text and audio packages were trained on all 5,531 eligible utterances after the cross-validation design and epoch counts were frozen. These packages have no independent test partition of their own. Their training behaviour and balanced-sample smoke checks are therefore integrity evidence, not model-performance results. Performance claims in this chapter come from the five session-held-out experiments.

Engineering test cases use controlled utterances and probability distributions to exercise known policy branches. Browser and full-stack tests automate test accounts and ratings. Their purpose is to demonstrate repeatability, persistence, recovery, and guard conditions, not to estimate human behaviour or the deployed model's accuracy. The supplementary user pilot is therefore interpreted independently of these tests.

5.3 Emotion-Recognition Evaluation Methodology

5.3.1 Benchmark Task and Speaker-Independent Splits

The primary benchmark uses the four IEMOCAP categories anger, happiness, neutral, and sadness. Excitement is merged with happiness, and utterances carrying other source labels are excluded. The resulting task contains 5,531 matched text–audio examples drawn from 151 dialogues across the five dyadic sessions of IEMOCAP [BBL+08]. The same utterance identifiers and label order are used for the text, audio, and fusion experiments.

Five folds provide speaker-independent coverage. In each fold, one complete session is reserved for testing and a different complete session for validation; the remaining three sessions are used for training. Every session therefore appears exactly once as test data. Keeping both speakers from a dyad within one session prevents an individual's utterances from appearing in the training portion of the fold in which that person is tested.

The utterance-only and contextual text conditions use the same DeBERTa-v3-small architecture, training settings, labels, seed, and folds. Their only intended difference is the input: the contextual condition prepends the previous three causal dialogue turns with speaker-relative markers. The audio condition uses wav2vec 2.0 on the paired 16 kHz waveform. This design supports paired comparisons because competing configurations produce predictions for the same examples.

5.3.2 Classification and Calibration Measures

Classification is reported through accuracy, macro-averaged F1, weighted F1, and per-class F1. Macro F1 gives each class equal weight and is the primary aggregate measure because the class frequencies are unequal. Weighted F1 complements it by weighting each class according to its support. Per-class results are retained so that an aggregate gain cannot conceal deterioration in one category.

Probability quality is considered separately. Negative log-likelihood and the multiclass Brier score are proper scoring rules over the complete predicted distribution. Expected calibration error (ECE) partitions examples by maximum confidence and accumulates the absolute difference between the mean confidence and empirical accuracy within each bin. Because ECE depends on the chosen bins and does not fully describe multiclass calibration, it is interpreted alongside the proper scores and reliability outputs rather than as a complete measure [GPSW17, KPNK+19].

Temperature scaling is fitted on validation logits only. For a logit vector $\mathbf{z}$ and positive temperature T, the calibrated distribution is

$$\hat{\mathbf{p}} = \operatorname{softmax}\left(\frac{\mathbf{z}}{T}\right). \tag{5.1}$$

A single temperature changes confidence concentration but not the predicted class. Fusion weights and a final fusion temperature are also fitted only from validation predictions in the deployment-oriented configurations.

5.3.3 Paired Dialogue-Level Statistical Comparison

Utterances from one conversation are not independent observations. Statistical comparisons therefore use a paired percentile bootstrap over the 151 complete dialogues rather than sampling individual rows. For each of 10,000 replicates, dialogues are sampled with replacement and the same sampled dialogue indices are applied to both compared systems. The difference in accuracy, macro F1, or weighted F1 is then recomputed. The 2.5th and 97.5th percentiles form the reported 95% interval. The procedure uses seed 20260808.

This interval describes uncertainty under resampling of the observed dialogue clusters. It is not a confidence interval over all cultures, microphones, languages, or spontaneous emotional settings. No participant or speaker subgroup inference is attempted because IEMOCAP contains only ten actors and was not designed to support such claims.

5.4 Contextual Text-Emotion Results

5.4.1 Primary Four-Class Comparison

Table 5.2 compares utterance-only text with the three-turn causal context condition. Context improved accuracy from 0.6624 to 0.7304 and macro F1 from 0.6651 to 0.7332. The absolute macro-F1 gain was 0.0681, with a dialogue-bootstrap 95% interval from 0.0504 to 0.0857. All three aggregate intervals excluded zero.

Table 5.2: Four-class contextual-text results on five held-out IEMOCAP sessions.

Metric	**Utterance only**	**Three-turn context**	**Difference**	**95% interval**
Accuracy	0.6624	0.7304	+0.0680	[0.0496, 0.0864]
Macro F1	0.6651	0.7332	+0.0681	[0.0504, 0.0857]
Weighted F1	0.6640	0.7300	+0.0661	[0.0477, 0.0842]

The per-class differences were not uniform. Happiness improved by 0.0882, neutral by 0.0338, and sadness by 0.1307, with intervals above zero. Anger improved by 0.0198, but its interval $[-0.0073, 0.0449]$ included zero. The result therefore supports an aggregate benefit from context but does not establish an anger-specific improvement.

The calibrated contextual model obtained pooled ECE 0.0311. This value describes the benchmark distribution after validation-fitted temperature scaling and does not validate the 0.55 application threshold on real AffectLab conversations.

5.4.2 Exploratory Six-Class Comparison

An exploratory task aligned additional source labels to anger, anxiety, frustration, joy, neutral, and sadness. Context again improved all aggregate measures, as shown in Table 5.3. The macro-F1 gain was 0.0732 with an interval from 0.0485 to 0.1009.

Table 5.3: Exploratory six-class contextual-text comparison.

Metric	**Utterance only**	**Three-turn context**	**Difference**	**95% interval**
Accuracy	0.5674	0.6228	+0.0554	[0.0382, 0.0734]
Macro F1	0.5490	0.6222	+0.0732	[0.0485, 0.1009]
Weighted F1	0.5675	0.6205	+0.0529	[0.0359, 0.0702]

The anxiety difference was comparatively large at 0.1650, but it had a wide interval $[0.0476, 0.3144]$ and arose from only 40 IEMOCAP fear annotations mapped to anxiety. Anger and neutral intervals included zero. Consequently, the six-class experiment is not used as the deployed classifier and does not support a strong conclusion about anxiety recognition.

5.4.3 Answer to RQ1

Within the declared IEMOCAP protocol, RQ1 is answered positively. Adding the three preceding causal turns improved the principal text model by approximately 6.8 percentage points in accuracy and macro F1 compared with the utterance-only condition. Pairing, folds, architecture, and optimisation settings were held constant, and the dialogue-level intervals for all aggregate measures excluded zero.

The result is evidence for contextual benchmark classification, not proof that context reveals a user's true emotion. It also does not isolate which part of the representation produced the gain: additional lexical content, speaker markers, or interaction history may each contribute. External validation would be required to determine whether the improvement persists for spontaneous microphone speech and user-authored rehearsal conversations.

5.5 Speech and Multimodal Results

5.5.1 Audio-Only Baseline

The wav2vec 2.0 audio classifier obtained accuracy 0.6279, macro F1 0.6349, weighted F1 0.6229, and calibrated ECE 0.0468. Its per-class F1 scores were 0.7142 for anger, 0.5782 for happiness, 0.5757 for neutral, and 0.6715 for sadness. Audio was weaker than contextual text in aggregate but stronger for anger than the overall pattern might suggest, indicating that the modalities contained different useful information.

The comparison does not imply that the acoustic model measured emotion independently of linguistic content: speech representations can encode phonetic, lexical, speaker, and recording characteristics as well as prosody. It establishes only that the audio branch produced non-redundant predictive information on the matched benchmark task.

5.5.2 Predeclared Equal-Weight Late Fusion

The principal fusion experiment averaged validation-calibrated text and audio posterior distributions with weights of 0.5. Table 5.4 shows that fusion improved accuracy, macro F1, and weighted F1 over contextual text on the same rows. The macro-F1 difference was 0.0551 with a 95% interval from 0.0407 to 0.0713. Every one of the 10,000 dialogue-bootstrap replicates favoured fusion for each aggregate metric.

Table 5.4: Predeclared equal-weight fusion compared with contextual text.

Metric	Context text	Equal fusion	Difference	95% interval
Accuracy	0.7304	0.7832	+0.0528	[0.0381, 0.0683]
Macro F1	0.7332	0.7883	+0.0551	[0.0407, 0.0713]
Weighted F1	0.7300	0.7815	+0.0514	[0.0366, 0.0671]

Fusion per-class F1 was 0.8277 for anger, 0.8163 for happiness, 0.7014 for neutral, and 0.8080 for sadness. Unlike the contextual-text comparison, the combined system showed substantial performance across all four categories. Because the fusion rule was fixed before inspection of the combined test outcome, this result is retained as the primary evidence that audio contributed complementary benchmark information.

5.5.3 Classification–Calibration Trade-Off

Equal averaging improved classification but produced ECE 0.1619. Separately calibrated modality probabilities therefore did not remain calibrated after fusion. This result is important for AffectLab because confidence, not only the top class, is consumed by the pacing policy. Treating the equal-fusion probability as reliable would have contradicted the evaluation evidence.

Fold-specific validation fitting selected text weights between 0.54 and 0.63 and then fitted a final temperature. This reduced ECE to 0.0283, while macro F1 decreased from 0.7883 to 0.7758. Table 5.5 summarises the principal configurations. The complete global chain fitted text and audio temperatures and then fusion parameters on 5,531 pooled out-of-fold validation predictions. It selected text weight 0.59, audio weight 0.41, and final temperature 0.5989.

Table 5.5: Classification and calibration trade-off among multimodal configurations.

Configuration	Accuracy	Macro F1	Weighted F1	ECE
Contextual text	0.7304	0.7332	0.7300	0.0311
Audio only	0.6279	0.6349	0.6229	0.0468
Equal-weight fusion	0.7832	0.7883	0.7815	0.1619
Fold validation-fitted fusion	0.7717	0.7758	0.7707	0.0283
Final global calibration chain	0.7736	0.7776	0.7728	0.0242

Figure 5.1 places the classification results and calibration trade-off on common visual scales. The utterance-only condition is omitted from the calibration panel because no canonical ECE is reported for that configuration.

The row-normalised confusion matrices in Figure 5.2 expose the class-specific complementarity behind the aggregate result. Audio recognised anger more often than contextual text (78.1% versus 69.6% recall), whereas contextual text was substantially stronger for happiness and neutral. Equal fusion raised recall to 85.1% for anger, 81.1% for happiness, 66.6% for neutral, and 85.8% for sadness. The final global chain retained a more balanced pattern while using probabilities suitable for operational confidence.

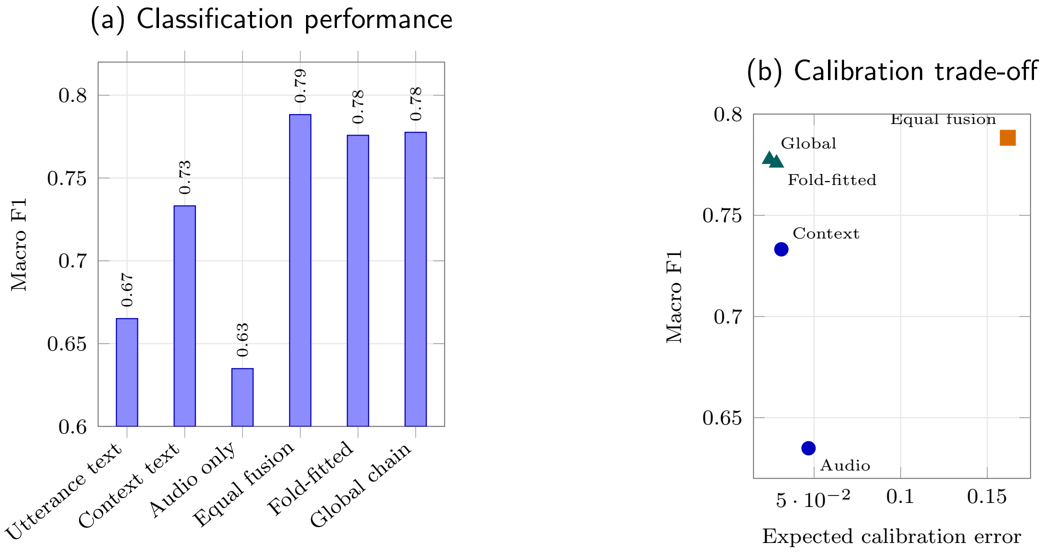


Figure 5.1: Aggregate four-class model results. Panel (a) compares macro F1, while Panel (b) shows why the highest-classification equal-weight fusion is not the preferred configuration when confidence is consumed operationally.

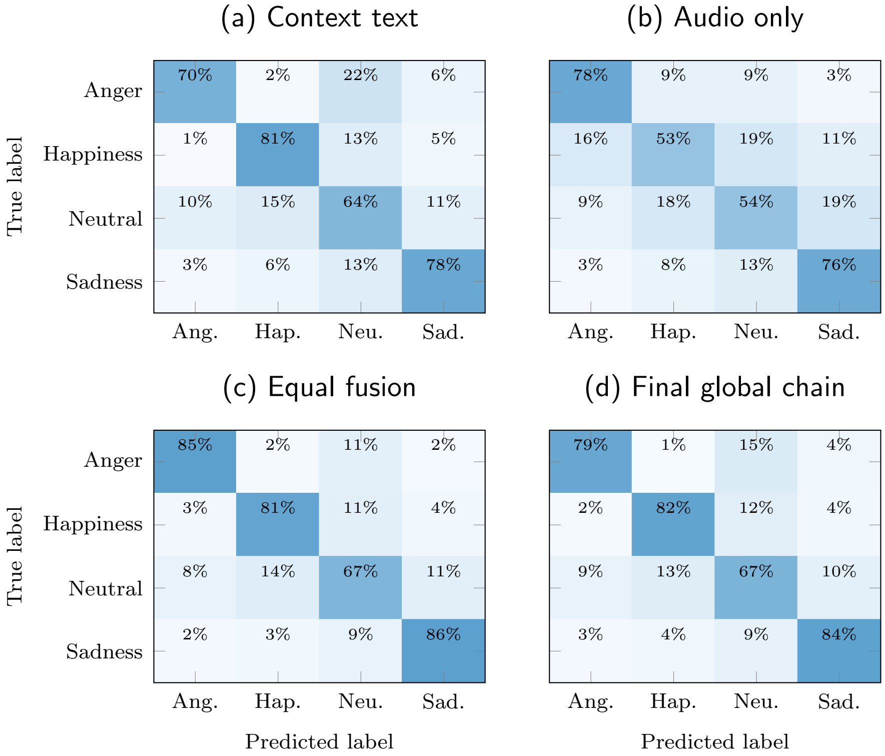


Figure 5.2: Pooled row-normalised confusion matrices over the five held-out IEMOCAP sessions ($n = 5{,}531$). Rows are true labels and columns are predicted labels; cell values are percentages of the corresponding true-label row. All panels use the same 0–100% colour scale. The fold-fitted matrix is omitted because its class pattern is nearly identical to the final global chain.

Figure 5.3 shows the corresponding bin-level calibration behaviour. Equal fusion lies systematically above the ideal diagonal through its populated mid-confidence region, indicating under-confidence despite its superior classification performance. Validation-fitted and global calibration move the fusion curve much closer to the diagonal. The isolated low-confidence points contain very few observations and should not be interpreted as stable local estimates.

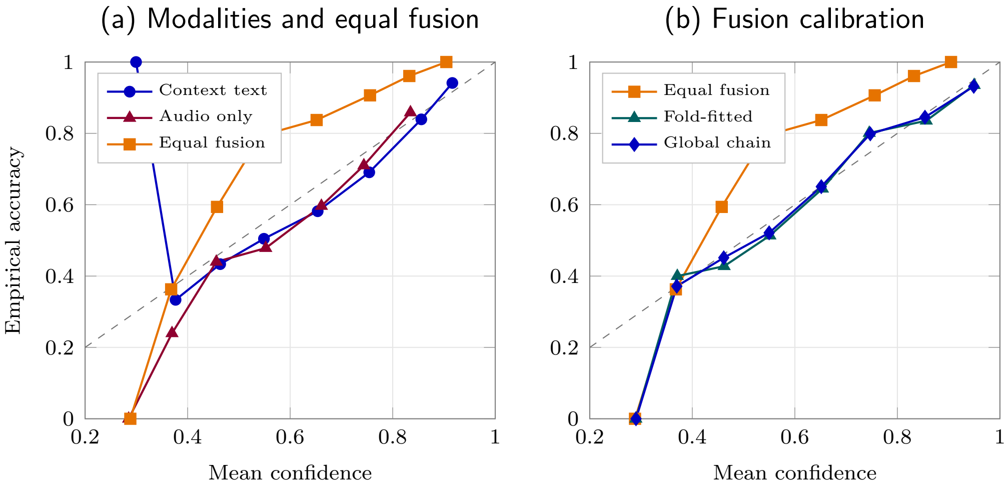


Figure 5.3: Reliability curves from pooled held-out predictions. Each point represents a non-empty equal-width bin of maximum class probability; the dashed diagonal denotes perfect calibration. Panel (a) compares the unimodal systems with predeclared equal fusion, and Panel (b) shows the effect of validation-derived fusion calibration. ECE values in Table 5.5 use the prespecified 15-bin calculation, while the curves use ten bins for legibility.

The final global chain also obtained negative log-likelihood 0.6075 and multiclass Brier score 0.3280. It is the configuration stored for full-data packaging because it provides the single parameter set required by one final text model and one final audio model. However, its pooled calibration fitting uses all out-of-fold validation rows and is an operational configuration analysis, not a new independent confirmatory comparison. The equal-weight result remains the classification headline.

5.5.4 Answer to RQ2

RQ2 is also answered positively within IEMOCAP. Equal-weight late fusion improved macro F1 by 0.0551 over contextual text and by 0.1534 over audio alone, showing complementary information in the textual and acoustic branches. The paired interval against contextual text excluded zero.

The answer includes a material trade-off: the strongest classifier was poorly calibrated, whereas validation-fitted and global calibration configurations sacrificed approximately one percentage point of macro F1 while reducing ECE substantially. AffectLab therefore uses the global validation-derived chain for operational confidence and retains conservative presentation rules. Calibration under IEMOCAP does not guarantee calibration after distribution shift [OFR+19].

5.6 Application Verification and Readiness

Table 5.6 provides a compact trace from the requirements in Chapter 3 to their implementation and verification evidence. Requirements are grouped only where they share the same implementation boundary, verification method, outcome, and principal limitation. Here, “passed” means conformance under the cited developer-led checks, not universal validation beyond the tested configuration.

5.6.1 Deterministic Dialogue Cases

The versioned engineering evaluator contains five authored trajectories for enhanced workload, boundary, and relationship practice. Each specifies the user turns and expected sequence of controller stages. All five enhanced trajectories matched their expected sequence, as shown in Table 5.7. The fixed controller is included only to expose the change in policy granularity; it stops according to its own earlier completion conditions and is not treated as a human-performance baseline.

These cases show that the enhanced controller requires intermediate evidence rather than completing as soon as one generic success phrase is detected. They do not show that its dialogue is more persuasive, realistic, or educational for a participant. Those properties require human evaluation.

5.6.2 Cognitive-Heuristic and Support-Routing Cases

The cognitive and strategy tests are small, implementation-authored examples rather than a labelled evaluation dataset. Table 5.8 states exactly what they establish. The combined sentence deliberately activates several rules at once; the test verifies that the documented precedence and extraction code are stable, not that a human assessor would assign the same psychological labels.

Table 5.6: Compact requirement-to-evidence traceability matrix.

Requirement IDs	**Implementation location**	**Verification method**	**Outcome**	**Principal limitation**
FR1, FR26; NFR5, NFR13	Authentication, persistence, isolation, and deletion (Sections 4.4.3, 4.4.4, and 4.9)	API, security, ownership, stale-write, deletion, and MongoDB recovery tests	Passed	No independent penetration test or deployed backup exercise
FR2, FR3, FR25; NFR3	Onboarding, preferences, takeaways, and action cards (Sections 4.7.2 and 4.6.4)	API and browser onboarding, preference, reflection, and recovery journeys	Passed	No independent usability assessment of these controls
FR4–FR8, FR15, FR19	Preparation, scenario configuration, lifecycle, and dialogue policies (Sections 4.7.3, 4.5.1, and 4.5.2)	Policy/unit, API, desktop/mobile browser, authored-trajectory, and smoke tests	Passed	Three standard scenarios and controlled inputs
FR9, FR11–FR13; NFR2, NFR4	Voice, causal-context inference, fusion, and fallback paths (Sections 4.3, 4.7.5, and 4.9.3)	Context, multimodal, transcription, transient-audio, missing-service, and browser tests	Passed in tested configuration	Limited live-device, provider, and private-model execution
FR10; NFR1	Crisis interruption and safety precedence (Section 4.5.6)	Crisis paraphrase, false-positive, precedence, API, and conversation tests	Passed	A bounded lexical detector cannot cover every expression
FR29	Cognitive heuristics and strategy selection (Subsection 4.4.7)	Authored unit and service cases for pattern, intent, clause extraction, pushback, strategy, and safety precedence	Passed as rule conformance only	No construct, predictive, clinical, comparative-helpfulness, or harm evaluation
FR14; NFR14	Bounded affect pacing and uncertainty presentation (Sections 4.5.4 and 4.7.6)	Seven authored policy cases plus preference, disagreement, failure, and replay tests	Passed	Controlled probabilities; no evidence of user benefit
FR16–FR18, FR20–FR21; NFR6, NFR8	Evidence, response realisation, fallback, and feedback (Sections 4.5.3, 4.5.5, and 4.6.1)	Determinism, evidence-linking, constrained-generation, fallback, feedback, and concurrency tests	Passed	Designed cases do not exhaust language variation
FR22–FR24	Replay, retry, branching, and comparison (Sections 4.6.2 and 4.6.3)	Backend, browser, stale-state, restart-persistence, and full-stack smoke tests	Passed	Synthetic histories rather than natural long-term use
FR27–FR28; NFR7	Consent, study progress, pseudonymous export, and versioning (Section 4.8 and Subsection 4.11.5)	Protocol, eligibility, withdrawal, export-exclusion, dataset-freeze, provenance, and smoke tests	Passed	No independent privacy or reproduction audit
NFR9	Health, bounded failures, recovery, and local responsiveness (Subsections 4.10.4 and 4.11.3)	Recovery journeys, health checks, full-stack restart, and performance regression artefact	Passed locally	No networked concurrent-load or provider-latency test
NFR10	Accessible and responsive interface (Subsections 4.7.2 and 4.11.2)	axe-core scans, keyboard assertions, and desktop/mobile browser runs	Passed tested views	No evaluation with users with disabilities
NFR11–NFR12	Modular and testable architecture (Sections 4.4.1 and 4.11)	Interface inspection, provider substitution, and automated suites across component boundaries	Shown	Maintainability was not measured longitudinally

Table 5.7: Authored enhanced-dialogue trajectory results.

Case	**Expected and observed enhanced stages**	**Final state**
Workload agreement	constraints → agree → resolved	Completed: success
Workload non-confirmation	constraints → agree → agree	Active; no false agreement
Boundary maintained	pressure → resolved	Completed: success
Relationship next step	perspective → agree → resolved	Completed: success
Relationship unresolved	perspective → unresolved	Completed: unresolved

Table 5.8: Implemented cognitive-heuristic and support-routing checks.

Authored input or condition	**Verified implementation result**	**What the check does not establish**
"They will definitely think I am a failure because I missed one deadline. What should I do?"	A cognitive-pattern rule matches; practical-help wins the intent precedence; the clause after *because* is copied as possible context	That the speaker has a cognitive distortion, that the copied clause is the true cause, or that practical advice is wanted
"But that won't work. This is pointless."	The explicit pushback vocabulary produces a resistance value above the selector threshold	Psychological resistance, its degree, persistence, or cause
"How do I handle this?" with the default lexical affect state	Practical-help routing makes practical suggestion the highest-scoring strategy	That the suggestion is helpful, well timed, or superior to clarification or reflection
Crisis flag true with high arousal	Safety escalation wins over all ordinary strategy scores	Sensitivity or specificity of crisis detection, or adequacy of the safety response
Two authored emotional-support messages	The analyser, selector, and offline template compose a non-generic emotion-specific response	Accuracy of the affect or intent label, user preference, therapeutic value, or absence of harm

These checks cover selected positive paths and one precedence condition. They do not systematically test all ten pattern families, all combinations and tie cases, negated and quoted pattern language, malformed causal clauses, demographic or dialectal variation, or user correction of a wrong inference. The browser suite verifies rendering of deterministic decision data but does not independently validate the semantics of the labels. Consequently, the appropriate result is “conforms to the tested rules”, not an accuracy percentage. The subsystem did not contribute an outcome measure to the pilot, and no claim is made that its labels or selected strategies improved the participant experience.

5.6.3 Evidence Extraction and Affect-Pacing Cases

Twelve human-authored feature judgements were compared with the deterministic extractor. The fixture produced five true positives, four true negatives, one false positive, and two false negatives. Reported speech containing “I need” was incorrectly attributed to the current speaker; an indirect refusal and the synonym “shift” were missed. With only twelve designed cases, these counts are diagnostic examples rather than a held-out accuracy estimate. Importantly, the errors expose the intended limitation of a bounded lexical policy and justify replaying the complete source utterance.

Seven controlled affect cases exercised matching confident estimates, modality disagreement, low confidence, missing audio, an unavailable model, disabled adaptation, and an explicit user choice. All seven produced the declared action. Matching high-confidence sadness allowed a brief acknowledgement; disagreement and low confidence offered a pacing choice; unavailable evidence and disabled adaptation preserved the baseline; and the explicit gentler preference took precedence. Separate tests verify that safety, completion, ineligible sessions, and required study tasks override adaptation.

The deterministic evaluator recorded median in-process service latency of approximately 0.64 ms and a lower-order-statistic 95th-percentile value of approximately 3.46 ms across 19 calls. These figures exclude HTTP, MongoDB, provider generation, and trained inference and are not a load test or production latency claim.

5.6.4 Automated and Full-Stack Verification Results

The default backend run performed for the completed implementation reported 240 passing tests and 13 skipped tests. The skipped group contains opt-in checks requiring an external environment, including real MongoDB or private model conditions; it is not counted as a passing result. The frontend production TypeScript build, Vite build, ESLint check, and backend Ruff check completed successfully. These runs verify the current code paths but do not replace the versioned evidence artefacts.

The saved full-stack smoke manifest reports two passing browser journeys against a compiled frontend, actual FastAPI process, and disposable MongoDB 8.0.1 database. The journeys included account access, pre-consent exclusion, enrolment, a required task and questionnaire, researcher export, enhanced dialogue, network-failure recovery, replay, branching, comparison, and action-card persistence. The backend was stopped and relaunched twice while MongoDB remained active, and persisted state was verified after each relevant restart. The recorded total run time was 95.72 seconds; this is demonstration timing, not a capacity benchmark.

Together, these complementary checks support the requirement outcomes summarised in Table 5.6; their principal scope limitations are reported in the same table.

5.6.5 Performance and Responsive-Interface Checks

A reproducible local benchmark exercised the HTTP boundary of the deterministic core using FastAPI's in-process transport and the memory repository. It included routing, bearer-token authentication, request validation, application services, and response serialisation. After warm-up, the benchmark collected 200 samples for read-only endpoints and 100 samples for each state-changing workflow operation. Table 5.9 reports the results from the recorded Windows 11 and Python 3.14.3 run.

Table 5.9: Local single-client application latency results.

Operation	**Samples**	**Median (ms)**	**95th percentile (ms)**
Liveness endpoint	200	1.332	1.866
Authenticated scenario catalogue	200	2.942	3.922
Session creation	100	2.988	3.961
Role-play start	100	3.396	4.437
Deterministic conversation turn	100	4.659	5.836

The benchmark is a fast regression check for local application overhead, not a production-capacity test. It excludes network latency, MongoDB, external generation and transcription providers, trained multimodal inference, and concurrent clients. Those variables depend on the deployed environment and require a separate staged load test. Within its declared scope, however, the result shows that the locally controlled text-first path adds only a few milliseconds per request and that optional services are not on the critical path when deterministic fallbacks are used. All five measurements passed their predeclared 95th-percentile regression budgets. The JSON evidence artefact records the environment, sample counts, budgets, source hashes, individual scope exclusions, and summary statistics.

Perceived responsiveness also depends on the browser rather than API latency alone. The frontend therefore runs as a compiled production build in both desktop Chromium and a Pixel 7 mobile profile. The browser suite verifies responsive layouts, keyboard interaction, focus behaviour, live status messages, preservation of unsent input after failure, and immediate recovery controls. These checks support the claim that the interface is responsive across the tested viewports; they do not replace field measurements of Core Web Vitals on the eventual hosting infrastructure.

5.6.6 Answer to RQ3

RQ3 asks how uncertain multimodal predictions can be integrated while keeping behaviour explainable, controllable, safe, and failure-tolerant. The implementation and engineering evidence provide a constructive answer: the classifier output is validated and stored as evidence; a deterministic policy maps confidence, agreement, availability, eligibility, and user preference to a small action set; the dialogue controller remains authoritative; and each action and fallback reason is persisted for replay.

The authored cases and automated tests show that low confidence and disagreement produce a user choice, missing inference preserves the baseline, explicit preference takes precedence, and safety bypasses model adaptation. Version-bound requests and optimistic persistence attach a prediction to its exact exchange. This establishes specified system behaviour under the tested conditions. It does not establish that the emotion label itself is correct for a real participant or that pacing improves their outcome.

5.6.7 Answer to RQ4: Integrated Application Quality

RQ4 is answered through the combined evidence traced in Table 5.6. The automated, browser, full-stack, recovery, and authored-policy checks passed in the tested configuration, while the locally controlled text-first operations remained below 7 ms at the 95th percentile. These results support readiness for a controlled production deployment with that configuration.

This conclusion is deliberately environment-specific. It does not claim internet-scale capacity or guarantee the latency of MongoDB, trained inference, or external providers. A public deployment should establish its own service-level objectives and repeat concurrent-load, network, backup, security, and observability checks in staging.

5.6.8 Supplementary User-Pilot Results

The supplementary single-arm pilot examined completion of the text-based rehearsal workflow and collected brief usability ratings. The analysis uses the AffectLab feasibility protocol v1.1 export, with the first attempt at each required intermediate-difficulty scenario selected for the protocol outcomes. The analytic export includes 15 participants and 46 rehearsal sessions, of which 42 were first required attempts and four were additional or retry sessions. Of the required attempts, 39 reached task completion (14 workload, 13 boundary, and 12 relationship). Twelve participants completed all three required scenarios and submitted the corresponding post-rehearsal questionnaires, giving a protocol-completion rate of 80.0% (12/15; two-sided 95% Wilson interval 54.8%–93.0%). The point estimate met the protocol's 80% feasibility target, although the interval remained wide at this sample size.

Table 5.10 summarises the available study-specific ratings. The post-rehearsal usefulness and realism scores had means of 5.33 and 4.85, respectively, on seven-point scales. Among 38 observations with both confidence ratings available, representing 14 participants, the mean within-task confidence change was +0.97 points (median +1; participant-cluster bootstrap 95% interval +0.67 to +1.26). The mean paired changes were +0.85 for workload ($n = 13$), +0.85 for boundary ($n = 13$), and +1.25 for relationship ($n = 12$). These within-task differences reflect immediate self-report, not independently verified communication improvement or an effect attributable to the application.

Table 5.10: Supplementary pilot questionnaire results from available first required attempts. Ratings use seven-point scales; paired change is post-confidence minus pre-confidence.

Measure	n	**Mean**	**SD**	**Median**	**Q1–Q3**
Pre-rehearsal confidence	40	3.80	1.04	4.0	3.0–4.0
Pre-rehearsal anxiety	40	4.58	1.28	5.0	4.0–5.25
Post-rehearsal confidence	40	4.75	1.37	4.5	4.0–6.0
Scenario realism	40	4.85	0.98	5.0	4.0–5.25
Feedback usefulness	40	5.33	0.92	5.0	5.0–6.0
Paired confidence change	38	+0.97	0.97	+1.0	0.25–1.75

The deterministic feedback also produced scenario-specific lexical indicators on the 39 completed required tasks. Mean scores included 42.3% for clear requests in workload rehearsal ($n = 14$), 59.0% for clear refusal or maintained boundaries ($n = 13$), and 44.4% for use of an I-statement in relationship rehearsal ($n = 12$). The configured absence-of-blame indicators reached 100% in workload and relationship tasks, and the no-repeated-apology indicator reached 100% in boundary tasks. These are averages of rule-defined scores rather than percentages of participants demonstrating validated skills; the indicators were not independently annotated or established as measures of real-world communication ability. They provide limited behavioural context to the stronger engineering evidence for the feedback implementation.

Two required attempts lacked a post-questionnaire, and only 38 of the 42 required attempts contained a valid pair of confidence ratings. Consequently, questionnaire summaries use available observations and report their denominators rather than imputing responses. The pilot offers preliminary evidence of workflow uptake and perceived usefulness, but the small convenience sample and uncontrolled design preclude causal, clinical, or generalisable effectiveness claims. It is therefore treated as a supplement to the model and system-verification results, not as the principal basis for deployment readiness.

5.7 Threats to Validity and Limitations

5.7.1 Model and Statistical Validity

The five test folds cover all IEMOCAP sessions, but the corpus contains only ten actors. Fold results are therefore dependent on a small set of speakers, acted or improvised tasks, recording conditions, and annotation practices. The dialogue bootstrap reflects the 151 observed conversations but cannot compensate for missing demographic, cultural, linguistic, and environmental diversity. Only one primary training seed was used after the pipeline check, so optimisation variance is not fully characterised.

Repeated GPU execution can produce small numerical variation. The reported paired comparisons use the latest internally consistent private prediction files, each checked against its neighbouring metrics record. This preserves internal pairing but illustrates why commit, configuration, and file hashes are necessary. The final full-data packages cannot be evaluated on their training rows, and their file-level provenance remains incomplete until the private directories are downloaded and hashed beside the application.

ECE is bin-dependent, and calibration can deteriorate under shift. The global deployment chain is fitted from pooled validation roles across the corpus; it is useful for choosing one full-data configuration but is not an independent external calibration study. The 0.55 and 0.75 presentation bands are operational thresholds rather than validated psychological boundaries.

5.7.2 External and Construct Validity

IEMOCAP labels describe annotated expressive categories, not direct measurements of internal emotion. Mapping excitement to happiness and fear to exploratory anxiety further simplifies the constructs. Model performance must not be interpreted as diagnosis, emotional truth, or equitable performance across accents, languages, devices, ages, disabilities, or cultures.

The role-play features are likewise operational constructs. A phrase detector can identify authored patterns such as a clear refusal or practical request, but it does not measure general communication ability or transfer to a real conversation. The diagnostic fixture already demonstrates missed synonyms, indirect language, and reported-speech error. Deterministic feedback is more auditable than an unconstrained judgement, but it remains limited by these definitions.

The rule-based cognitive fields have an even narrower evidential basis. Familiar labels such as cognitive distortion, intent, cause, readiness, and resistance may suggest assessment of an internal state, but the implementation observes only the current message and assigns fixed values when short patterns occur. No clinician, validated questionnaire, independent annotator, held-out corpus, or participant self-report was used as a reference standard. The tests reported in Subsection 5.6.2 establish software conformance, not accuracy, construct validity, or clinical validity. Because the pilot neither isolated this subsystem nor measured its helpfulness or harms, the selected support strategy cannot be claimed to be optimal or beneficial. False and overly authoritative labels remain a risk, especially for negation, quotation, indirect language, cultural variation, and messages containing several possible requests.

The engineering evaluation largely uses deterministic templates and test accounts. It does not evaluate live provider variability, private-model latency under concurrent participant use, real microphone noise, or long-duration deployment. The successful smoke journey establishes integration and restart persistence only for its tested path.

5.7.3 Supplementary User-Evidence Limitations

The supplementary pilot was single-arm and included only 15 participants in the analytic export, limiting precision and generalisability. The wide confidence interval around protocol completion illustrates this uncertainty. Missing ratings, incomplete rehearsals, fixed scenario order, repeated exposure, self-selection, and demand characteristics may influence the descriptive findings. The study-specific ratings are not validated clinical scales, and the deterministic lexical scores were not validated against independent assessments of communication quality. Without a control condition or follow-up assessment, the observed immediate confidence change cannot establish causation, sustained skill development, or transfer to real conversations. These limitations are why the pilot contributes contextual usability evidence rather than the principal validation of the application.

5.8 Chapter Summary

The evaluation provides completed answers to the two machine-learning questions. Three-turn causal context improved the four-class text model from 0.6651 to 0.7332 macro F1, with a paired dialogue-bootstrap interval excluding zero. Equal-weight text–audio fusion further improved macro F1 to 0.7883, demonstrating complementary benchmark information. However, its ECE of 0.1619 exposed a classification–calibration trade-off. Validation-derived calibration reduced ECE substantially at a modest classification cost, motivating the global text-dominant configuration used for final packaging.

The application evaluation answers RQ3 and RQ4 at the level of specified, reproducible system behaviour. All five authored enhanced dialogue trajectories and all seven affect-pacing decisions matched expectations. Diagnostic evidence cases exposed one false positive and two false negatives rather than concealing lexical limitations. A 240-test backend run, production frontend checks, desktop/mobile browser and accessibility tests, two full-stack journeys, persistence across backend restarts, and local HTTP timings verify complementary functional and non-functional boundaries. The deterministic conversation path recorded a median of 4.659 ms and a 95th percentile of 5.836 ms in the scoped local benchmark.

The small pilot remains supporting evidence: 12 of 15 participants completed the three required rehearsals and questionnaires, while mean realism and usefulness ratings were 4.85 and 5.33 out of seven. These descriptive observations help identify workflow strengths and possible refinements but do not determine the quality or readiness of the entire application. Overall, AffectLab is a fully implemented communication-practice system whose model behaviour, core policies, interfaces, persistence, accessibility, recovery, and local responsiveness have been tested extensively by the developer and through automation, with deployment-scale performance appropriately reserved for environment-specific staging tests.

Chapter 6

Conclusions and Future Work

6.1 Chapter Overview

This dissertation investigated how multimodal emotion recognition can be incorporated into a helpful application for rehearsing difficult interpersonal conversations without allowing uncertain predictions to control consequential decisions. The work combined speaker-independent emotion-recognition experiments, an uncertainty-aware interaction policy, structured role-play scenarios, evidence-grounded feedback, a production-oriented full-stack application, and a layered developer-led verification strategy. AffectLab demonstrates one way of using affective information as bounded conversational evidence rather than as a declaration of a user's internal state.

This final chapter synthesises the findings in relation to the four research questions, identifies the principal contributions, and restates the boundaries of the conclusions. It then proposes a prioritised programme of future work spanning deployment-scale performance, operational governance, model generalisation, policy comparison, feedback validation, accessibility, and longer-term product research. The chapter closes by considering what the implemented and tested application establishes and what remains environment- or study-dependent.

6.2 Summary of the Dissertation

The dissertation began from two connected problems. First, emotion recognition in conversation is uncertain, sensitive to context, and vulnerable to distribution shift. Second, a role-play application needs predictable progression, evidence-grounded feedback, safety precedence, and user control even when machine-learning or generative services are unavailable. Treating these as separate problems would make it easy either to build a strong benchmark classifier with no responsible interaction design or to build an interactive application whose affective behaviour could not be evaluated independently.

AffectLab addresses this tension through separation of responsibilities. Contextual text and speech models estimate an affective probability distribution. A versioned adaptation policy considers confidence, agreement, availability, eligibility, and user preference before selecting a small pacing action. Deterministic scenario controllers remain authoritative for dialogue stage, completion, and unresolved outcomes. Observable language features determine feedback metrics, while an optional language model is restricted to response realisation. Safety handling bypasses affect adaptation, and missing external services preserve the baseline text workflow.

The implementation supports three fixed scenarios: workload negotiation, boundary-setting, and expression of relationship needs. It also provides preparation, replay, rewind, branching, comparison, takeaways, and action cards so that a rehearsal can be inspected and revisited. Authentication, ownership checks, retention controls, transient voice handling, health checks, production configuration validation, containerised deployment, deterministic fallbacks, backup verification, and optional research-data controls make the application suitable for controlled production use and future study.

Evaluation was deliberately divided by claim. IEMOCAP experiments support classification and calibration findings on the benchmark. Authored trajectories, 240 passing backend tests, production frontend checks, desktop/mobile browser and accessibility suites, database-backed full-stack journeys, restart recovery, and local HTTP timings support application-quality claims. The scoped performance run recorded a 4.659 ms median and 5.836 ms 95th percentile for deterministic conversation turns. Section 5.6.8 retains the small pilot results as supplementary user feedback, separate from the primary model and application evidence.

6.3 Conclusions for the Research Questions

6.3.1 RQ1: The Contribution of Conversational Context

RQ1 asked whether preceding conversational turns improve text-based emotion recognition compared with independent utterance classification. Under the declared four-class, speaker-independent IEMOCAP protocol, the answer is yes. Adding the previous three causal dialogue turns increased accuracy from 0.6624 to 0.7304 and macro F1 from 0.6651 to 0.7332. The paired dialogue-bootstrap macro-F1 difference was +0.0681, with a 95% interval from 0.0504 to 0.0857. Accuracy and weighted-F1 intervals also excluded zero.

The result is strengthened by the controlled comparison: architecture, label set, data partitions, seed, and optimisation settings were held constant, leaving the input context as the intended experimental difference. The exploratory six-class comparison produced the same aggregate direction, although its anxiety category was based on only 40 mapped fear examples and cannot support a stable anxiety-recognition claim.

The conclusion remains task-specific. Context improved annotated emotion classification in IEMOCAP, but the experiment does not establish that the model recovers a person's private emotional state or that the same gain will transfer to spontaneous AffectLab conversations. Per-class results also show why the aggregate conclusion should not be generalised indiscriminately: the anger improvement was small and its interval included zero. The contribution of context is therefore a benchmark classification finding rather than a claim of general emotional understanding.

6.3.2 RQ2: Multimodal Complementarity and Calibration

RQ2 asked whether contextual text and acoustic information are complementary and what trade-off exists between classification and confidence calibration. The audio-only model achieved macro F1 0.6349, below the contextual text result of 0.7332. Nevertheless, combining the two calibrated probability distributions with the predeclared equal-weight rule increased macro F1 to 0.7883 and accuracy to 0.7832. The macro-F1 improvement over contextual text was +0.0551, with a dialogue-bootstrap 95% interval from 0.0407 to 0.0713; all 10,000 bootstrap replicates favoured fusion. Acoustic information therefore contributed complementary benchmark information even though it was the weaker modality in isolation.

The strongest classifier was not the best calibrated configuration. Equal-weight fusion produced ECE 0.1619, compared with 0.0311 for contextual text. Validation-fitted fusion reduced ECE to 0.0283 while retaining macro F1 0.7758. The final full-data calibration chain, selected for operational packaging rather than confirmatory comparison, obtained pooled validation-role macro F1 0.7776 and ECE 0.0242 with a text-dominant weight of 0.59.

These results answer both parts of RQ2. Text and speech were complementary for classification, but probability combination sharpened confidence enough to require a separate calibration decision. Consequently, an application should not choose a fusion configuration solely because it maximises test-set F1. The distinction between classification performance and operational confidence is central when probabilities influence thresholds, user-facing uncertainty, or adaptive behaviour.

6.3.3 RQ3: Bounded Integration of Affective Evidence

RQ3 concerned the responsible integration of uncertain multimodal predictions into structured role-play. The project provides a constructive engineering answer. AffectLab stores the complete prediction and provenance, validates it against the current session and turn, and maps it through a deterministic policy to one of a restricted set of pacing actions. A fused or component confidence below the configured low-confidence boundary, or modality disagreement, offers the user a pacing choice; matching anger or sadness estimates for which all three confidences meet that boundary may permit a modest acknowledgement, including in the moderate presentation band; an explicit preference can preserve baseline behaviour; unavailable inference falls back without blocking the conversation; and safety interruption takes precedence over all affective adaptation.

This separation prevents the classifier from deciding whether a scenario succeeds, what feedback score is assigned, whether a participant is eligible, or whether a safety response is required. Optional language generation can phrase a selected action but cannot replace the recorded decision. Optimistic persistence, request identifiers, model versions, source fields, and fallback reasons make the resulting behaviour replayable and auditable.

The engineering evaluation supports this answer within the tested conditions. All five authored enhanced dialogue trajectories and all seven affect-pacing cases matched their expected outcomes. The full automated backend suite passed, and two integrated browser journeys completed against the disposable database while surviving actual backend restarts. These results establish policy conformance, recovery, ownership, persistence, and failure tolerance. They do not establish that an affect prediction was correct for a human user or that adaptation improves a user's rehearsal outcome.

6.3.4 RQ4: Integrated Application Quality and Readiness

RQ4 asked whether the completed application satisfies its declared functional and non-functional requirements under reproducible engineering evaluation. The answer is positive for the tested configuration. The backend suite reported 240 passes, the frontend passed strict production compilation and linting, browser checks exercised desktop and mobile layouts with automated accessibility scanning, and two full-stack journeys verified real API serialisation, MongoDB persistence, network-failure recovery, branching, export, and state preservation across backend restarts. Authored cases additionally verified the deterministic dialogue and affect-pacing policies.

Responsiveness was tested separately from correctness. In a single-client, in-process HTTP benchmark, liveness, scenario retrieval, session creation, role-play start, and a deterministic conversation turn all remained below 7 ms at the 95th percentile. The benchmark excludes network transit, MongoDB, trained inference, providers, and concurrent users, so it supports a local-regression claim rather than internet-scale capacity. Taken together with fail-fast production configuration, health checks, ownership protection, optimistic concurrency, fallbacks, backup exercises, and containerisation, the evidence supports readiness for a controlled production deployment. Public scale still requires environment-specific load, security, observability, and incident-response validation.

6.4 Principal Contributions

6.4.1 Empirical and Machine-Learning Contributions

The first contribution is a controlled, speaker-independent comparison of utterance-only and causal-context text emotion recognition. By pairing predictions at dialogue level and reporting aggregate, per-class, and exploratory six-class results, the study demonstrates a clear contextual benefit while preserving class-specific uncertainty.

The second contribution is a comparative text–audio pipeline that retains unimodal baselines and distinguishes predeclared fusion from validation-fitted operational configurations. Reporting calibration beside accuracy and F1 exposes a trade-off that would be hidden by classification metrics alone. The final pipeline also records the temperatures, modality weights, label order, and provenance needed to reproduce the operational transformation from logits to probabilities.

6.4.2 System and Interaction-Design Contributions

The principal system contribution is a hybrid architecture in which trained models and generative services can influence presentation without becoming authoritative for dialogue outcomes, assessment, or safety. The uncertainty-aware affect policy turns confidence, agreement, availability, and preference into inspectable behaviour. Deterministic fallbacks preserve a useful interaction when optional services fail.

The three scenario controllers contribute explicit models of different interpersonal objectives. Workload negotiation seeks a practical prioritisation decision, boundary-setting recognises a respectfully maintained refusal without requiring compliance, and relationship practice seeks a specific need or next step while allowing disagreement to remain unresolved. This avoids reducing every role-play to one generic success condition.

Evidence-grounded feedback and reflective tools form a further applied contribution. Metrics point to specific conversational turns rather than issuing an unexplained global judgement. Replay, rewind, branching, and comparison support counterfactual practice from the same context. Together, these features make a conversation an inspectable practice artefact instead of a transient exchange ending in an opaque score.

6.4.3 Application-Verification and Infrastructure Contributions

The dissertation contributes a layered verification framework that keeps model performance, application conformance, performance scope, and supporting user feedback separate. This prevents benchmark accuracy from being interpreted as product reliability and prevents a small pilot from overshadowing the much broader technical verification. The framework combines deterministic fixtures, automated unit and API suites, real persistence checks, desktop/mobile browser and accessibility tests, production builds, recovery journeys, source-hashed evidence artefacts, and a repeatable latency benchmark.

The full-stack application also provides deployment and research infrastructure. Production configuration fails closed on weak secrets, origins, hosts, HTTPS, and rate limiting; health endpoints distinguish core readiness from optional model availability; Docker packaging, database persistence, backups, ownership enforcement, transient audio, and deterministic fallbacks are implemented rather than described only as future requirements. Separate consent, pseudonymous export, withdrawal, and dataset freezing remain available when the optional research workflow is used.

Finally, the evidence package and versioned configurations improve auditability. Model metrics, paired comparisons, policy fixtures, automated tests, performance summaries, smoke manifests, and protocol files identify which claim is supported by which artefact. Some private model-package hashes remain to be finalised, but the overall approach demonstrates how a production-oriented dissertation application can make its evidence boundaries inspectable.

6.5 Limitations of the Conclusions

6.5.1 Model and Data Limitations

The emotion-recognition conclusions derive from IEMOCAP, a corpus of ten actors completing acted or improvised English interactions. The dataset does not represent the full variation of spontaneous emotion, language, accent, age, disability, culture, device quality, or background noise that AffectLab may encounter. The four-class mapping simplifies expressive categories, and the exploratory mapping from fear to anxiety is especially weak. One primary training seed limits the assessment of optimisation variability.

Speaker-independent session folds reduce identity leakage but do not constitute external validation. Calibration can also deteriorate under domain shift even when it is strong on pooled validation roles. The final full-data packages have no independent test set because all eligible benchmark examples are used for training after configuration selection. Their operational thresholds must therefore be treated as design parameters, not validated psychological boundaries.

6.5.2 Application and Measurement Limitations

The dialogue policies cover three bounded scenarios and cannot represent the ambiguity, power imbalance, culture, history, or stakes of all interpersonal situations. Feature extraction is transparent but lexically limited: paraphrases can be missed, indirect speech can be misread, and detected phrases do not establish communication competence. Generative responses remain dependent on an external provider when enabled, while template fallbacks trade linguistic variety for predictability.

The separately implemented cognitive-pattern, intent, possible-cause, readiness, and resistance fields are lexical support-routing heuristics, despite terminology that may imply psychological assessment. Their fixed rules have no construct- or clinical-validity evidence, and the present tests establish only selected input–output behaviour. The pilot did not isolate the feature or evaluate whether its labels and strategy choices were helpful, inaccurate, stigmatising, or harmful. The dissertation therefore makes no claim that these fields reveal cognition, identify a true cause or intention, quantify readiness or resistance, or select an optimal intervention.

Engineering evaluation used authored cases, deterministic services, synthetic accounts, and a small number of integrated journeys. The local latency benchmark used an in-process transport and memory repository. These establish behaviour and regression performance on the tested paths but not prolonged concurrent capacity, internet latency, real provider response time, adversarial traffic, diverse microphones, or every assistive technology. A controlled production deployment is supported by the present evidence, while wider or higher-risk deployment still requires independent security review, monitoring, incident response, secret management, and jurisdiction-specific governance.

6.5.3 Supporting User-Evidence Limitations

The supplementary pilot included ten enrolled users, only six of whom completed all three required tasks. It had no randomised control condition and used study-specific ordinal items. Incomplete tasks, missing ratings, demand characteristics, repetition, familiarity, and self-selection may all influence the summaries. The pilot therefore informs product refinement but does not establish sustained skill acquisition, causal benefit, or transfer to real conversations. AffectLab is not a diagnostic, clinical, crisis, or therapeutic system.

6.6 Future Work

Future work should proceed in an order that strengthens the weakest evidence before increasing system autonomy. Table 6.1 summarises the proposed sequence.

Table 6.1: Prioritised future-work roadmap.

Priority	**Work package**	**Evidence produced**
Immediate	Staging performance, load, and resilience evaluation	Response-time distributions, throughput, saturation points, recovery time, and resource use with production persistence, models, providers, and expected concurrency.
Near term	External and stress-condition model evaluation	Cross-corpus classification, calibration under shift, device and noise robustness, and appropriately powered subgroup analyses.
Near term	Validate dialogue features and feedback with human annotation	Construct validity, inter-rater agreement, paraphrase coverage, error categories, and revised evidence rules.
Near term	Reframe or validate cognitive support-routing labels	Decision on removal or plain-language renaming, expert and user review, false-positive and false-negative cases, and comparison with a neutral routing baseline.
Medium term	Controlled comparison of baseline and affect-aware interaction	Supporting evidence about whether bounded adaptation changes pacing preference, task completion, perceived helpfulness, or intrusiveness.
Longer term	Longitudinal, multilingual, and participatory accessibility work	Retention, transfer, broader inclusion, privacy, and governance evidence.

6.6.1 Validate Deployment Performance and Reliability

The immediate next step is to repeat the local performance check in a staging environment that matches the intended deployment. This should use the production Nginx–FastAPI–MongoDB path, packaged text and audio models, configured external providers, realistic network conditions, and representative session histories. Warm and cold starts, deterministic fallback turns, multimodal inference, transcription, replay, export, and branch creation should be measured separately so that a slow optional service is not hidden inside an aggregate average.

A concurrent-load tool should report throughput, median and tail latency, error rate, queue time, CPU, memory, and database utilisation as client count increases. Service-level objectives should be declared before the run, and tests should identify the saturation point rather than merely recording one successful request. Soak testing, forced provider timeouts, database restart, backup restoration, and process replacement should additionally measure recovery behaviour and verify that incomplete requests do not create duplicate turns or corrupt session versions.

Browser performance should be measured on the deployed build using Core Web Vitals and interaction traces on desktop and representative mobile hardware. These staging results would complement the current responsive-layout and accessibility suite and turn the existing production-ready architecture into deployment-specific operational evidence.

6.6.2 Broaden Model Validation and Improve Uncertainty Handling

The model pipeline should be tested across corpora and, with explicit consent, on data closer to the intended use context. Cross-corpus evaluation would reveal how much of the IEMOCAP result depends on actors, recording conditions, and label conventions. Stress tests should vary microphones, compression, background noise, transcript errors, utterance length, and missing modalities. Multilingual work should begin with a new construct and annotation review rather than directly transferring the English label mapping.

Repeated training seeds and stronger baselines would better quantify optimisation uncertainty. Future comparisons could investigate longer or learned context selection, speaker-aware aggregation, modality-specific missingness training, and alternative late-fusion rules. These additions should retain paired evaluation and should not tune headline claims on the test partition.

Calibration requires continued attention after deployment. Reliability should be monitored by source, device condition, and data period where lawful and statistically justified. Selective prediction, abstention, or distribution-shift indicators may be more appropriate than forcing every input into an affect label. Thresholds should be revised only with held-out or prospectively collected evidence, and the application should preserve an accessible baseline regardless of model availability.

6.6.3 Evaluate and Refine the Adaptation Policy

The existing policy is intentionally small and deterministic. Future evaluation should ablate its inputs to determine whether confidence, modality agreement, and explicit preference each contribute useful information. Logs can identify how often actions are selected, declined, reversed, or followed by manual pacing changes. Qualitative interviews could reveal whether acknowledgements feel supportive, inaccurate, intrusive, or distracting.

Personalisation should remain transparent and opt-in. A user might prefer fewer affect acknowledgements, text-only operation, slower pacing, or different uncertainty wording. Such preferences can be stored as explicit controls rather than inferred as stable psychological traits. Any learned policy should first be evaluated offline against safety and autonomy constraints and compared with the deterministic baseline. Reinforcement learning based on engagement alone would be inappropriate because longer interaction is not equivalent to better rehearsal.

6.6.4 Validate Feedback and Communication Measures

The deterministic feature rules should be compared with independent human annotations of the same turns. This would quantify precision, recall, agreement, synonym coverage, and systematic error for features such as a clear request, boundary maintenance, non-blaming language, and a specific need. The analysis should include indirect phrasing, negation, reported speech, culturally varied wording, and speech-to-text errors.

The cognitive support-routing component should be treated as a separate decision point rather than silently included in that validation. One option is removal or replacement of psychological terms with directly observable descriptions such as "asks for steps", "contains a causal connector", or "pushes back on a suggestion". If the named constructs are retained, the work would require operational definitions, appropriate domain expertise and ethics review, independently annotated and representative cases, participant correction mechanisms, error and subgroup analysis, and a comparison showing that construct-labelled routing adds benefit over a neutral intent menu or clarification question. Clinical wording or use should remain out of scope unless supported by an appropriately governed clinical study.

Semantic or learned detectors could improve paraphrase coverage, but they should return evidence spans and uncertainty rather than opaque trait scores. Feedback usefulness should also be evaluated separately from detector accuracy: a technically correct observation may still be poorly timed or unhelpful. Co-design with communication educators and target users could refine wording while preserving the distinction between observable behaviour and personal judgement.

6.6.5 Study Transfer, Accessibility, and Long-Term Use

Later studies should examine whether users return voluntarily, whether repeated practice remains useful, and whether any measured change persists. Transfer to real conversations would require independent outcomes, such as user-reported application of a planned request or externally rated role-plays, not merely improvement on the same scenario. Such studies should avoid clinical claims unless they adopt an appropriate clinical design and oversight.

Accessibility evaluation should involve users of keyboards, screen readers, speech input, captions, reduced-motion settings, and different cognitive or communication needs. The current interface and automated checks provide a foundation but cannot replace participatory accessibility testing. Broader language and cultural adaptation should include local review of scenario norms, refusal styles, power relationships, consent materials, and safety resources.

6.6.6 Strengthen Deployment, Privacy, and Governance

Before expanding beyond a controlled deployment, the system should complete staging load tests, an independent security review, production backup-and-recovery exercises, dependency and model monitoring, operational alerting, and documented incident response. Provider changes should be treated as versioned operational factors because they can alter transcription, generation, latency, or failure modes. Model and policy artefacts should be signed or hashed, and the remaining private-package provenance gaps should be closed.

Data minimisation should continue to guide development. Audio should remain transient unless a separately justified protocol requires retention, and derived research records should avoid reconstructable conversation content. Retention, deletion, export access, and withdrawal should be tested periodically in the deployed environment. Governance should also define who may change scenarios, thresholds, prompts, safety resources, or analysis code and how those changes are reviewed and recorded.

6.7 Final Conclusion

This dissertation shows that multimodal affect recognition can be connected to conversational role-play without making the classifier the authority over the interaction. Causal dialogue context improved textual emotion classification on the chosen benchmark, and audio supplied complementary information that improved fused classification. The accompanying calibration results demonstrated that stronger classification does not automatically produce trustworthy confidence. This finding shaped the application design: uncertainty is retained, adaptation is bounded, users can decline it, and missing inference preserves the core workflow.

AffectLab's broader contribution is therefore practical, architectural, and methodological as much as predictive. Deterministic scenario progression, evidence-grounded feedback, constrained generation, safety precedence, provenance, and complementary verification layers make the system's behaviour more inspectable. The completed engineering evidence establishes that this design works end to end across its tested policies, interfaces, persistence, accessibility, recovery, and local-response paths. It is a helpful communication-practice application, not merely a model demonstration.

The completed system is ready for controlled production deployment under the tested configuration: it has production builds and configuration checks, health and recovery paths, security and ownership controls, deterministic fallbacks, broad automated coverage, real-database smoke journeys, desktop/mobile accessibility checks, and measured local responsiveness. That claim remains narrower than internet-scale capacity or guaranteed third-party latency, which must be established in the target environment. Likewise, benchmark performance does not establish emotional truth, and the small uncontrolled pilot does not establish participant benefit. Within those boundaries, the dissertation delivers a thoroughly developer-tested and reproducibly evaluated application for emotion-aware communication rehearsal while preserving uncertainty and user agency.

Bibliography

[BAM19a] Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2):423–443, 2019.

[BAM$^+$19b] Lisa Feldman Barrett, Ralph Adolphs, Stacy Marsella, Aleix M. Martinez, and Seth D. Pollak. Emotional expressions reconsidered: Challenges to inferring emotion from human facial movements. *Psychological Science in the Public Interest*, 20(1):1–68, 2019.

[BBL$^+$08] Carlos Busso, Murtaza Bulut, Chi-Chun Lee, Abe Kazemzadeh, Emily Mower, Samuel Kim, Jeannette N. Chang, Sungbok Lee, and Shrikanth S. Narayanan. IEMOCAP: Interactive emotional dyadic motion capture database. *Language Resources and Evaluation*, 42(4):335–359, December 2008.

[BR09] Dan Bohus and Alexander I. Rudnicky. The RavenClaw dialog management framework: Architecture and systems. *Computer Speech & Language*, 23(3):332–361, 2009.

[BSN$^+$12] Hans Martin Bosse, Jobst-Hendrik Schultz, Martin Nickel, Thomas Lutz, Andreas Möltner, Jana Jünger, Sören Huwendiek, and Christoph Nikendei. The effect of using standardized patients or peer role play on ratings of undergraduate communication training: A randomized controlled trial. *Patient Education and Counseling*, 87(3):300–306, June 2012.

[BZMA20] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[CWC$^+$22] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Jian Wu, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, Xiangzhan Yu, and Furu Wei. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019.

[DFC24] Julian De Freitas and I. Glenn Cohen. The health risks of generative AI-based wellness apps. *Nature Medicine*, 30:1269–1275, 2024.

[EAKK11] Moataz El Ayadi, Mohamed S. Kamel, and Fakhri Karray. Survey on speech emotion recognition: Features, classification schemes, and databases. *Pattern Recognition*, 44(3):572–587, 2011.

[Ekm92] Paul Ekman. An argument for basic emotions. *Cognition and Emotion*, 6(3–4):169–200, 1992.

[ESS$^+$16] Florian Eyben, Klaus R. Scherer, Björn W. Schuller, Johan Sundberg, Elisabeth André, Carlos Busso, Laurence Y. Devillers, Julien Epps, Petri Laukka, Shrikanth S. Narayanan, and Khiet P. Truong. The geneva minimalistic acoustic parameter set (GeMAPS) for voice research and affective computing. *IEEE Transactions on Affective Computing*, 7(2):190–202, 2016.

[Eur16] European Parliament and Council of the European Union. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation). Official Journal of the European Union, L 119, 2016. Accessed 6 October 2026.

[Eur24] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, 2024.

[GMG$^+$20] Deepanway Ghosal, Navonil Majumder, Alexander Gelbukh, Rada Mihalcea, and Soujanya Poria. COSMIC: COmmonSense knowledge for eMotion identification in conversations. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2470–2481. Association for Computational Linguistics, 2020.

[GMP$^+$19] Deepanway Ghosal, Navonil Majumder, Soujanya Poria, Niyati Chhaya, and Alexander Gelbukh. DialogueGCN: A graph convolutional neural network for emotion recognition in conversation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 154–164. Association for Computational Linguistics, 2019.

[GPSW17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.

[HBT+21] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

[HGC21] Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. arXiv preprint arXiv:2111.09543, 2021.

[HT07] John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007.

[KPNK+19] Meelis Kull, Miquel Perello-Nieto, Markus Kängsepp, Telmo Silva Filho, Hao Song, and Peter Flach. Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration. In *Advances in Neural Information Processing Systems*, volume 32, pages 12316–12326, 2019.

[LJG25] Dongxu Lu, Johan Jeuring, and Albert Gatt. Evaluating LLM-generated versus human-authored responses in role-play dialogues. In *Proceedings of the 18th International Natural Language Generation Conference*, pages 20–40. Association for Computational Linguistics, 2025.

[LKK+20] Juhye Lee, Haerim Kim, Kyoung Hwa Kim, Daesung Jung, Tanisha Jowsey, and Craig S. Webster. Effective virtual patient simulators for medical communication training: A systematic review. *Medical Education*, 54(9):786–795, September 2020.

[LL22] Joosung Lee and Wooin Lee. CoMPM: Context modeling with speaker's pre-trained memory tracking for emotion recognition in conversation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5669–5679. Association for Computational Linguistics, 2022.

[LOG+19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint, arXiv:1907.11692, 2019.

[LSR+24] Inna Lin, Ashish Sharma, Christopher Rytting, Adam Miner, Jina Suh, and Tim Althoff. IMBUE: Improving interpersonal effectiveness through simulation and just-in-time feedback with human-language model interaction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 810–840, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[LSS+17] Yanran Li, Hui Su, Xiaoyu Shen, Wenjie Li, Ziqiang Cao, and Shuzi Niu. DailyDialog: A manually labelled multi-turn dialogue dataset. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 986–995. Asian Federation of Natural Language Processing, 2017.

[LWC+22] Jinchao Li, Shuai Wang, Yang Chao, Xunying Liu, and Helen Meng. Context-aware multimodal fusion for emotion recognition. In *Proceedings of Interspeech 2022*, pages 2013–2017, Incheon, Republic of Korea, September 2022. International Speech Communication Association.

[MPH+19] Navonil Majumder, Soujanya Poria, Devamanyu Hazarika, Rada Mihalcea, Alexander Gelbukh, and Erik Cambria. DialogueRNN: An attentive RNN for emotion detection in conversations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 6818–6825, 2019.

[OFR+19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[PHM+19] Soujanya Poria, Devamanyu Hazarika, Navonil Majumder, Gautam Naik, Erik Cambria, and Rada Mihalcea. MELD: A multimodal multiparty dataset for emotion recognition in conversations. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 527–536. Association for Computational Linguistics, 2019.

[RSLB19] Hannah Rashkin, Eric Michael Smith, Margaret Li, and Y-Lan Boureau. Towards empathetic open-domain conversation models: A new benchmark and dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5370–5381. Association for Computational Linguistics, 2019.

[Rus80] James A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980.

[Sch05] Klaus R. Scherer. What are emotions? and how can they be measured? *Social Science Information*, 44(4):695–729, 2005.

[TBL+19] Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J. Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6558–6569. Association for Computational Linguistics, 2019.

[TSG25] Hovhannes Tamoyan, Hendrik Schuff, and Iryna Gurevych. LLM roleplay: Simulating human-chatbot interaction. In *Proceedings of the*

Third Workshop on Social Influence in Conversations, pages 1–26. Association for Computational Linguistics, 2025.

[Wor25] World Health Organization. From loneliness to social connection: Charting a path to healthier societies. Technical report, World Health Organization, Geneva, Switzerland, June 2025.

[ZHZ+18] Hao Zhou, Minlie Huang, Tianyang Zhang, Xiaoyan Zhu, and Bing Liu. Emotional chatting machine: Emotional conversation generation with internal and external memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

Appendix A

Generative AI Tools Usage

Generative AI tools were used as supplementary resources during the development and documentation of this thesis. Their use was limited to supporting literature exploration, improving the clarity of academic writing, assisting with LaTeX formatting, and providing guidance during technical problem-solving. All generated suggestions were reviewed before being incorporated into the thesis.

An overview of the generative AI tools used is presented in Table A.1.

AI Tool	Use Case	Used in	Remarks
ChatGPT 5.6 Sol / 6 Astra	Literature exploration and identification of relevant scientific publications	Chapter 2	Suggested references were independently verified
	Improvement of academic phrasing, grammar, and text clarity	Chapters 1–5	Suggestions were reviewed and adapted
	Assistance with LaTeX syntax, formatting, and document structure	Entire thesis	Formatting suggestions were manually checked
	Technical guidance and debugging suggestions	Implementation chapters	Suggestions were evaluated and tested before use

Table A.1: Overview of generative AI usage

A.1 ChatGPT Details

ChatGPT (versions 5.6 Sol and 6 Astra, developed by OpenAI) was used as a supplementary assistant throughout selected stages of the thesis. Its contributions are described below.

A.1.1 Literature Exploration

Purpose: To support the identification of relevant scientific publications and research directions related to the topics investigated in this thesis.

Method: ChatGPT was prompted to suggest academic publications, relevant research topics, and potential directions for organizing the state-of-the-art review in Chapter 2.

Verification: Suggested publications were checked against academic sources to verify their existence, relevance, and bibliographic information. The original publications were consulted before being referenced.

Implementation: The verified references and research directions were used as supporting resources when developing the literature review.

A.1.2 Academic Writing Assistance

Purpose: To improve the readability, grammatical correctness, and academic style of selected passages.

Method: Selected paragraphs were provided to ChatGPT with requests to improve sentence structure, clarity, or academic phrasing while preserving their original meaning.

Verification: The suggested revisions were reviewed to ensure that the technical meaning and intended interpretation remained unchanged.

Implementation: Selected linguistic improvements were incorporated into the thesis after manual revision.

A.1.3 LaTeX Formatting Assistance

Purpose: To assist with document formatting and the resolution of LaTeX-related issues.

Method: ChatGPT was consulted for guidance on LaTeX commands, table formatting, document structure, and compilation errors.

Verification: Suggested solutions were checked by compiling the document and inspecting the resulting output.

Implementation: Validated formatting solutions were applied where appropriate.

A.1.4 Technical Guidance and Debugging

Purpose: To obtain supplementary explanations and troubleshooting suggestions during the implementation process.

Method: ChatGPT was occasionally consulted to clarify technical concepts, explain error messages, and suggest possible approaches to resolving implementation difficulties.

Verification: Suggestions were assessed against the project requirements and, where applicable, tested in the development environment.

Implementation: Only solutions considered appropriate and successfully validated were incorporated into the implementation.

Generative AI was used exclusively as a supporting tool. The selection of research methods, system design decisions, implementation validation, interpretation of experimental results, and final conclusions remained the responsibility of the author.